

Master's Programme in Security and Cloud Computing

A Graph-Based Provenance Model for Human-Agent Software Workflows

Artem Sokhin

Copyright © 2026 Artem Sokhin

This thesis is submitted for evaluation to both Aalto University and the Technical University of Denmark (DTU), as part of the SECCLO Erasmus Mundus double-degree programme.

Author Artem Sokhin		
Title A Graph-Based Provenance Model for Human-Agent Software Workflows		
Degree programme Security and Cloud Computing		
Major Security and Cloud Computing		
Thesis supervisor and advisor Associate Professor Bjørn Sand Jensen (DTU), Assistant Professor Fabian Fagerholm (Aalto University)		
Collaborative partner Technical University of Denmark (DTU)		
Date 28 July 2026	Number of pages 81+18	Language English

Abstract

Coding agents complete software tasks autonomously over many steps, yet the record they leave behind is incomplete: a finished session preserves its outcome, while the process that produced it survives only as an unstructured event log that can be reread but not queried. This thesis introduces the **Interaction Graph**, a typed provenance model for human-agent software workflows lifted from a raw OpenTelemetry span stream, extending the W3C PROV standard for language-model agent runs. It is evaluated under a Design Science methodology through three research questions, each against objectives fixed in advance.

The first question concerns representation. The lift reconstructs the version-controlled history and classifies each action at a level of agreement with human annotators sufficient to certify the graph as a reliable substrate. The representation strictly dominates the flat evaluation log in query expressiveness, answering every question the flat log admits plus a further set answerable only after the lift, one a language model recovers from the flat log alone only in part.

The second question concerns diagnosis. Recurring struggle motifs are expressed as deterministic subgraph queries, validated against a human-annotated gold standard in which a fired detector is almost never a false alarm. Motifs' association with failure stays statistically indistinguishable from chance at this sample size. The queries nonetheless execute at a fraction of the cost of a per-trace language-model judge.

The third question concerns memory. A null-tolerant feasibility pilot integrates the graph as a cross-session memory a coding agent queries during a task. Task resolution is unchanged, as expected of a process-oriented memory, while tool-call repetition is measurably reduced, though one configuration also shows a regression on a failure-predicting motif, visible only in the graph's typed layers.

A single typed representation, lifted once from the event stream, thus serves as a query substrate, a diagnostic surface and a cross-session memory without modification. This unification constitutes the central contribution of the thesis.

Keywords provenance, interaction graph, software-engineering agents, PROV, observability, large language models

Acknowledgements

I am deeply grateful to the SECCLO program and its staff for the remarkable opportunity to pursue this double degree at Aalto University and the Technical University of Denmark. I also extend my sincere thanks to my supervisors for their invaluable guidance and support throughout this work.

To my SECCLO cohort: thank you for the incredible times we've shared. You made the past two years truly unforgettable, and I look forward to the memories we have yet to make. A special thank you goes to my best buddy, Amir.

To my friends, thank you for encouraging me to pursue this degree and for the constant support along the way.

To my girlfriend, thank you for your unwavering support.

To my family, thank you for always believing in me.



Co-funded by
the European Union

Contents

Abstract	3
Acknowledgements	4
Contents	5
1 Introduction	7
1.1 Research questions	8
1.2 Thesis outline	9
2 Background and related work	10
2.1 Foundations: provenance and event-log modeling	10
2.2 Provenance models for AI agents	12
2.3 Agent failure attribution and trace analysis	13
2.4 The bridge: the Interaction Graph	15
3 Research approach	16
3.1 Methodology	16
3.2 Research questions	17
4 The Interaction Graph	18
4.1 Ontology	18
4.2 Data model	21
4.3 Construction: the lifting pipeline	24
5 Evaluation design	33
5.1 The coding-agent setting and reliability vocabulary	33
5.2 Representations, the lifting function and the corpus	34
5.3 Measurement protocol	34
5.4 Summary of measurements	41
6 Representation evaluation	42
6.1 Construction coverage	42
6.2 Lifting fidelity	42
6.3 Query expressiveness	43
6.4 Summary	50
7 Diagnostic application	51
7.1 Motif catalog and cross-walk	51
7.2 Prevalence	52
7.3 Validation	53
7.4 Diagnostic value	54
7.5 Explaining struggle: the causal-relation layer	55
7.6 Scalability under growing logs	57
7.7 Summary	58

8	Memory application	59
8.1	Background: memory for autonomous agents	59
8.2	Cross-attempt retrieval design	59
8.3	Feasibility pilot	60
8.4	Findings	61
8.5	Summary	64
9	Discussion	66
9.1	What the measurements establish	66
9.2	Limitations	67
9.3	Threats to validity	68
9.4	Struggle motifs as a process-quality target	70
10	Conclusion and future work	71
10.1	Conclusion	71
10.2	Future work	72
	Bibliography	74
A	RQ1 query-expressiveness reproducibility	82
A.1	Competency questions and their graph queries	82
A.2	Prompts	84
A.3	Model, provider and run configuration	86
B	RQ2 diagnostic-application reproducibility	87
B.1	Motif detectors	87
B.2	Nine-smell cross-walk, full rationale	88
B.3	Prevalence, per-motif detail	88
B.4	Human-gold annotation protocol	89
B.5	Per-motif confusion detail	89
B.6	Relation-layer derivation and edge inventory	90
B.7	The LLM-judge baseline: prompt, glosses and rendering	91
B.8	Model, provider, run and log configuration	92
C	RQ3 memory-application reproducibility	94
C.1	The three memory arms	94
C.2	Retrieval tools and glosses	94
C.3	Leave-one-out construction and guard enforcement	95
C.4	Model, provider and run configuration	95
C.5	Full process-metric results	96
C.6	Typed motif and signal deltas	96
C.7	Retrieval intensity and outcome	97
C.8	Outcome churn is run-to-run noise	97
C.9	Move-classifier reliability	99

1 Introduction

Agentic AI systems, autonomous agents that write, test and debug code through sequences of tool invocations, are increasingly deployed in software development [1]. Their capability has advanced quickly. On SWE-bench, a benchmark of real-world GitHub issues, the strongest models resolved under 2% of tasks when it was introduced [2], and autonomous resolution rates have risen steeply across successive model and agent generations [1]. Such gains are pushing established benchmarks toward saturation [3], and evaluation is shifting to longer-horizon autonomy. The length of task a frontier model can complete on its own, measured as a time horizon, has doubled approximately every seven months since 2019 [4].

Despite this progress, a basic limitation persists: each session begins with no record of how earlier sessions actually worked. Users re-specify the same context and constraints, from the layout of the repository and the commands that build and test it to the conventions the project follows and the approaches that have already been tried and failed. Organizations, meanwhile, accumulate execution logs and transcripts that remain unstructured and unexploited. Recurring failure patterns and effective solution strategies are, in effect, discarded at the session boundary.

Existing augmentation strategies do not close this gap. Retrieval over raw transcripts offers contextual similarity but imposes no semantic structure, so it cannot answer targeted questions about failure types or causal tool sequences. Skills libraries record intended behavior while leaving observed execution history unrecorded [5]. Memory mechanisms are increasingly built in. Coding harnesses persist notes and preferences across sessions and distill summaries of earlier ones [6, 7], and dedicated frameworks maintain long-term stores over past facts and messages [8, 9], including knowledge-graph stores [10, 11]. Knowledge to recall is recorded, but how the agent worked is not. A session leaves behind its result, the code and the commit in the git history, while what the agent did to get there stays unknown: what it tried, what failed and how it reached the outcome. That this episodic record is missing has been argued to be the key gap for long-term agents [12, 13]. A run can be reread but not queried, because the structure that links a result to the actions that produced it was never recorded. The limitation is sharpened by trace length, as reasoning over an ever-longer flat log costs more. Recent work on agent failure attribution reports that detection accuracy over such logs also degrades as they grow [14], and it explicitly calls for graph-based reasoning over agent interactions as a way forward [15].

The gap is therefore one of process, in how a result was produced. Whether it was correct is a separate question. This thesis addresses that gap by proposing the **Interaction Graph** (IG), a typed provenance graph of an agent’s run that records how evidence, actions and outcomes connect, for human-agent software workflows, a graph that functions both as a cross-session memory mechanism and as a reusable diagnostic artifact. The IG extends the World Wide Web Consortium (W3C) Provenance (PROV) model [16] with node types specific to Large Language Model (LLM) agent workflows (Intent, ToolAction, Artifact, Evidence, Outcome, Signal and MoveMotif), while

keeping a provenance base that remains standards-compatible. The work follows a Design Science research methodology [17]: the artifact itself is the contribution, evaluated against defined objectives of lifting fidelity and query expressiveness, whereas a single performance comparison would not suffice.

Relative to PROV-AGENT [18], the closest existing system, this thesis differs in three respects. PROV-AGENT captures agent activity at the instrumented Model Context Protocol boundary, so an action outside that boundary goes unrecorded. The IG instead reconstructs typed provenance from the raw OTel event stream, so it records actions no instrumented tool exposes. PROV-AGENT’s own query layer, an in-memory DataFrame, cannot follow the multi-hop causal traversal a failure diagnosis needs, a limit its authors state directly [19]. The IG’s typed subgraph queries support exactly that traversal. PROV-AGENT reports a use-case demonstration with no quantitative metric, where this thesis measures lifting fidelity against a human gold, formal query-expressiveness containment and a functional-equivalence reproduction rate.

1.1 Research questions

The thesis is organized around three research questions, one on the representation itself and two on applications of it.

RQ1: Representation. *Can agent execution traces be automatically lifted into a typed provenance graph that faithfully captures human-agent software workflows, and what queries does this representation support that flat execution logs and raw telemetry spans do not?* The question has two parts. The first asks whether the graph matches what the agent actually did, the operations it ran, the files it changed and how one step led to the next. The second asks whether the graph answers provenance and lineage queries that the raw event stream cannot express.

RQ2: Diagnostic application. *Can recurring struggle motifs, such as repeatedly editing and re-running a failing test, be expressed and detected as reusable, composable subgraph queries over the Interaction Graph, and what does this query-based mechanism offer over alternatives such as inspecting each trace with a language-model judge?* The comparison examines two practical dimensions: how the cost of detection scales as traces grow and whether a finding can be traced to the evidence behind it.

RQ3: Memory application. *Can the Interaction Graph serve as a structured cross-session memory source for a coding agent, and what does a feasibility pilot indicate about its effect on task success?* The aim is to show that the graph can be connected as a memory the agent can query, and to observe from the usage logs whether and when the agent actually uses it. Because this is a feasibility study, the contribution holds even if the graph does not raise task success. The pilot compares an agent given this memory with the same agent given none, leaving comparisons against other memory systems, whether flat-transcript or graph-based, to future work.

1.2 Thesis outline

The remainder of this thesis is organized as follows. Chapter 2 reviews the background and related work, situating the Interaction Graph among provenance models for AI agents and the failure-attribution literature. Chapter 3 sets out the research approach and fixes the research questions. Chapter 4 presents the artifact itself, the Interaction Graph ontology and its lifting pipeline. Chapter 5 defines the measurement protocol that the later chapters report against. Chapter 6 evaluates the representation, Chapter 7 demonstrates the diagnostic application and Chapter 8 the memory application. Chapter 9 discusses limitations and threats to validity, and Chapter 10 concludes and outlines future work.

2 Background and related work

The Interaction Graph bridges two established areas, neither novel on its own: provenance models for AI agents, and the post-hoc diagnosis of agent failures from execution traces. This chapter first develops the provenance, process-mining and event-stream foundations the graph inherits, then surveys each area and the gap it leaves before assembling the evidence that agent reliability degrades across multi-turn, cross-session work, which is the empirical motivation for the approach. It closes by positioning the Interaction Graph as the bridge between the two.

2.1 Foundations: provenance and event-log modeling

Provenance is the recorded history of how an artifact or data item was produced, the entities and activities that created it, the agents responsible for them and the derivation links that connect each step to the next [16]. It answers, for any result, where that result originated, which process generated it, what it was derived from and who was accountable, so the result can be traced to its sources and assessed for quality and reliability. The most widely adopted standard for expressing provenance is the **W3C PROV family of specifications**, which the Interaction Graph inherits. Its core is PROV-DM, a conceptual data model that defines the provenance concepts and the relations among them. PROV-O renders that model as an ontology in the Web Ontology Language (OWL), mapping its concepts to Resource Description Framework (RDF) triples so that provenance can be published and queried on the semantic web [20], and PROV-N provides a human-readable textual notation for recording provenance instances directly [21]. PROV organizes provenance around three primitives, an entity (a thing whose origin is described), an activity (a process that acts on entities over time) and an agent (a party responsible for an activity), related by a small set of relations such as *used*, *wasGeneratedBy*, *wasDerivedFrom* and *wasInformedBy* [16]. This vocabulary is deliberately generic. It carries no commitment to any domain, and its core types must be specialized before the model says anything concrete about a particular setting. PROV is therefore a provenance base that the Interaction Graph specializes into a representation of agent behavior, and it has a track record of being extended for new domains, including database provenance and deployed semantic-web pipelines [22, 23].

The second foundation is process mining, the discipline that reconstructs process knowledge from the event data that information systems record during execution. Process mining assumes an *event log*, a structured collection of cases, each a time-ordered trace of events, as its input. Such a log is rarely recorded directly. Fahland frames **event-log extraction as an act of modeling**, in which the analyst must consciously choose a case identifier and an event classifier, decisions that determine which features of the raw data describe which behavior of which entity and that admit

many valid, divergent logs over the same data [24]. This framing puts the lifting step at the center of the Interaction Graph, because deciding what an event stream becomes (an intent, a tool action, an artifact) is the same kind of analyst-driven modeling choice. Fahland also notes that a flat single-case-identifier log cannot represent the one-to-many and many-to-many relations among co-evolving entities, and identifies graph-based representations as the remedy [24]. This limitation is acute for the present setting. A human-agent coding session contains many such co-evolving entities, the intents, tool actions, artifacts and test outcomes. The relations among them are one-to-many and many-to-many, since one intent drives many tool actions and a single file is edited by many actions across several attempts. A single-case-identifier log, the session for instance, would collapse these relations, so the graph form that the Interaction Graph adopts is motivated from within the process-mining literature itself.

Process mining and the Interaction Graph differ in what they produce. Process mining generalizes many recorded executions into a model of the runs a process can exhibit. The Interaction Graph makes no such generalization, recording one execution as it actually happened, together with the causal and derivation links between its specific intents, actions and artifacts, which makes it an instance-level provenance graph grounded in PROV-O. It therefore borrows process mining’s vocabulary and its view of extraction as an act of modeling, without performing the process discovery that builds a generalized model. Any such generalization instead happens at query time, when a motif query (in the diagnostic application) or the agent (in the memory application) searches the accumulated graph.

The raw material that lifting consumes is supplied by a recent event-stream standards layer for instrumenting language-model applications. The **OpenTelemetry (OTel) generative artificial intelligence (GenAI) semantic conventions** extend the OTel observability standard with attribute and event definitions for model invocations, tool calls and agent steps, giving a vendor-neutral schema for the telemetry an agent emits [25]. **OpenInference**, a complementary set of conventions and instrumentation libraries layered on OTel, captures the surrounding application context of a language-model call, including retrieval, tool use and external calls [26]. Both define an event stream, a time-ordered sequence of spans and attributes that records that an agent called a tool or produced an output. The stream is not wholly flat, since each span carries a parent reference and a trace identifier, so the spans form a call tree that records which span ran within which. This structure captures execution nesting and ordering, and it does not express the typed provenance relations a graph needs, which artifact an action generated, what an output was derived from and which earlier step informed a later one. Both conventions omit the ontological types that a provenance graph of an agent interaction requires, so an edited file is not an entity in the stream and two actions that change the same file remain unconnected sibling spans.

These foundations together leave a clear gap. PROV supplies a generic graph vocabulary that must be specialized before it describes agent work. Process mining supplies an extraction discipline whose output is a generalized model. The OTel GenAI and OpenInference conventions supply an untyped event stream whose structure is execution nesting. All three stop short of bridging that stream into a typed agent-interaction graph in which causality and the roles of intent, tool action, artifact,

evidence, outcome, signal and move motif are first-class and queryable. The remainder of this chapter surveys the work that begins to close this gap.

2.2 Provenance models for AI agents

The closest prior work is *PROV-AGENT*, which treats an agent’s run as a provenance record. It extends the authors’ existing workflow-provenance framework with a W3C PROV model for agents and captures agent activity at the Model Context Protocol (MCP) boundary. It models each LLM invocation and each MCP tool call as a typed activity linked to the data it consumed and produced [18]. The Interaction Graph reuses this structural move, recording agent calls as PROV activities. PROV-AGENT validates that W3C PROV is a sound substrate for agent provenance. It is limited relative to the present work in three respects. A human operator queries the graph to debug a workflow. The agent itself never consults it, so the record is write-only. PROV-AGENT evaluates the system as a use-case demonstration on a cross-facility additive-manufacturing workflow, reporting example queries but no quantitative metric for recall, latency or query performance. PROV-AGENT captures at the MCP boundary, so it records only the tool calls exposed as instrumented MCP tools. Local edits and shell steps outside that boundary go unrecorded. The instrumentation itself supplies the structure. Its provenance is therefore structured at the point of capture, since only instrumented tasks are recorded [18]. The Interaction Graph instead reconstructs this structure from the OTel stream, so it records actions no instrumented tool exposes.

The same group also built the query side: an LLM agent that turns a natural-language question into a structured query over already-captured workflow provenance [19]. This thesis reuses two parts of that work. The first is its taxonomy of query classes, which divides queries into targeted lookups that fetch a single fact and multi-hop traversals that follow a chain of links across the graph. RQ1 reuses this taxonomy to classify its own competency questions. The second is its functional-equivalence criterion, under which two answers match when they carry the same information regardless of form [19]. RQ1 adopts this criterion to score whether the flat stream reproduces each answer the graph gives.

The same work also points to the gap this thesis fills. Its query layer uses an in-memory DataFrame, a flat table that cannot follow long chains of links, and section 5.4 says so directly: deep multi-hop traversal for causal analysis exceeds what a DataFrame can represent, and the problem is left unsolved [19]. That deferred capability, structural subgraph querying, is exactly what RQ2 depends on.

A contrasting research direction treats the agent event stream as flat telemetry. Telemetry-aware tooling for AI-assisted development surfaces token counts, latency, evaluation scores and trace excerpts inside the integrated development environment (IDE), stores them in a queryable telemetry server reached at the MCP boundary and proposes monitor agents that read the stream to flag recurring issues [27]. The example here is the Opik MCP server, one of the LLM-observability platforms in wide use. This class confirms a premise that underpins the Interaction Graph: an agent

Table 2.1: Placement of the Interaction Graph among prior agent-provenance systems, by trace source, representation and trust function.

System	Trace source	Representation	Trust function	Multi-hop
PROV-AGENT [18]	MCP boundary, instrumented	typed PROV graph, write-only	human debugging	deferred
Interactive workflow provenance [19]	captured provenance	DataFrame over provenance	natural-language query	no
Telemetry tooling [27]	OTel span stream	flat log with attribute filters	monitoring	no
Interaction Graph (this work)	reconstructed from OTel	typed queryable graph	diagnosis and memory	yes

run produces a rich trace stream over OTel. What this tooling leaves out is where the present work begins. Its store is just a log of metrics and traces, searched with attribute filters. It has no typed nodes, no causal edges and no ontology, so it cannot answer a structural question, such as which intent a regression came from [27].

Although these works differ in where they capture, how they query and what they are built for, they share one structural limitation. PROV-AGENT extends W3C PROV, but it captures provenance that is already structured at the source. The interactive-workflow line queries that provenance, but it defers multi-hop structural traversal. Telemetry-aware tooling keeps the raw OTel stream, but it stops at filterable trace logs with no typed structure. All three stop short of lifting a typed, queryable agent-interaction graph from the raw event stream. That gap motivates the Interaction Graph, and the failure-attribution literature surveyed next sharpens it from the diagnostic side.

The systems placed so far invite a summary comparison. Each captures the provenance of an agent run in a different form, and they separate along one central tension, chronological logging against semantic provenance, the same distinction the representation evaluation of this thesis measures as the flat log against the lifted graph. Placing them by trace source, representation and trust function locates each in a narrow region of that space. Table 2.1 states the placement for the systems above. The Interaction Graph occupies the semantic-provenance end, and by serving both a diagnostic and a memory trust function over a reconstructed trace, it spans a wider region than the prior systems, which instrument theirs.

2.3 Agent failure attribution and trace analysis

The second cluster the Interaction Graph bridges concerns how the failures of an agent execution are diagnosed after the fact. Zhang et al. name and formalize **automated failure attribution**, the problem of reading a failed execution log and identifying both the responsible agent (the “who”) and the decisive error step (the “when”), and release the Who&When dataset of 184 hand-annotated attribution tasks drawn from 127 multi-agent systems [14]. The result that motivates this thesis is negative. Across three baseline methods (All-at-Once, Step-by-Step and Binary Search), each

an instance of LLM judging over a flat transcript, the best configuration reaches only 53.5% agent-level accuracy and around 14.2% step-level accuracy, with some method and metric combinations falling below random and reasoning models such as o1 and DeepSeek R1 offering no consistent improvement [14]. The decisive observation is that step-level accuracy collapses as the log grows, converging toward 0% on the longest traces, because an LLM judge cannot localize the decisive step inside a long linear transcript [14].

TraceElephant rebuilds this benchmark under *full execution observability*, recording the complete input context, prompts and tool interactions of every step, where the prior dataset captured only output traces [15]. Its 380-trace benchmark, which includes a single-agent SWE-bench scaffold alongside multi-agent web and question-answering systems, shows that full traces improve attribution accuracy by up to roughly 76% relative at the step level and 22% at the agent level over the output-only counterpart, with a replayable environment adding a further 10% at step level [15]. Trace completeness is therefore a precondition for attribution to work at all, and the headline figures supply the empirical justification for the claim that flat-log failure detection degrades as the log grows. The same paper floats as unaddressed future work “graph-based reasoning over agent interaction networks” and the reconstruction of control-flow structure from execution traces, gesturing at a typed graph substrate it captures the fields for but never builds [15].

Where the attribution benchmarks localize blame, a parallel line of work catalogs the failures themselves. Zhang et al. distill 60,949 coding dialogues into a taxonomy of nine process-level **interaction smells**, recurring pathologies of the interaction process that final-output benchmarks miss, including Code Rollback (reverting to a previously resolved erroneous state) and Partial Functionality Breakdown (a new feature regressing a working one) [28]. The taxonomy is built by manual open card sorting, in which human annotators sort real dialogues into smell categories. Prevalence is then estimated by an automated LLM evaluation, an extended WildBench user simulator paired with an LLM scoring oracle. Both mechanisms operate on flat text with no typed structural representation [28]. This catalog supplies the motif vocabulary the diagnostic application of the Interaction Graph adopts, so the contribution is claimed at the detection layer, taking the taxonomy itself as given.

A parallel taxonomy generalizes beyond coding dialogues. Cemri et al. study why multi-agent language-model systems fail and distill the observed breakdowns into MAST [29], a canonical taxonomy of agent failure modes spanning specification, inter-agent misalignment and task verification that, like Who&When and the interaction-smell catalog, is derived from and read over flat execution transcripts, with no typed structural substrate over which a failure mode can be expressed as a query.

Taken together, this cluster shares one diagnostic substrate. Detection is a flat-text LLM-as-judge over transcripts, with no typed structural representation over which a failure pattern can be expressed as a query, no reach across sessions and no causal grounding back to the tool actions and test outcomes that produced the failure. The two attribution benchmarks make the length-fragility of that substrate concrete, and one of them names the graph-based approach as unaddressed future work [14, 15]. A fair head-to-head comparison is unavailable, since these systems operate over multi-agent

web and question-answering tasks while this thesis targets a single-agent coding workflow on SWE-bench. The Interaction Graph therefore relates to this cluster as a mechanism the cluster calls for, a typed provenance graph used as a structural query substrate for failure-motif diagnosis, and the relation is one of complementarity.

These failures concentrate in exactly the settings where coding-agent benchmarks are weakest. SWE-bench frames each task as a single autonomous attempt against a fixed issue [2], and a line of follow-on benchmarks reports one finding in common, that agent performance falls sharply once the task becomes multi-turn, iterative or cross-session. FeatureBench moves the target to end-to-end feature development and finds that an agent scoring well on SWE-bench resolves only a small fraction of feature-level tasks [30]. CodeFlowBench evaluates iterative implementation over dependency chains and reports a sharp drop from single-turn to multi-turn performance [31]. MT-Sec rewrites single-turn tasks as three-turn conversations and attributes a correctness loss to evolving requirements while controlling for context length [32]. LoCoBench-Agent finds cross-session memory retention to be the universally weak capability and concludes that external memory architectures are needed [33]. This last result motivates the cross-session memory application of RQ3, and the broader degradation across multi-turn work motivates the diagnostic application of RQ2. These benchmarks frame the problem the thesis addresses. Its evaluation uses SWE-bench traces, where each session is a long multi-step execution and multiple same-repository sessions accumulate into a cross-session record, the setting Chapter 5 defines. The Interaction Graph addresses both applications as the bridge developed next.

2.4 The bridge: the Interaction Graph

The Interaction Graph sits at the intersection of the two clusters and addresses the gap they leave. It is a typed provenance graph that extends W3C PROV, the contribution of the first cluster, and it doubles as a structural query substrate for failure-motif diagnosis, the need of the second cluster, and as a cross-session memory source.

The contribution is this combination, and it is claimed as four points. The first is lifting a typed provenance graph from a raw OTel event stream. The second is a set of prescriptive node types that serve as a query substrate. The third is cross-session accumulation across attempts. The fourth is the human-agent software-engineering instantiation.

The novelty is the unification. Adding agent types to PROV and diagnosing agent failures both appear in the literature surveyed above, but no prior system combines them into one artifact, and that combination supplies the structural, cross-session querying the provenance-query and failure-attribution literatures leave unsolved or name as future work [19, 15]. The chapters that follow evaluate the artifact in each of these three roles.

3 Research approach

This chapter sets out the research approach. It states the methodology the thesis follows and fixes the research questions, decomposing each into the objectives that the later evaluation operationalizes.

3.1 Methodology

This thesis follows **Design Science Research (DSR)** [17], the paradigm in which knowledge is produced by building a purposeful artifact that addresses a class of problems and then evaluating it. The artifact here is the Interaction Graph ontology together with its lifting pipeline, and it is the contribution of the thesis. Evaluation in DSR assesses the utility, quality and efficacy of the artifact against objectives stated in advance [34], so the governing question is whether the artifact meets those objectives, a standard distinct from a statistically significant improvement over a baseline.

The evaluation follows the *Technical Risk and Efficacy* strategy of the Framework for Evaluation in Design Science [35], which positions evaluation episodes on two axes, their functional purpose (formative or summative) and their paradigm (artificial or naturalistic). A formative episode produces findings used to improve the artifact while it is still under development, feeding back into the design, whereas a summative episode judges the finished artifact against the objectives fixed in advance. The strategy favors artificial, controlled evaluation that begins formative and moves toward summative, and its aim is to establish efficacy, meaning that the observed benefit can be attributed to the artifact itself once confounding factors are excluded. In this thesis the formative episodes are the iterations of the lifting pipeline during development, in which evaluation findings fed back into the operation classifier and the artifact-recovery stage, and the summative episodes are the final measurements reported in the evaluation chapters against the yardstick fixed in Chapter 5.

This strategy fits the present work because at the proof-of-concept maturity of the artifact the dominant risk is technical, namely whether the construct works and meets its objectives. The size of any effect in real use is a separate question. Establishing such an effect would call for a powered hypothesis test, a comparison whose sample is sized in advance, through a power analysis, so that a real effect of the expected magnitude would be detected with high probability. This thesis defers that kind of study in favor of the demonstration and technical-experiment methods that design science sanctions for evaluation [36], which assess utility against stated objectives. Deferring it is also a practical matter, because adequately powering such a comparison would require running the agent over far more instances, repetitions and models than the computational resources available for this thesis allow. The choice is disclosed as a deliberate scope decision. The evaluation establishes technical efficacy under controlled conditions and leaves the assessment of usefulness in a naturalistic setting with real users, where a powered comparison would belong, to future work.

Within this frame the guiding principle is intrinsic measurement, quantified

wherever it is feasible. Every property asserted of the artifact is operationalized as a measurement with a reproducible artifact, so that each later claim is grounded in a defined quantity. Where a property is genuinely not measurable, for example a query that cannot be answered at all because the data it needs is absent from the event stream, the limitation is stated openly and the property is demonstrated structurally instead, by showing what the representation does and does not support.

3.2 Research questions

The three research questions stated in Section 1.1 are taken here as the objects of measurement, each decomposed into the objectives the measurement protocol operationalizes.

RQ1: Representation. This question decomposes into a faithfulness objective, addressed by construction coverage and lifting fidelity, and a query-expressiveness objective, addressed by a competency-question evaluation.

RQ2: Diagnostic application. This question asks what a query-based mechanism for detecting struggle motifs offers over per-trace LLM-as-judge detection, and is addressed by motif prevalence, detector validation and a comparison of the two on how detection cost scales and on evidence traceability.

RQ3: Memory application. This question asks whether the Interaction Graph can serve as a structured cross-session memory source for a coding agent, and is addressed by a null-tolerant feasibility pilot against a no-memory baseline.

The artifact these questions evaluate, the Interaction Graph ontology and its lifting pipeline, is presented in Chapter 4. The measurement protocol that operationalizes the objectives above is defined in Chapter 5, and the evaluation chapters report their results against it.

4 The Interaction Graph

This chapter presents the designed artifact, first the ontology and data model, then the lifting pipeline that constructs a graph from an event stream. The measurement protocol and the evaluation of the artifact follow in the chapters that follow.

4.1 Ontology

The “Interaction Graph” introduces eight node types. Seven are specific to agent workflows and form its primary contribution: *Intent*, *ToolAction*, *Artifact*, *Evidence*, *Outcome*, *Signal* and *MoveMotif*. Each names something an LLM agent run produces that standard provenance models cannot express. The *Agent* node is a generic actor, incorporated unchanged from the PROV ontology as a `prov:SoftwareAgent` [20]. The evaluated corpus is single-agent, so the actor requires no specialization. A multi-agent setting would likely require specializing the *Agent*, distinguishing the several actors and their roles. The *Intent* layer is similarly designed for human-agent workflows and is thinner in the autonomous corpus, a scope limitation discussed in Chapter 9.

These seven nodes are divided into two groups based on how they are generated. *Intent*, *ToolAction*, *Artifact*, *Evidence* and *Outcome* are lifted directly from the event logs of the agent’s run. *Signal* and *MoveMotif* are created in the second phase from the finished sequence analysis. A *Signal* evaluates the sequence of actions to highlight behavioral struggles, such as a repetitive edit-test-fail loop. A *MoveMotif* evaluates the sequence of reasoning moves to highlight similar struggles in the agent’s thought process, such as rapidly switching between writing fixes and testing. Therefore, a *MoveMotif* is the cognitive equivalent of a *Signal* and is only generated if the reasoning classifier is used.

The graph is compatible with the PROV standard, because each node type is a subclass of a PROV class, and each relation is a PROV property or a subproperty of one. Compatibility matters for two reasons. First, any tool that reads PROV can read, query and visualize an IG session without knowing the IG vocabulary, because every IG type already answers as its PROV superclass (Listing 4.1). Second, the design stays honest about its novelty, since the IG is precisely what these declarations contribute beyond PROV.

```
# An IG node carries both its ig: type and its PROV superclass
ev:9f3a  a ig:Evidence, prov:Entity ;
        prov:wasGeneratedBy act:trace1:span30 ;    # the test
        action
        ig:observes art:latex.py:v2 .

# A PROV-only query (no ig: terms) still recovers the generation
lineage
SELECT ?entity ?activity WHERE {
  ?entity  a prov:Entity    ; prov:wasGeneratedBy ?activity .
```

```

?activity a prov:Activity .
}

```

Listing 4.1: An IG node written with both its `ig:` type and its PROV superclass, so a query phrased only in PROV recovers the session lineage without any `ig:` term.

Table 4.1 outlines the eight node types and their corresponding PROV base classes. Table 4.2 details the eleven relationship edges. Five of these directly reuse existing PROV properties to avoid redundant vocabulary. The other six lack direct PROV equivalents and are introduced as `ig:` subproperties linked to the closest matching PROV property. Three relations are structural (`fulfills`, `indicates`, `observes`). The remaining three (`contradict`, `invalidate`, `trigger`) form a semantic layer. A rule pass runs after the lift and adds these edges by checking the ToolAction and Evidence nodes already in the graph. The rules use no language model and are fully deterministic, so the same graph always produces the same edges. Each edge records why something happened during the run, linking a result to the earlier action or result that explains it. PROV has no vocabulary for such links. `contradict` connects a failing test to an earlier passing one for the same target, `invalidate` connects a failing test to the edit it ran against, and a `trigger` edge connects a failed action to the action that immediately follows it. Composed, these three edges turn a behavioral motif into a traversable causal chain. The `contradict` edge coincides with the regression Signal, one edge for each detected regression, while `invalidate` and `trigger` are finer-grained edges derived from the same test, edit and Evidence structure and are therefore more numerous than any single Signal. The relation layer complements the motif signals without duplicating them, one a count over the session and the other a path a reader follows in one hop per edge. The chain then joins a failure to the edit it ran against and to the recovery it prompted, illustrated on one real session fragment in Figure 4.2. The diagnostic value of that path is evaluated in Chapter 7.

In addition to the classes and relations, the ontology formally declares one datatype property, `ig:contentHash` (domain *Artifact*, range `xsd:string`). It records the SHA-256 fingerprint of an Artifact's bytes, which reveals when two versions are exactly the same, for example when a file is reverted to a previous state, where a version number gives only the order of edits. The other node fields of Section 4.2 are also written into the exported graph as `ig:` literals, so they remain queryable and the export round-trips losslessly, but they carry no formal declaration. Only content identity is promoted to a declared ontology term, because it is a semantic primitive that PROV cannot express and that the query-expressiveness evaluation of Chapter 6 reads. Those fields are not added to the ontology, because that would only copy the data model into it.

Figure 4.1 presents the schema as a graph, the eight classes connected by the eleven typed relations of Table 4.2. The three semantic-relation edges (`contradict`, `invalidate`, `trigger`) are dashed, and the self-relations (`informed_by`, `derived_from`, `contradict`, `trigger` and the sub-intent `fulfills`) appear as loops.

The ontology is released as a standalone OWL file, `ig.ttl`, that `owl:imports`

Table 4.1: The Interaction Graph node types and the PROV-O class each specializes

Node type	PROV-O superclass	Role
Intent	prov:Entity	a goal: the session issue or a per-turn sub-goal
ToolAction	prov:Activity	one tool invocation, the spine, carrying the classified operation
Artifact	prov:Entity	a versioned file touched by an edit, with a content fingerprint
Evidence	prov:Entity	observational tool output (test result, trace, read, command output)
Outcome	prov:Entity	the session result: resolved , termination , resource accounting
Signal	prov:Entity	a rule-based struggle indicator over the action spine
MoveMotif	prov:Entity	a derived struggle indicator over the reasoning-move sequence, the cognitive counterpart of Signal
Agent	prov:SoftwareAgent	the actor running the tools

Table 4.2: The graph’s typed relations

Relation	PROV-O property	From $\rightarrow$ to, meaning
used	prov:used	ToolAction $\rightarrow$ Intent: the action serves the session goal
generated	prov:wasGeneratedBy	Artifact / Evidence / Outcome $\rightarrow$ ToolAction: produced by the action
derived_from	prov:wasDerivedFrom	Artifact $\rightarrow$ Artifact: the version lineage of a file
informed_by	prov:wasInformedBy	ToolAction $\rightarrow$ ToolAction: the action follows a prior one (the spine)
associated_with	prov:wasAssociatedWith	ToolAction $\rightarrow$ Agent: the agent ran the action
fulfills	prov:wasInfluencedBy	sub-Intent / Outcome $\rightarrow$ Intent: advances the issue
indicates	prov:wasInfluencedBy	Signal $\rightarrow$ ToolAction or MoveMotif $\rightarrow$ sub-Intent: the action or turn evidences the motif
observes	prov:used	Evidence $\rightarrow$ Artifact: a test result observes the artifact version it ran against
contradict	prov:wasInfluencedBy	Evidence $\rightarrow$ Evidence: a failing test contradicts the earlier passing one for the same target (regression)
invalidate	prov:wasInfluencedBy	Evidence $\rightarrow$ ToolAction: a failing test invalidates the edit it ran against
trigger	prov:wasInfluencedBy	ToolAction $\rightarrow$ ToolAction: the action is triggered by the prior failed one (recovery)

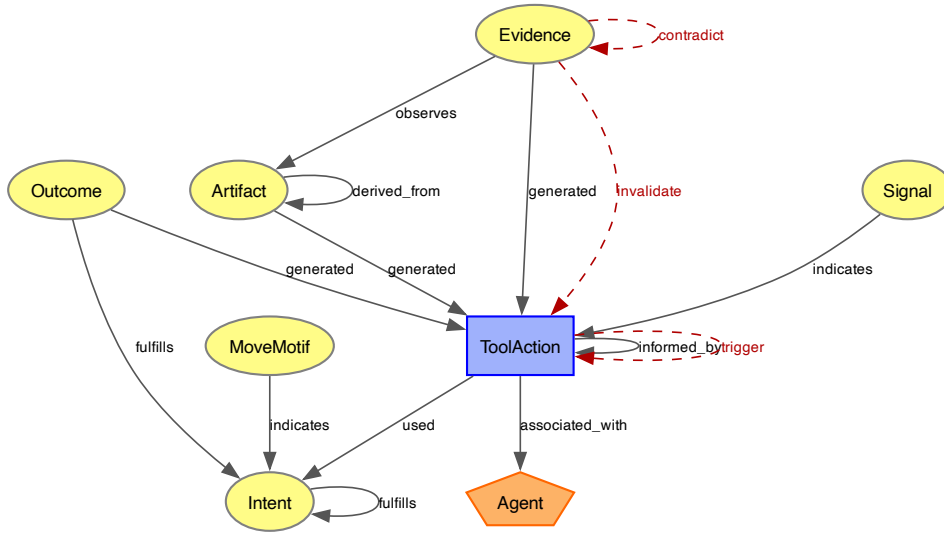


Figure 4.1: The Interaction Graph schema as a relation graph

PROV-O and declares the classes and properties above.¹ Each exported session also carries these definitions inside the file, so the graph explains its own types in PROV terms and can be read on its own, without the separate `ig.ttl`.

4.2 Data model

Each node type becomes a record with named fields that hold data. Each record carries only the fields its role needs, listed with their types in Table 4.3. An edge is the triple (source, target, type) over two node identifiers. The graph lives in memory as these records. The in-memory form is the single source of truth and the other forms described below are all derived from it.

Node identifiers are designed to make the lift reproducible. Each identifier is computed from stable information about the node, so re-running the lift on the same session produces the identical identifiers, and a node that would appear twice merges into one instead of being duplicated. How a node is identified depends on what makes it unique, which gives two schemes.

The first approach identifies a node based on its structural *position* within the execution trace, which is suitable for entities inherently defined by their temporal occurrence. For instance, a ToolAction corresponds to a single tool invocation captured as a distinct telemetry span. Its identifier concatenates the trace ID and the span ID (e.g., `act:f2ffa...:1ff9...`), guaranteeing uniqueness within the session. Similarly, an Artifact represents a file whose identifier combines its file path with a

¹Namespaced under <https://artemsokh.in/ns/interaction-graph#>.

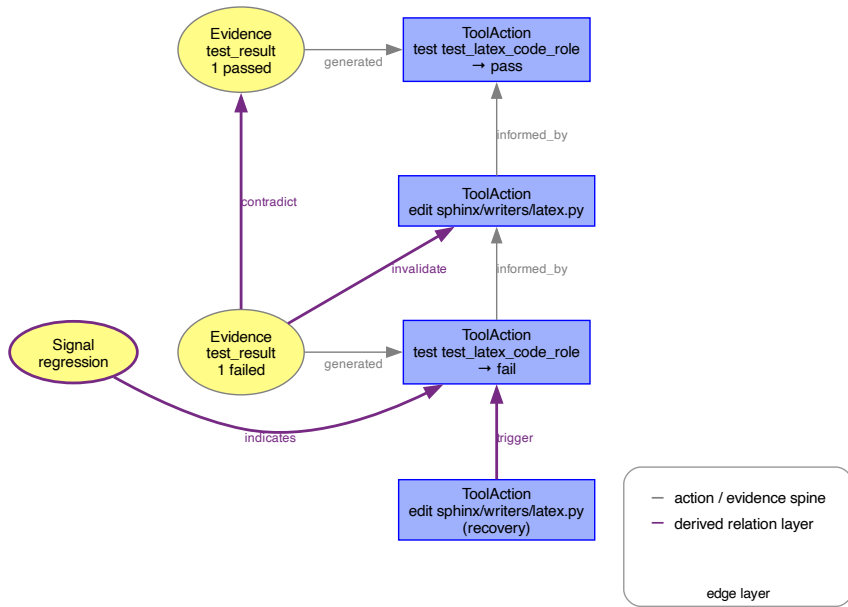


Figure 4.2: The regression motif as a causal chain, on one real session fragment (sphinx-doc__sphinx-10435). A test passes, an intervening edit is applied and the same test then fails. The derived relation layer (purple) makes the motif traversable. The failing test_result contradicts the earlier passing one and invalidates the edit it ran against, a trigger edge connects the failed test to the recovery edit that immediately follows it, and the regression Signal indicates the failing action. Gray edges are the action-and-evidence spine. Fill follows the PROV base class, a yellow oval for each `prov:Entity` and a blue box for each `ToolAction`.

Table 4.3: The fields each node type has in the typed data model, with their in-memory types. Only `content_hash` is promoted to a declared ontology datatype (`xsd:string`), while the remaining fields export as plain `ig:` literals.

Node type	Fields
Intent	<code>text</code> (str), <code>kind</code> (str: issue or sub_intent), <code>turn</code> (int), <code>move_type</code> (str)
ToolAction	<code>tool</code> (str), <code>op</code> (str), <code>canonical</code> (dict: runner, target, flags, effect hints), <code>status</code> (str), <code>turn</code> (int), <code>failed</code> (bool), <code>test_failed</code> (bool), <code>latency_s</code> (float), <code>input_tokens</code> (int), <code>output_tokens</code> (int)
Artifact	<code>path</code> (str), <code>version</code> (int), <code>repo</code> (str), <code>change</code> (str: create or modify), <code>content_hash</code> (str), <code>raw_cmd</code> (str)
Evidence	<code>kind</code> (str), <code>summary</code> (str), <code>status</code> (str), <code>passed</code> (int), <code>failed</code> (int)
Outcome	<code>resolved</code> (bool), <code>termination</code> (str), <code>turns</code> (int), <code>tool_calls</code> (int), <code>redundant_tool_calls</code> (int), <code>total_tokens</code> (int), <code>wall_time_s</code> (float)
Signal	<code>motif</code> (str), <code>severity</code> (int), <code>span_ids</code> (list)
MoveMotif	<code>motif</code> (str), <code>severity</code> (float), <code>turns</code> (list)
Agent	<code>name</code> (str)

sequential version number (e.g., `art:...:sphinx/writers/latex.py:v1`). By incrementing the version number with every edit, the system preserves an explicit provenance history while maintaining the file as a continuous logical entity. To achieve identification agnostic to the environment, absolute file paths are normalized to relative to the repository. This ensures that identical files maintain consistent identifiers independent of how their absolute paths are structured.

The second identifies a node by its *content*, and it is used when identical nodes should merge. An Evidence node is an observation, such as a test result or a file read, and its identifier is `ev:` followed by a short hash of its kind and summary, as in `ev:0aaeb6532bd904b8`. If the agent repeats an observation and receives the same output, both produce this same identifier and merge into one node. Positional identifiers would fail to capture this equivalence, making content-based identification essential. The other nodes instead use a readable role-and-position identifier (turn or ordinal), since these nodes are not content-addressed. Under this convention, the root issue Intent is labeled `intent:...:`, per-turn sub-goal is `intent:...:t1`, an Outcome is `outcome:...:e2`, a Signal is `sig:...:edit_test_fail_loop:0` and the Agent is `agent:inspect-react`.

The graph supports lossless serialization to two file formats: JSON and RDF/Turtle. JSON file format serves as the operational data store and is structured, written as one file per session. On load, each record is dispatched on its `type` field back to the matching dataclass, so a save-and-load round-trip returns identical typed nodes. RDF/Turtle acts as a standards-compliant export. In this representation, each node declares its specific `ig:` class and its PROV superclass via `rdf:type`. RDF format enables standard provenance querying by remaining fully PROV-compliant.

For richer queries, the corpus is also loaded into a Neo4j graph database, where

each node becomes a database node and its fields become properties.² Because this is a database load and not a file, it is a further representation separate from the two serializations above. The competency questions of the query-expressiveness evaluation (Chapter 6) are written as named Cypher queries that run over this loaded corpus. The same questions also run directly over the in-memory typed graph, and the two paths return identical rows, proven equal by test. The graph therefore has one canonical form, the in-memory records and three derived representations.

4.3 Construction: the lifting pipeline

For each session, the pipeline ingests an OpenTelemetry trace and outputs a typed graph. Because it targets a vendor-neutral specification, the same lifter applies to any agent that emits OpenInference spans, independent of the scaffold³ that produced them. OpenInference offers built-in auto-instrumentation for popular frameworks like LangChain, LlamaIndex and DSPy, eliminating the need for bespoke logging implementations [26]. The dataset analyzed in this thesis is constructed from the Inspect evaluation harness [37], where the embedded `react` agent [38] is explicitly instrumented to generate a span upon each tool execution.

Figure 4.3 shows the pipeline as a C4 component diagram with each component and its role, the external systems and the simplified data flow. The pipeline runs end to end in seven stages, each described in turn below. All but two stages are deterministic and reproducible. Move classification (stage three) and the MoveMotif portion of the sixth stage are post-hoc enrichment passes run by a language model after the deterministic lift completes, so they alone are not reproducible.

1. **Session skeleton**
2. **Operation classification**
3. **Move classification** (*post-hoc enrichment, LLM, run after the deterministic lift*)
4. **Artifact recovery**
5. **Evidence extraction**
6. **Signal and move-motif derivation** (*the move-motif portion is post-hoc LLM enrichment, run after the deterministic lift*)
7. **Outcome**

Table 4.4 summarizes the node types, edge types and fields each stage contributes.

²The load uses the Neo4j driver, writing each node type in a parameterized UNWIND batch and merging on its (`id`, `run_id`) key behind a uniqueness constraint, the recommended driver-side import pattern.

³We use *scaffold* for the framework that exposes an agent’s tools and drives its action loop, and *harness* for the evaluation framework that runs tasks and grades outcomes. Inspect provides both.

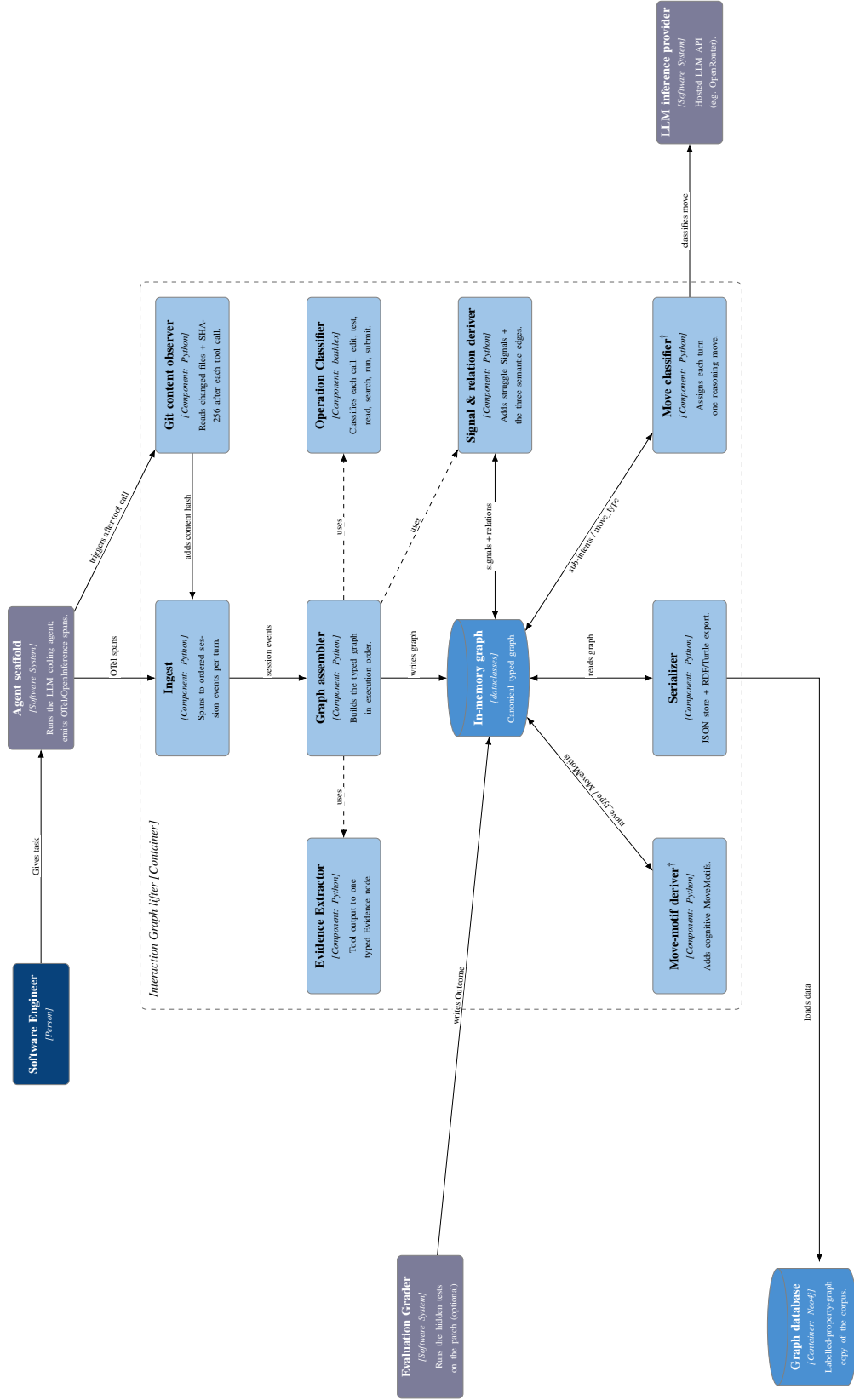


Figure 4.3: The lifting pipeline as a C4 component diagram. Navy marks the Person, gray-purple an external Software System, light blue a Component, and a blue cylinder a data store. The dashed boundary marks the *Interaction Graph lifter* container. A dashed arrow marks a “Uses” dependency; a solid arrow marks data flow, labeled with what moves along it. [†] marks the two post-hoc, LLM-driven components (Move classifier, Move-motif driver).

Table 4.4: What each construction stage adds to the graph. *Fields set* names attributes a stage writes onto existing nodes. Each node’s own fields are in Table 4.3. Stages marked † are post-hoc enrichment run by a language model after the deterministic lift. Every other stage is deterministic and reproducible.

Stage	Nodes added	Edges added	Fields set
Session skeleton	Intent, sub-Intent, Agent, ToolAction	used, associated_with, informed_by, fulfills	–
Operation classification	–	–	op, canonical
Move classification†	–	–	move_type
Artifact recovery	Artifact	generated, derived_from	–
Evidence extraction	Evidence	generated, observes	test_failed
Signal and move-motif†	Signal, MoveMotif	indicates, contradict, invalidate, trigger	–
Outcome	Outcome	generated, fulfills	–

Session skeleton. The pipeline begins by creating a session skeleton, establishing the foundation for later processing stages. The ingested trace consists of two span categories: LLM spans (capturing the assistant’s reasoning and tool requests) and tool spans (capturing the agent’s tool executions). Turn numbers are not present in the raw telemetry, so the pipeline reconstructs them by iterating through the spans and incrementing a counter for each LLM span. Tool spans inherit the turn number of their requesting LLM call, ensuring every action is assigned to a specific turn. This allows subsequent stages to accurately append sub-intents and reasoning moves.

The session’s Intent and its Agent nodes are created once, and each assistant turn that has reasoning becomes a per-turn sub-intent linked to the issue by `fulfills`. Every tool span becomes a ToolAction on the spine, associated with the Agent, used by the issue Intent and informed by the action before it. The graph holds one chronological chain through the session, because the actions form a single spine, each `informed_by` the action before it. Following the spine backward from a late failure therefore recovers the earlier actions that led to it, including the edit that produced it. Figure 4.4 shows this lift on a short fragment, where each user, assistant and tool message block of the log becomes a skeleton node.

Operation classification. We classify operations into six core types, *edit*, *test*, *read*, *search*, *run* and *submit*, a framework-agnostic abstraction of agent behavior over the tools a given scaffold exposes, at a granularity external standards do not fix. The nearest prior vocabularies are present at a different layer, modeling a human in an editor, from Ko et al.’s study of corrective and perfective maintenance tasks [39] to Mylyn’s fine-grained interface events, which have no notion of running a test or submitting a fix [40].

Agent scaffolds each expose their own set of tools, and they package the same work differently. SWE-agent has dedicated commands for navigating and searching the code, yet it runs tests through a plain shell command [41], while other agents expose a single shell tool that reads, searches and tests alike. The name of the tool does not always say what an action did, because different agents give the same work

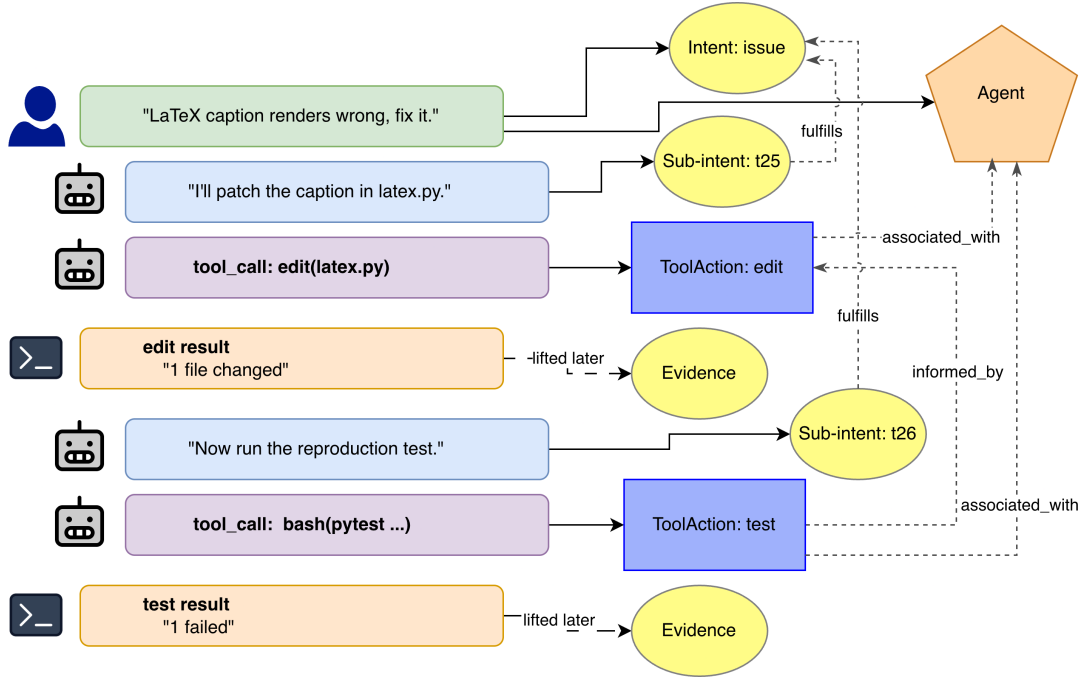


Figure 4.4: The Session skeleton stage. Each log message block lifts to a skeleton node. Evidence is lifted only at a later stage (marked *lifted later*).

different names. The classifier instead reads what the command does to classify type. A `bash` call that runs `pytest` is a *test*, and a `bash` call that runs `grep` is a *search*, `text_editor` call with command `str_replace` is an *edit* and one with `view` is a *read*. A tool without a matching rule is assigned the residual *run* until one is added. Because the label comes from the content of each action, the six operations form a substrate-independent layer, the same labels describing a run whether the agent drove a shell, a text editor or a built-in tool the classifier covers. One operation is recorded per command, so a command that bundles several substantive actions, such as an edit conjoined with a test, is labeled by the first matched command. It is a limit Chapter 9 treats as a threat to validity. Every label is assigned deterministically, which makes the lift reproducible. Its lifting fidelity against a two-annotator human gold is reported in Chapter 6.

The implementation also offers an optional language-model fallback that labels only the unknown commands, but it is disabled for the evaluated corpus. Enabling it would risk non-determinism and carry the reliability risks of a single-shot model judgment, so the corpus uses only the rules.

Independent of the task classification, the classifier also records each action's effect as four independent boolean hints in the Model Context Protocol tool-annotation vocabulary (read-only, destructive, idempotent, open-world) [42]. Both the operation and these effect hints are recorded on the ToolAction, the label as its `op` field and the hints, `runner` and `target` in its `canonical` record.

Move classification. Each per-turn sub-intent carries a classified reasoning *move*, the reasoning before the operation, recording *why* the agent acted on a turn while

the operation records *what* the tool did. The vocabulary adapts the eight action categories of Bouzenia and Pradel’s thought-action-result trajectory study [43] as its reasoning moves, namely *explore*, *locate*, *search*, *reproduce* (a test that confirms the bug, labeled *produce* in the lift), *generate* (a fix), *run_tests*, *refactor* and *explain*, and adds a residual *other* bucket for nine move labels in all. A move could not be identified deterministically from reasoning text, so it is assigned post-hoc by a single-turn language-model classifier reading the turn’s text and its operations. Its lifting fidelity against a human gold is reported in Chapter 6.

The operation and the move are kept as two layers because the operation underdetermines the move. The same operation can serve different purposes, an *edit* writing a fix, a reproduction test or a cleanup, and a *read* being broad exploration or the pinpointing of a specific culprit, while running a test or an untooled narration maps near-deterministically to one move. A single merged label would discard the reasoning the operation leaves open, so the move is kept as its own layer. They are joined by the turn, since the operation is a field of the ToolAction and the move a field of the turn’s sub-intent, and one turn carries a single move over all its actions (Figure 4.5). The move attaches to the turn because an assistant turn carries a single block of reasoning over the actions it issues, and the turn number already stamped on every node joins the two without a new edge.

Keeping them as two layers lets the move say what the operation cannot, and it lets the graph capture turns where the agent’s stated reasoning and its action disagree, a case the trajectory literature notes [43, 29]. Separating what an action is from why it was done also follows long-standing practice in software engineering, where defects are classified on several independent axes at once [44], and the structure of a code change is classified apart from its purpose because the structure alone does not reveal the purpose [45, 46].

Artifact recovery. Whenever an action changes a file, the lift records a new version of that file as an Artifact node, one node per file the action wrote. An action that writes both `a.py` and `b.py` therefore produces two Artifacts, one for each file. To find which files an action changed, the lift prefers the *git observer*, a component that watches the repository during the run and records which files each action changed together with their content. When the observer is present it reports the exact set of changed paths for any action, whatever its operation. Without it the lift falls back to reading the command itself, and only for an *edit*, picking out where the command writes, a shell redirect, `tee`, `sed -i`, `cp/mv` or `patch`. Each new version is linked to the file’s previous version by `derived_from`, so the graph holds the file’s full history. When a command writes a file in a way the parser does not recognize, the path is stored as `<unknown>` and the raw command is kept.

Every version also carries a content fingerprint, the SHA-256 hash of the file’s bytes, taken from the observer. Because the hash depends only on the content, two versions with the same hash are byte-for-byte identical, which makes it easy to ask, for example, whether a later edit reverted the file to an earlier state.

The observer is a read-only hook that wraps each of the agent’s tools. After every tool call it runs `git status` in the work tree, reads the files `git` reports changed and records their hashes on that action’s span. The same hook works for any agent that runs

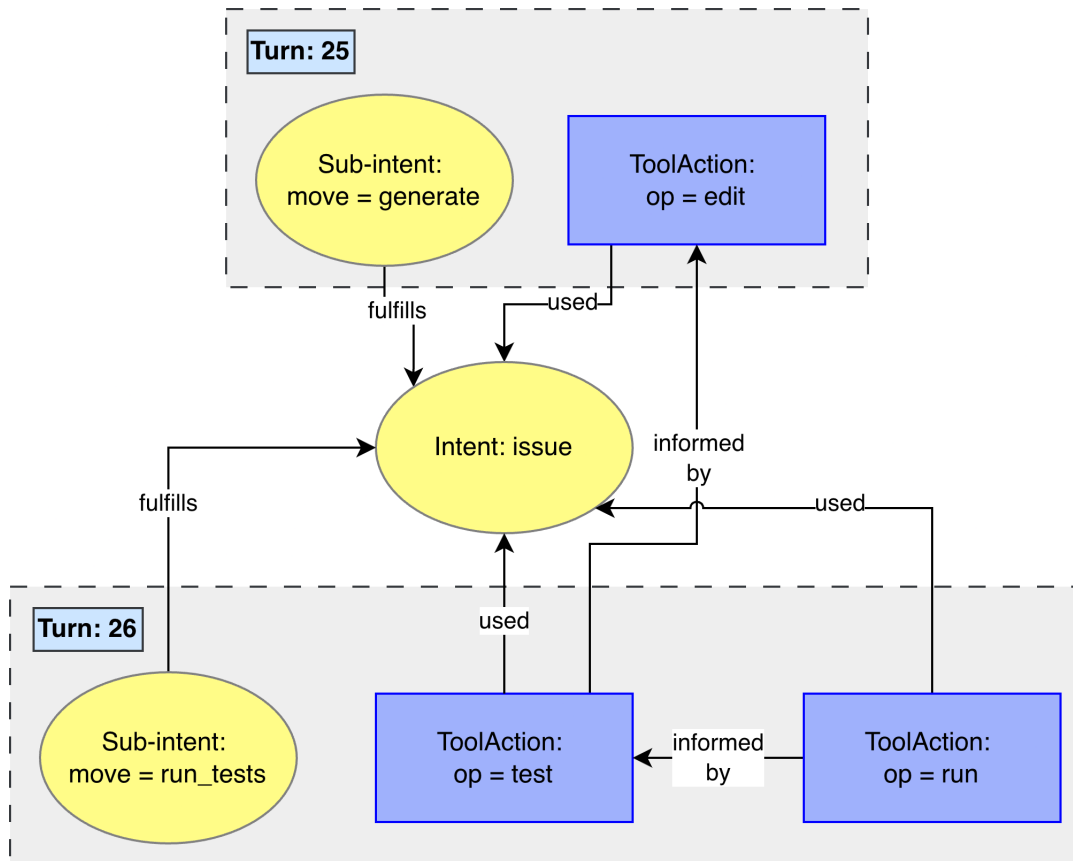


Figure 4.5: An operation and a move connect through the turn. Each dashed box is one turn, and within it the move-carrying sub-intent and the op-carrying ToolActions share no edge, so the turn is their only join. Each attaches to the issue Intent separately, the sub-intent by *fulfills* and each action by *used*. Turn 26 carries a single move over two actions, *test* and *run*.

in a git repository. This makes the observer a reusable way to recover file provenance, and leaves the command parser as a fallback.

Evidence extraction. Each action’s output becomes at most one Evidence node. The lifter labels it as a test result, a stack trace, a file read or command output. The parser covers pytest and unittest, the frameworks in the evaluated corpus, and a test under another framework is recorded as plain command output until its summary format is added. For a test failure, it sets the action’s `test_failed` flag. This flag is what the struggle motifs later react to. Each Evidence node is generated by its action. A test result also *observes* the current version of every file the session has touched, which ties the failure to the file state that produced it.

Signal and move-motif derivation. Once the spine is complete, a rule layer scans it for the six struggle motifs listed in the upper half of Table 4.5. Each detected motif becomes a Signal whose `severity` counts the spans evidencing it and which indicates the implicated actions. Signals are computed post-hoc over the finished spine, after execution completes. The same post-hoc pass also adds the semantic relation edges (`contradict`, `invalidate` and `trigger`, Table 4.2), which link a result to the earlier action or result that explains it.

The struggle detectors require a pattern to recur before firing. A single edit-then-fail cycle is ordinary iterative development, so `edit_test_fail_loop` requires two or more such cycles, and the other repetition motifs apply the same principle at their own counts. These thresholds are set by inspection to keep the detectors conservative, matching the precision-over-recall design the diagnostic evaluation confirms (Chapter 7).

Two further content-identity motifs, `no_net_change` and `cross_session_same_state`, are defined at diagnosis time over the Artifact content fingerprints. They are excluded from the per-session Signal layer and cataloged with the diagnostic evaluation (Chapter 7), where the motif catalog totals eight.

A parallel rule layer scans the reasoning-move sequence for the six cognitive motifs in the lower half of the same table. Each becomes a MoveMotif node, the cognitive-layer counterpart of a Signal, which indicates the implicated sub-intent turns. The MoveMotif layer is derived only after the reasoning-move enrichment has run, so it is absent from a graph lifted without the move classifier. Both layers are validated against a human gold and scored as RQ2 diagnostic detectors (Chapter 7). The MoveMotif layer also supplies the process-level lessons the memory application retrieves (Chapter 8) and grounds the process-quality argument of the discussion (Chapter 9). Because it is a rule layer over an LLM-classified move sequence, its detection inherits the move classifier’s reliability, a caveat the deterministic Signals lack.

Outcome. A single Outcome node closes the graph, recording the session result from the grader: the verdict in `resolved`, the `termination` reason and the resource accounting in `turns`, `tool_calls`, `redundant_tool_calls`, `total_tokens` and `wall_time_s`. It is generated by the last action and fulfills the session issue.

Figure 4.6 shows the complete lifted graph on one real session fragment, one instance of every node type, each badged with the stage that adds it.

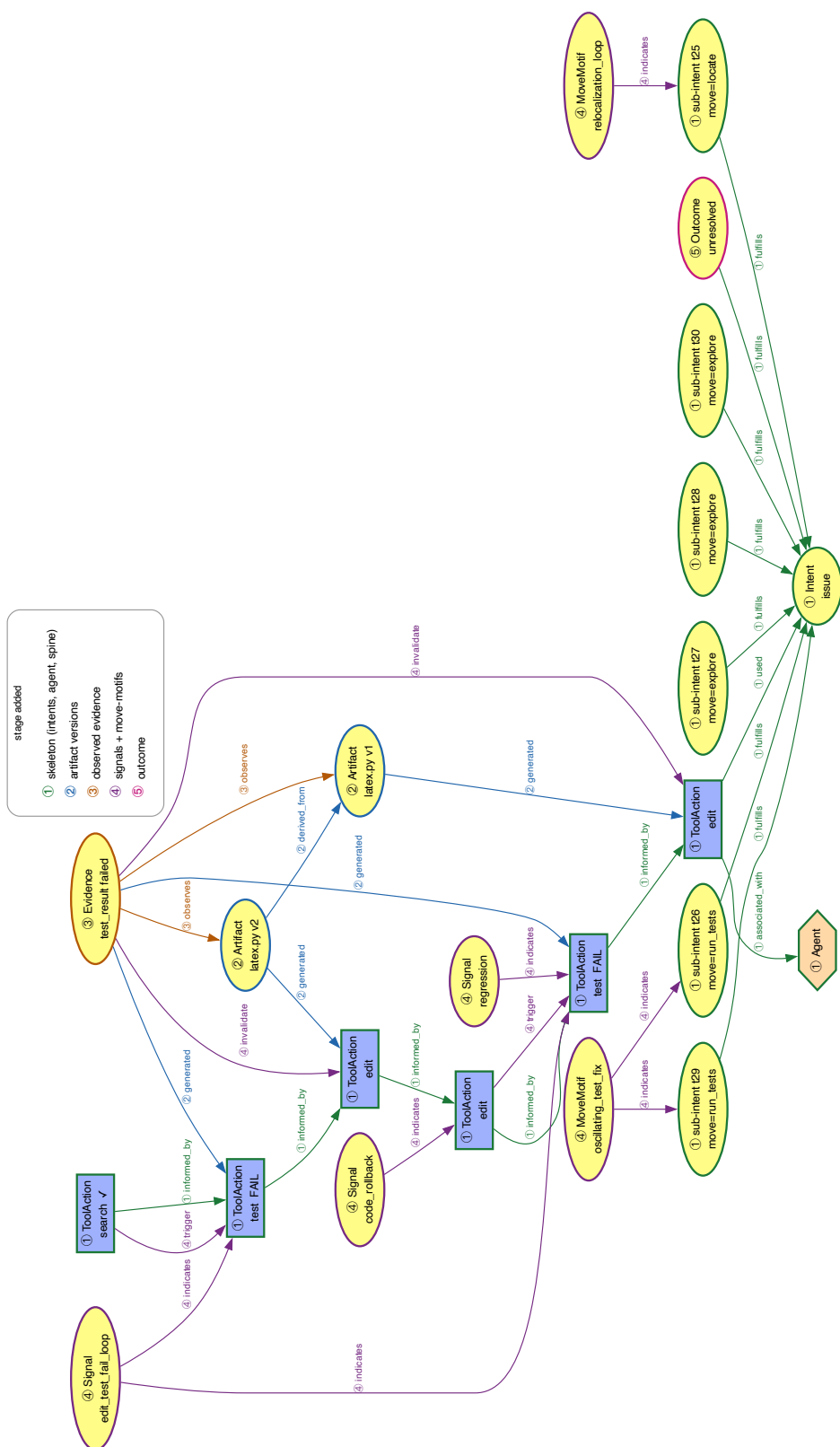


Table 4.5: The detected struggle motifs and when each fires

Motif	Fires when
<i>Signal, over the ToolAction spine</i>	
<code>edit_test_fail_loop</code>	an edit is followed by a failing test, for two or more cycles
<code>redundancy</code>	an action repeats work whose result is still available
<code>repetition</code>	three or more consecutive identical actions (same tool, operation and target)
<code>error_burst</code>	three or more consecutive failing actions
<code>regression</code>	a test target that passed, then failed after an intervening edit
<code>code_rollback</code>	an edit that reverts an earlier edit
<i>MoveMotif, over the reasoning-move sequence</i>	
<code>repeated_generation</code>	two or more consecutive <code>generate</code> moves
<code>missing_verify</code>	a fix is written but no test is run after it
<code>investigation_heavy</code>	at least 60% of turns are <code>explore</code> or <code>search</code>
<code>relocalization_loop</code>	the agent returns to locating the bug after it began fixing
<code>oscillating_test_fix</code>	three or more alternations between fixing and running tests
<code>skipped_reproduce</code>	a fix is written but no reproducing test is ever produced

5 Evaluation design

This chapter defines the measurement protocol. With the artifact of Chapter 4 in place, it sets the representations that are compared, the corpus and, for each research question, what is measured and how. These are decided here, before any results, so the evaluation chapters report their numbers against a fixed standard.

5.1 The coding-agent setting and reliability vocabulary

Before the protocol, this section fixes the task source and the reliability vocabulary the measurements below use. The task source is **SWE-bench**, which consists of real GitHub issues along with the hidden test suites that decide whether an agent’s patch resolves the issue. SWE-bench hands the agent one fixed problem description and grades a single self-directed solution [2]. Although each task is a single attempt, SWE-bench provides the two settings this evaluation requires. Each session is a long multi-step execution of edits and test runs, lifted into the graph the diagnostic application reads for failure motifs. Multiple sessions on the same repository accumulate into the cross-session record the memory application retrieves. The multi-turn and cross-session benchmarks surveyed in Section 2.3 motivate why these settings matter, and SWE-bench supplies realistic, reproducible traces.

A second strand supplies the evaluation’s reliability vocabulary and two caveats on benchmark realism. The framework of Rabanser et al. argues that a single mean success rate hides behavioral properties that matter for deployment, and decomposes reliability into four dimensions, consistency, robustness, predictability and safety [47]. This evaluation adopts the principle without instantiating the full framework: where it reports a success rate, it accompanies the mean with a paired comparison and a confidence interval. Two studies then bound how far such rates generalize. The benchmark-mutation study of Garg et al. rewrites SWE-bench issues into the short, telemetry-grounded style developers actually use with chat assistants and reports relative drops in success of 20–50% on the same underlying bugs [48]. From a complementary angle, a controlled study of the τ -bench user-simulator protocol finds that simulated users create an easier setting than real users, volunteering complete information and cooperating through agent errors [49, 50]. Both indicate that the resolution rates reported here are an optimistic bound. That bound applies to the resolution rates, while the lifting and querying claims use only the telemetry an agent emits and the structure the lift recovers, with one exception, the Outcome node’s resolved verdict, which is the benchmark grader score. The Interaction Graph extends none of these works.

5.2 Representations, the lifting function and the corpus

The query-expressiveness measurement in Section 5.3.1 compares two ways of representing the same agent run: a flat evaluation log and a typed graph. This contrast instantiates the distinction between chronological logging and semantic provenance, the second carrying the structure needed to answer how a result was produced. The evaluation log (Σ_{eval}) is the raw OpenTelemetry and OpenInference span stream, plus the per-sample transcript and the final grader score. It is the most detailed flat log of the run. The Interaction Graph (Σ_{IG}) is the same run lifted into a typed graph. The lifting function $L: \Sigma_{\text{span}} \cup \{\text{score}\} \rightarrow \Sigma_{\text{IG}}$ is the deterministic pipeline that produces the graph. It reads the span stream and the grader score. Every element of the graph comes from the span stream alone, except the Outcome node’s verdict, which comes from the score. The graph therefore contains exactly what these two inputs allow, and nothing more. The evaluation log already holds both inputs, plus the transcript, so it contains everything the graph does. Any query the graph can answer but the log cannot must therefore come from the graph’s typed structure. That is the difference the measurement isolates.

The frozen evaluation corpus (C) is built from 30 SWE-bench Verified instances [2] distributed evenly across three repositories (`django`, `sympy` and `sphinx`), each run for three epochs and lifted into one graph per session, giving 90 sessions in total ($|C| = 90$). The per-repository breakdown is reported with the memory pilot in Chapter 8. The corpus is content-addressed and version-controlled. A second corpus of the same 30 instances, generated by a stronger agent (Qwen3-235B) over three epochs, serves as a cross-model robustness check for the RQ2 diagnostic measurements. The lift and the formal classification are deterministic and reproduce exactly. The empirical layers, which call an LLM, reproduce in distribution, stabilized by a fixed decoding policy (temperature zero and a pinned model provider). Construction coverage and lifting fidelity are measured over the full corpus, while the empirical query-expressiveness layers (reproduction and groundedness) are reported over a repo-stratified sample of it, their per-question rates being stable across sessions. Each representation Σ is summarized by its primitive signature $P(\Sigma)$: the typed vocabulary of entities, relations and attributes it exposes, read straight from its schema. For the Interaction Graph, that schema is the ontology of Chapter 4. These signatures are what the query-expressiveness measurement in Section 5.3.1 uses.

5.3 Measurement protocol

5.3.1 RQ1: Representation

Construction coverage measures how completely the lift turns the raw event stream into typed nodes, the precondition for every later measurement. Three steps of the lift can leave a node incomplete, and each yields a coverage figure. The first is path recovery: an edit’s Artifact node needs the file path, which the lifter parses from the tool

command, and the unknown-path rate is the share of artifacts left with no recoverable path. An edit that names its file, such as a patch to `django/forms/fields.py`, yields a recovered path, while one whose path the command only assembles at run time is recorded as an unknown path. The second is command parsing: the rule-based classifier parses each ToolAction’s command into a structured operation record, and classifier coverage is the share of actions it parsed successfully, the remainder being degraded records. A `pytest` call is parsed as a test, a `grep` as a search and a patch as an edit. The third is content fingerprinting: each version-controlled Artifact version should carry a content fingerprint recovered from the git observer, and content-hash coverage is the share of those versions that do, measured over the version-controlled history. Writes the observer cannot fingerprint, those outside version control, are not counted here. They surface instead in the edit-action path recall reported with lifting fidelity. A final quantity, the distribution of operations across the corpus, is descriptive. It shows how the agents spent their actions, and a grossly implausible shape, such as zero edits or one operation accounting for nearly every action, would flag a broken classifier. A low unknown-path rate and a high classifier coverage together mean the typed graph represents the run with few gaps, which is a separate question from whether each label is correct, addressed by lifting fidelity next.

Lifting fidelity measures the correctness of the two rule-based lifter decisions against an independent gold standard, the classification of each tool action into an operation and the recovery of the file or files an edit writes. The protocol draws a blind, operation-stratified sample of tool actions, where the labeling sheet shows the raw command only and never the lifter’s prediction. Two independent annotators, the author and Seyedamirreza Hashemi, both at DTU, label the sample from the command alone, their disagreements are adjudicated into a single human gold standard, and their inter-annotator reliability is reported as raw agreement and Cohen’s κ with a percentile-bootstrap confidence interval [51]. The lifter is then scored against this gold, as per-operation precision, recall and F_1 , overall accuracy and macro- F_1 for the operation and a chance-corrected Cohen’s κ with a bootstrap confidence interval, with set-level precision and recall for the edited paths. This two-annotator gold supersedes an earlier single-rater design, in which one human spot-check validated a large-language-model labeler. That design was adopted on the evidence that such models annotate closed-class tasks at the level of crowd and trained annotators [52] and match human agreement even on open-ended preference judging [53], while two independent human annotators give a stronger, model-free gold standard. The residual threats, that the sample is a stratified subset of the corpus and that two annotators may share blind spots, are stated in Chapter 9.

The lift’s third typed decision, the per-turn reasoning move, is validated on its own gold, because a language-model classifier produces it, whereas the rule-based operation parser produces the other decisions. The protocol draws a blind sample of per-turn sub-intents, stratified across the move classes and augmented with a contested subset on which two candidate prompts disagree, and a first annotator labels it against the move rubric without sight of the classifier’s output. The classifier is scored against this gold as a chance-corrected Cohen’s κ , held to the same 0.6 reliability floor, and the contested subset settles the choice between the two candidate prompts. A second

annotator independently re-labels the representative sample, and their agreement is reported as raw agreement and Cohen’s κ , matching the two-annotator design of the operation gold.

Query expressiveness is the central RQ1 measurement. It is framed as a competency-question evaluation [54] and built in three layers. Competency-question evaluation is the established, requirements-driven member of the standard families of ontology evaluation [55]. The Interaction Graph is judged by whether it serves its intended use, because it is purpose-built and there is no reference ontology to compare it against. A fixed list of questions measures expressiveness on its own. Judging the graph by a single application built on it would instead blur expressiveness together with how good that one application happens to be.

The first layer is formal. A representation can answer a question when it holds every primitive the answer needs. This follows two traditions: database theory, which compares representations by *which* questions they can answer and defines completeness independently of efficiency [56], and data-provenance research, which separates models by the provenance questions their primitives support [22]. The protocol fixes a catalog Q of competency questions, each tagged with its class and with a required-primitive set $\rho(q)$, the primitives an answer needs. Writing $P(\Sigma)$ for the representation’s signature, a question q is natively expressible over Σ exactly when

$$E(q, \Sigma) \iff \rho(q) \subseteq P(\Sigma).$$

Each question then falls into one of three classes. It is native when the flat log already answers it. It is derivable when only the graph answers it, because the lift supplies the missing primitive. It is impossible when a required primitive lies in the range of no representation. Recovering a file’s exact bytes is impossible in this sense, since it needs file bytes that neither representation carries.

The catalog is composed along three axes, so that it measures expressiveness in a way that resists flattering the graph. The first is the query-scope taxonomy of Souza et al. [19]: every question is a targeted lookup or a multi-hop traversal, and both are represented. The second is ontology coverage: every node type and relation the lift builds, with two deliberate exceptions, is required by at least one question, so the schema drives the catalog while the questions the graph happens to answer well do not. The first exception is the Agent and its association edge, which the single-agent corpus makes constant and therefore indiscriminating, so it is omitted here and reinstated as an agent-responsibility question only if the corpus becomes multi-agent. The second is the Outcome node and its grader verdict, which no question reads, so that no reported result depends on the evaluation harness’s pass/fail signal, a signal with no counterpart in deployment. The coverage criterion governs the provenance core that the expressiveness catalog measures. The lift also derives a three-edge diagnostic relation layer (`contradict`, `invalidate` and `trigger`), defined with the schema in Chapter 4 and excluded from the expressiveness catalog measured here. This is a deliberate division in which RQ1 measures what the provenance core expresses, while the relation layer refines the struggle motifs of Chapter 7 to a finer, per-event granularity. The third is class balance: the catalog fixes explicit anchors in all three

formal classes, native questions the flat log should and does answer (a tool-name count, the tool calls under one assistant turn), derivable questions and an impossible anchor that needs a primitive no representation carries (reasoning the agent never emitted), so the set stays balanced across the three classes while the derivable case the graph wins remains one anchor among them. Several traversal questions instantiate documented diagnostic motifs, the `regression` and `code_rollback` questions following the interaction smells of Zhang et al. [28].

The second layer checks this classification empirically, adapting the functional-equivalence protocol of Souza et al. [19]. For each question the graph query produces its answer $A_{IG}(q)$, and the same answer is attempted over the flat log to produce $A_{flat}(q)$, which is scored for functional equivalence ($\equiv$) against the graph’s answer. To separate reader capability from representation, the flat answer is attempted by three readers of increasing capability, a matched 30B coder, a stronger 235B instruct model and a frontier reasoning model, each provider-pinned to the corpus quantization regime, and the rate is reported for each. Equivalence requires the flat answer to be *directly usable*: it must state the asked-for value. An answer that only references raw identifiers such as span ids or file line numbers, which would still have to be resolved against the log, does not count as a reproduction. The equivalence judge is itself a language model, validated against a human reading on a blind subset with agreement reported as Cohen’s κ , as for lifting fidelity. The reported quantity is the reproduction rate, the fraction of the *answerable* cells the flat log reproduces. A cell is one (session, question), answerable when the graph produces an answer for it, so impossible questions, which have no graph answer, and the occasional derivable question whose graph answer is empty for a given session are excluded, because reproducing an answer the graph does not give is undefined. The two cross-session questions are also excluded, because their graph answer joins the whole corpus while each reader sees one session, which makes them a Layer-1 reach result. Writing C_{ans} for the answerable cells,

$$\text{Repro} = \frac{|\{ (s, q) \in C_{ans} : A_{IG}(q, s) \equiv A_{flat}(q, s) \}|}{|C_{ans}|},$$

reported overall and per query-scope class (lookup or traversal). One derivable question, `struggle_signals`, is scored apart from this rate. A holistic flat judgment rarely matches the graph’s fine-grained motif detector cell-for-cell, so the question is posed as a per-motif detection task, the reader marking each of the six motif definitions present or absent, scored against the fired set as per-motif recall and precision.

The three reasoning-move questions augment the original catalog and are scored as a separate block over the full corpus, because their move-sequence answers are produced for every session. This block uses a matched-class equivalence judge validated against a human gold, where the main reproduction table uses a frontier judge over the sample. The matched judge shares the model of the matched reader, so a judge-reader affinity may favor that reader’s phrasing, a caveat reported alongside the block’s result.

The third layer measures how well each answer is grounded, which is separate from whether it is correct, in the spirit of the supporting-fact axis of multi-hop question

answering [57]. Let $\text{supp}(\Sigma, q)$ be the evidence supporting an answer to q over representation Σ . For the graph this is the bounded subgraph the query matched, a named set a reader can inspect, while for the flat log it is the transcript spans the answer cites. Citation-support runs over the same answerable cells the functional-equivalence layer scores, so groundedness continues that layer question by question. The evidence footprint is additionally reported for the two cross-session questions, where it records the graph’s whole-corpus reach against a flat log that cannot answer them. The first is the evidence footprint, $|\text{supp}(\Sigma_{\text{IG}}, q)|$ against $|\text{supp}(\Sigma_{\text{eval}}, q)|$ per question, counted in evidence items (graph nodes against cited spans), which captures how localized the evidence is on each side. The second is citation-support: whether the answer is genuinely backed by the evidence it cites, in the sense of attribution to identified sources [58]. The two representations differ in kind here. On the graph, the answer just *is* the result of the query, and the evidence is the exact subgraph that query matched, so the answer is read from that subgraph and cannot fail to be supported by it. On the flat log the answer and its cited spans are independent: an LLM produces the answer and points to transcript spans as support, but nothing guarantees those spans actually back it. Citation-support is therefore measured on the flat side, as binary textual entailment in the manner of citation evaluation [59, 60]: does the cited text entail the answer, so that the answer is derivable from the citation alone? An automated rater decides this and is validated against a human reading on a blind subset, with agreement reported as Cohen’s κ and, because one label dominates and deflates κ [61], the prevalence-robust Gwet AC_1 [62] beside it. A graded full/partial/none rater was tried first and failed this validation. The binary entailment formulation is the one that validates, reported in Section 6.3.3. Writing C_{cite} for the flat answers that cite at least one span, the reported rate is

$$\text{Support} = \frac{|\{(s, q) \in C_{\text{cite}} : \text{supp}(\Sigma_{\text{eval}}, q, s) \text{ entails } A_{\text{flat}}(q, s)\}|}{|C_{\text{cite}}|}.$$

Because the graph’s support holds by construction, the finding is the gap: an answer grounded in a small, checkable subgraph, against one the flat log produces but cannot localize or, for a derived value, entail.

5.3.2 RQ2: Diagnostic application

Prevalence describes the corpus. It does not test whether a detector is correct. For each motif m it reports two numbers, how many sessions the motif fires in and its total severity across the corpus,

$$\text{prev}(m) = |\{s \in C : m \text{ fires in } s\}|, \quad \text{sev}(m) = \sum_{s \in C} \text{severity}(m, s).$$

The first number records the sessions in which the motif is present, so a session in which m fires several times counts once. The second, $\text{severity}(m, s)$, is a whole number measuring how large the pattern is within one session, for example how many redundant reads it has or how long a repetition run is. Each motif measures severity in

its own units, so severity totals are meaningful within a motif but cannot be compared between motifs. Prevalence says how widespread a motif is, and severity says how heavy it is. Both numbers are produced by rule-based detectors that read the graph directly, with no model involved, so they are exact and repeatable. Every motif in the catalog is listed even when it never fires, because a zero is itself a result. Motifs that cannot occur in this corpus at all, such as ones that need interactive user turns when the sessions are autonomous, are excluded from the catalog and do not appear as zero entries. The validation measurement below decides whether a non-zero count is a real pattern or an over-eager rule.

Validation checks whether the detector is correct, using the same protocol as lifting fidelity in Section 5.3.1, now applied to motif presence. The labeled unit is a session-and-motif pair, and the question for each is whether that motif is present in the session. Two annotators read the rendered trace and mark every pair present or absent, blind to the detector’s verdict. Their consensus forms the gold, contested pairs are recorded and excluded until adjudicated, and their inter-annotator reliability is reported as Cohen’s κ . The detector is then scored against the gold as per-motif precision, recall and F_1 . The exact content motifs, whose detection is a deterministic structural comparison, are validated by unit tests instead, so the human gold targets the behavioral motifs that require perceptual judgment. The cognitive reasoning-move motifs are validated the same way against a second two-annotator gold over the rendered move sequence, which scores the detectors given the classified moves and leaves the move classifier’s own fidelity to the representation evaluation of Chapter 6.

Diagnostic value asks whether the pattern a detector fires on is consequential, the question a diagnostic application ultimately answers. Each session carries an Outcome node, so motif presence and severity join to the session’s resolution verdict and its cost in turns, tokens and wall-clock time with no further instrumentation. The measurement pools the primary and robustness corpora and restricts to the 171 sessions carrying a classified reasoning-move sequence, so the action and cognitive layers are read on the same base. Because every epoch runs the same instances and the pool mixes two agents, observations cluster within instance and within agent, and the same instance contributes several correlated rows. The measurement therefore fits, per motif, a population-averaged logistic model of resolution on motif presence by generalized estimating equations (GEE) [63], clustering on instance under an exchangeable working correlation and holding the agent and epoch as fixed covariates, and reads the odds ratio of the presence term through robust standard errors, so the within-instance and within-agent dependence is modeled directly and the estimate is adjusted for agent capability and epoch. A motif whose present group separates the outcome perfectly or holds too few sessions is left descriptive, its Woolf odds ratio reported without a model fit. Testing every motif’s association with resolution at once inflates the family-wise error rate, so the motifs with a real model fit carry a joint Benjamini-Hochberg false-discovery-rate correction [64] across that family, while the descriptive motifs, already excluded from the model fit, stay outside it. Struggle severity is related to each cost measure by rank correlation, and trace length is controlled wherever severity is at issue, following the software-engineering practice of relating structural patterns to fault-proneness with bounded effects [65, 66].

Causal explanation reads the relation layer as a measurement. The lift derives three edge types (`trigger`, `invalidate` and `contradict`, Chapter 4) that decompose the struggle motifs into per-event chains. The measurement reports their inventory, the edge count and the sessions each type spans, and a localization check on the `invalidate` edges, namely whether the edit a failing test invalidates targets the module the test exercises. The small edge counts make this a coverage check and a worked chain, too few for a powered statistic.

Scalability under growing logs makes the comparison against the per-trace LLM-as-judge quantitative, targeting a known problem: the failure-attribution literature finds that an LLM reading a flat log detects problems less reliably as the log grows [14, 15]. The measurement therefore treats reliability as a function of trace length. Scoring the judge against gold for accuracy would instead conflate whether the judge is right with whether it stays stable as traces grow, so the measurement leaves correctness to the validation measurement above. Following the standard treatment of long-context degradation [67, 68, 69], a corpus that spanned a wide enough length range would group trajectories into length bins and compare the judge and detector across bins on their agreement, abstention and cost. The detector is length-invariant by construction, so the prediction is that the judge degrades with length while the detector stays flat. Only a single agent is measured here, and because its traces are short the realized measurement reduces to aggregate cost scaling, a lower bound on the gap that stops short of a demonstrated length curve.

5.3.3 RQ3: Memory application

The memory feasibility pilot runs a no-memory baseline and three Interaction-Graph-augmented arms on the same instances, each augmented arm retrieving from earlier sessions on the same repository, excluding the current task (leave-one-out, whose construction and guard enforcement are set out in Appendix C.3). With thirty instances, one pinned model (Qwen3-235B, with its provider and seed fixed in Appendix C.4) and three epochs per arm, the pilot asks whether retrieval from the graph can be delivered and how it is used, and is not powered to estimate an effect size [70]. It therefore reports descriptive metrics: the resolution rate and a cost and process family per arm (trajectory length, token cost, a length-normalized redundancy rate and the typed motif and signal rates), the retrieval rate (the fraction of sessions that made at least one retrieval call) and a retrieval-by-outcome crosstab. Because the arms run on the same instances, every difference from the baseline is reported with a paired bootstrap confidence interval that gives its direction and uncertainty. One contrast is pre-specified as confirmatory, the free arm against the no-memory baseline on the redundancy rate, and is exempt from the family correction. The remaining process-metric contrasts are exploratory and carry a Benjamini-Hochberg false-discovery-rate correction [64] across the family, keeping the pilot’s few inferential claims honest without treating it as a powered study. The contribution is that the graph can serve as a retrieval source, with the telemetry showing when it is used.

Table 5.1: Measurements, metrics and the research question each serves.

RQ	Quantity	Metric
RQ1	Construction coverage	unknown-path rate, classifier coverage, content-hash coverage
RQ1	Lifting fidelity	operation accuracy, macro- F_1 and Cohen's κ , edited-path precision and recall
RQ1	Query expressiveness	class coverage, reproduction rate, evidence footprint, citation-support
RQ2	Motif prevalence	sessions and severity per motif
RQ2	Motif validation	detector precision and recall versus labels
RQ2	Diagnostic value	motif presence and severity versus resolution and cost
RQ2	Causal explanation	relation-edge inventory and invalidate localization
RQ2	Scalability under growing logs	cost per mechanism versus corpus size
RQ3	Memory pilot	resolution rate and cost per arm, retrieval rate, retrieval-by-outcome crosstab, paired CI on rate difference

5.4 Summary of measurements

Table 5.1 lists every measurement, its metric and the research question it serves.

6 Representation evaluation

This chapter answers RQ1. It reports the faithfulness of the lift, through construction coverage and lifting fidelity, and the query expressiveness of the graph, through a competency-question evaluation. The measurement protocol is the one fixed in Chapter 5.

6.1 Construction coverage

Derived from 30 SWE-bench Verified instances, the resulting corpus consists of 3,730 tool actions spanning 90 agent sessions. The number of actions per session varies from a minimum of 10 to a maximum of 87, with a median of 41. The distribution of operations is primarily led by *run* (31.4%) and *search* (28.7%), followed by *read* (18.9%), *edit* (14.9%) and *test* (6.1%).

The completeness of the lift is bounded by the three coverage figures fixed in Chapter 5. The rule-based classifier parses every action into a structured operation record (classifier coverage 100%), and no artifact is left with an unrecoverable path (unknown-path rate 0%). The git observer fingerprinted all 440 version-controlled edited-file artifact versions (content-hash coverage 100%), each carrying a concrete repository path and a verifiable content fingerprint. This artifact-version coverage is measured over the version-controlled history. It is distinct from the edit-action path *recall* reported in the fidelity section below, which is bounded by non-version-controlled writes the observer does not fingerprint. The lift rebuilds the entire action sequence and file-editing lineage with zero data loss over this version-controlled history.

6.2 Lifting fidelity

Following the lifting-fidelity protocol (Section 5.3.1), fidelity is assessed on an operation-stratified sample of 75 actions from the lifted corpus. Inter-annotator agreement is strong, with raw agreement of 89.3% and a Cohen’s κ of 0.87 (95% confidence interval [0.78, 0.95], percentile bootstrap), which establishes the human gold as reliable before it is used to judge the lift. The eight disagreements were adjudicated, yielding a gold of 75 actions (67 agreed, 8 adjudicated).

The lift pipeline identifies the operation with 92.0% accuracy, a macro- F_1 of 0.919 and a chance-corrected Cohen’s κ of 0.90 (95% confidence interval [0.81, 0.97]). Per-operation scores (Table 6.1) put F_1 highest for *edit* (1.00), *test* and *search* (0.97) and *read* (0.88). The weakest is *run* (0.79). Because *run* serves as the residual bucket described in Chapter 4, it absorbs any command no rule matches. Four of the six sample errors (Table 6.2) involve *read* or *search* actions incorrectly labeled as *run*, which puts its precision down to 0.73. The single error on the run/test boundary mirrors human annotator difficulties. This boundary caused five out of eight human

Table 6.1: Per-operation lifting fidelity against the adjudicated gold. *Gold/Pred* are gold and lifter action counts, *Human* the annotators’ agreement. Overall row: accuracy, macro- F_1 , Cohen’s κ (95% bootstrap interval).

Operation	Gold	Pred	Precision	Recall	F_1	Human
edit	15	15	1.00	1.00	1.00	1.00
read	17	15	0.93	0.82	0.88	0.88
run	13	15	0.73	0.85	0.79	0.65
search	16	15	1.00	0.94	0.97	1.00
test	14	15	0.93	1.00	0.97	1.00
Overall	75	75	accuracy 0.92, macro- F_1 0.919, κ 0.90 [0.81, 0.97]			

Table 6.2: Lifter confusion matrix against the adjudicated gold, gold operation in rows and lifter-assigned in columns. All six errors touch *run*.

Gold	Predicted					n
	edit	read	run	search	test	
edit	15					15
read		14	3			17
run		1	11		1	13
search			1	15		16
test					14	14
n	15	15	15	15	15	75

disagreements, which the gold standard resolved by classifying inline `python -c` scripts as *run*.

Regarding edited-file recovery, the pipeline is highly precise but slightly under-complete. Every path identified by the lifter is correct (precision 1.00), resulting in an 80% recovery rate of the changed files in the sample (recall 0.80, F_1 0.89). These missed paths are non-version-controlled creations, such as `.backup` copies, which the git observer does not fingerprint and the lifter consequently ignores as artifacts.

Per the move-classifier gold protocol (Section 5.3.1), 140 sub-intents were labeled, 90 stratified across move classes and 50 contested. For the 90 representative items, the classifier achieved a chance-corrected Cohen’s κ of 0.74 against the first annotator, and a second annotator’s independent labels agree with the first at an inter-annotator κ of 0.76 (raw agreement 79%), above the 0.6 floor. On the 50 unclear items, the labels confirmed the LLM prompt selection, with the chosen prompt achieving 24 matches compared to the alternative’s 19. Annotation disagreements clustered around the boundaries of *explore*, *locate* and *search*, as Table 6.3 shows.

6.3 Query expressiveness

This is the central RQ1 result. A catalog of competency questions is evaluated against the flat evaluation log and the typed graph, and each is classified as **native**, **derivable**

Table 6.3: Per-move fidelity of the reasoning-move classifier against the human gold (90 cells, inter-annotator $\kappa = 0.76$). Small per-move supports make the overall κ the reliable figure.

Move	Gold	Precision	Recall	F_1
explain	12	0.91	0.83	0.87
explore	16	0.75	0.56	0.64
generate	12	1.00	0.67	0.80
locate	8	0.70	0.88	0.78
other	8	0.42	0.62	0.50
produce	10	1.00	0.90	0.95
refactor	5	0.71	1.00	0.83
run_tests	11	0.91	0.91	0.91
search	8	0.60	0.75	0.67
Overall	90	accuracy 0.77, macro- F_1 0.77, κ 0.74		

only after lifting or **impossible**.

6.3.1 Formal expressibility

The first layer is deterministic. Each competency question is classified by the containment criterion fixed in Chapter 5, $E(q, \Sigma) \iff \rho(q) \subseteq P(\Sigma)$, evaluated against both signatures. Consequently, the outcome depends only on the properties of the two schemas themselves.

The flat evaluation log exposes seven primitives,

$$P(\Sigma_{\text{eval}}) = \{\text{tool_name, raw_command, message_role, span_parent, token_usage, turn, resolved}\},$$

the raw tool name and arguments, the message role and call-nesting parent recoverable from the OpenTelemetry spans, the token usage, the message order and the final grader verdict, the one primitive specific to the evaluation harness. The graph signature subsumes these and adds the typed vocabulary of the Chapter 4 data model read as a primitive set, its seven entity types, eight typed relations and seven node attributes, which with the seven subsumed flat primitives make twenty-nine in total. The three semantic-relation edges Chapter 4 adds beyond this provenance core are excluded here, a diagnostic refinement of the struggle motifs that lies outside the provenance core this catalog measures. The inclusion $P(\Sigma_{\text{eval}}) \subseteq P(\Sigma_{\text{IG}})$ holds by construction. The harness raises an error on any question that would violate it, and none did.

The `content_hash` primitive enters $P(\Sigma_{\text{IG}})$ only when the corpus carries it, the signature being read from the lifted data while no primitive is assumed in advance. On the evaluated corpus every one of the 440 artifact versions holds a fingerprint (Section 6.1), so `content_hash` $\in P(\Sigma_{\text{IG}})$ and the content-identity questions are classified against real coverage, the signature reflecting a measured property of the corpus.

Table 6.4 classifies the twenty competency questions into three categories: two are native, seventeen become derivable only after lifting and one remains impossible in both representations. The table doubles as an index: its *Measured* column points to where each question’s empirical result is reported, so the reproduction and detection tables below read against it. The split is structured. The two native questions are basic lookups that only need flat primitives, while all thirteen traversal questions require the lift and are thus derivable. The entire catalog is harness-independent. Because no question reads the graph’s `outcome` node, the results do not depend on an evaluation signal. The lift is essential for the traversal questions because it provides primitives absent from the call-structured flat log. The single impossible question requests reasoning the agent never emitted. It needs a primitive (`hidden_reasoning`) that is absent from both representations.

Because the lift fingerprints every artifact version with a `content_hash` (Chapter 4), the graph answers relational questions such as whether a file returned to its starting state or whether two sessions reached identical outcomes, both derivable and both impossible over a flat log, even though recovering the file’s exact bytes remains impossible for either representation.

6.3.2 Functional equivalence

This layer reports the reproduction rate defined in Chapter 5, its reader ladder instantiated as `qwen3-coder-30b` (matched), `qwen3-235b` (strong) and `minimax-m3` (frontier, reasoning enabled), the per-role model, provider and decoding pins recorded in Appendix A.3.

The judge, whose full prompt is in Appendix A.2, is validated before its verdicts are read. On a blind, verdict-stratified sample of 45 cells a single human annotator labeled equivalence without seeing the judge’s call. Agreement is Cohen’s $\kappa = 0.80$ (raw 0.91), and on the 15 contested cells where the flat answer is a bare identifier dump it reaches $\kappa = 1.00$. Unlike the two-annotator lifting and motif golds, this judge-validation gold is single-coded, a stated internal-validity limitation.

Over the 290 answerable cells per reader the reproduction rate is 0.03, 0.18 and 0.23 along the ladder (Table 6.5), excluding the two cross-session questions and the empty-answer `no_net_change` cell for the reasons fixed in Chapter 5. The rate rises with reader capability but stays far below one, and it separates by class. Lookups reproduce at 0.08, 0.29 and 0.38, traversals at 0.00, 0.09 and 0.13.

Table 6.6 presents the reproduction rate for each question, revealing a substantive pattern. Each per-question rate is computed over that question’s answerable cells only, and every cell is a single draw at temperature 0, so questions answerable in few sessions are descriptive, with too few cells for a stable rate. The reproduction rates of the two native questions, answerable in theory without the lift, expose a gap between holding a primitive and successfully recovering its value.

For example, the frontier model achieves a perfect 1.00 on the tool-call count, but the other models fail. On per-turn tool calls the pattern inverts: the strong 235B reader scores 0.57 while the frontier reader nearly fails at 0.07 and the matched reader scores zero. The reason is that both answers must be assembled across the whole log and

Table 6.4: Master index of the twenty competency questions. *Type* is the query scope, $\rho(q)$ the required primitives, *Class* the containment verdict and *Measured* where the result appears: reproduction (Table 6.6), detection (Table 6.7), *excluded* or *none* (impossible). Primitives are PascalCase nodes with dotted attributes (Node.attr) and lowercase relations. Full questions in Appendix A.1.

Question	Type	$\rho(q)$	Class	Measured
tool-call count	lookup	tool_name	native	Tab. 6.6
per-turn tool calls	lookup	message_role, span_ parent	native	Tab. 6.6
operation distribution	lookup	ToolAction.op	derivable	Tab. 6.6
test outcome	lookup	ToolAction.{op, test_ failed}, Evidence	derivable	Tab. 6.6
struggle signals	lookup	Signal.{motif, severity}	derivable	Tab. 6.7
edit_test_fail_loop	traversal	ToolAction.{op, test_ failed}, Evidence, generated, informed_ by_path	derivable	Tab. 6.6
regression	traversal	ToolAction.{op, test_ failed}, Evidence, generated, informed_ by_path	derivable	Tab. 6.6
action spine	traversal	ToolAction.op	derivable	Tab. 6.6
intent to artifact	traversal	Artifact, Intent, ToolAction, generated, used	derivable	Tab. 6.6
evidence for action	traversal	Evidence, ToolAction, observes	derivable	Tab. 6.6
signal-implicated actions	traversal	Signal.{motif, severity}, ToolAction, indicates	derivable	Tab. 6.6
version lineage	traversal	Artifact.version, derived_from	derivable	Tab. 6.6
sub-goal decomposition	traversal	Intent, ToolAction, fulfills	derivable	Tab. 6.6
cross-session files	traversal	Artifact	derivable	excluded
no_net_change	traversal	Artifact.{version, content_hash}	derivable	excluded
cross_session_same_ state	traversal	Artifact.content_hash	derivable	excluded
move distribution	lookup	Intent.move_type	derivable	Tab. 6.6
fix without localize	traversal	Intent.{move_type, turn}	derivable	Tab. 6.6
repro before fix	traversal	Intent.{move_type, turn}	derivable	Tab. 6.6
reasoning recovery	lookup	ToolAction, hidden_ reasoning	impossible	none

Table 6.5: Reproduction rate of the flat log along the reader ladder (two cross-session questions excluded). Each cell: rate with reproduced count in parentheses, over the header’s n answerable cells. *Reason.* flags reasoning-enabled decoding.

Reader	Model	Reason.	Overall $n=290$	Lookup $n=120$	Traversal $n=170$
matched	qwen3-coder-30b	no	0.03 (9)	0.08 (9)	0.00 (0)
strong	qwen3-235b	no	0.18 (51)	0.29 (35)	0.09 (16)
frontier	minimax-m3	yes	0.23 (68)	0.38 (46)	0.13 (22)

the readers fail in two ways. On the tool-call count they return a directly-usable but wrong number, undercounting the calls, apart from the frontier reader, which counts correctly. On the per-turn map they return an incomplete or misaligned mapping, and the weakest reader returns a bare list of numbers in place of the tool names, which the directly-usable criterion rejects. That the same reader can lead on one native lookup and trail on the other is a first sign that the per-question pattern does not track model scale. The equivalence judge, validated at $\kappa = 1.00$ on the contested identifier-dump cells, scores each of these non-equivalent. Some derivable traversals reverse the pattern for the larger models. The regression question reproduces fully (1.00), and intent-to-artifact (0.62) and test-outcome (0.60) clear one half. Each reconstructs a short causal chain the transcript already narrates, a test that passed, an edit, then the same test failing. Such a chain is easier for a reader to follow than a count it must aggregate. Formal expressibility therefore bounds what is *possible*, while the reproduction rate measures what a reader *realizes* and they diverge.

The bottom panel of Table 6.6 presents three reasoning-move questions appended after the main catalog. Because their move-sequence answers are produced for every session, they are scored as a separate block over all 90 sessions by a matched-class equivalence judge, the same 30B model as the matched reader, distinct from the frontier judge used for the main reproduction table and itself validated against a blind 40-cell human sample at Cohen’s $\kappa = 0.95$ (raw agreement 0.975), likewise single-coded. These metrics reveal that reproduction rates do not grow cleanly with model scale. The 30B model and the frontier model perform comparably, both scoring higher than the 235B model. This variance is primarily driven by the boolean `fix_without_localize` question. While the matched and frontier models reproduce it almost perfectly, the 235B model fails completely due to a tendency to over-claim a fix without any preceding localization. Because the matched judge shares the model of the matched reader, a judge-reader affinity may favor that reader’s phrasing, so the matched reader’s near-perfect `fix_without_localize` score is read with that caveat. The `repro_before_fix` boolean achieves a roughly 50% reproduction rate across all readers. The move and operation distribution sit near zero and this low score occurs because two generated distributions rarely achieve a cell-for-cell match under the strict evaluation criterion.

Following the protocol in Chapter 5, the `struggle_signals` question is scored apart from the reproduction rate as a per-motif detection task, the reader marking each

Table 6.6: Per-question reproduction rate by reader, with column n the answerable-session denominator each rate is computed over. The reasoning-move rows (bottom panel) use a matched-class judge (the 30B reader’s model, validated $\kappa = 0.95$) over all 90 sessions, the rest a frontier judge over the 30-session sample. Most questions are answerable in all 30 sessions. The `regression` and `edit_test_fail_loop` rows are answerable in 1 session each. Version lineage covers 19 sessions and intent to artifact 29. Full record in Appendix A.1.

Question	Class	Scope	n	30B	235B	M3
tool-call count	native	lookup	30	0.03	0.00	1.00
per-turn tool calls	native	lookup	30	0.00	0.57	0.07
test outcome	derivable	lookup	30	0.27	0.60	0.37
operation distribution	derivable	lookup	30	0.00	0.00	0.10
<code>regression</code>	derivable	traversal	1	0.00	1.00	1.00
intent to artifact	derivable	traversal	29	0.00	0.52	0.62
version lineage	derivable	traversal	19	0.00	0.00	0.05
action spine	derivable	traversal	30	0.00	0.00	0.07
evidence for action	derivable	traversal	30	0.00	0.00	0.00
signal-implicated actions	derivable	traversal	30	0.00	0.00	0.00
sub-goal decomposition	derivable	traversal	30	0.00	0.00	0.00
<code>edit_test_fail_loop</code>	derivable	traversal	1	0.00	0.00	0.00
<i>Reasoning-move questions</i>						
move distribution	derivable	lookup	90	0.00	0.01	0.00
fix without localize	derivable	traversal	90	0.99	0.03	1.00
repro before fix	derivable	traversal	90	0.53	0.40	0.43

of the six motifs present or absent against the set the graph fired, reported as per-motif recall and precision (Table 6.7).

Motif detection scales predictably with model size. The matched coder detects nothing, recall zero on every motif. The strong reader recovers the common redundancy motif at recall 0.60 and perfect precision while over-flagging the rare ones, and the frontier reader reaches recall 0.90 on redundancy and 1.00 on error-burst and regression, at high precision on the common cases. These results fall in the indicative range of the human agreement reported in the original interaction-smell study (Cohen’s κ 0.78–0.82) [28], though κ and recall are not the same quantity and the comparison is only suggestive. They indicate that, while the flat log contains the motif data, extracting it requires a frontier reader to read the entire trace, whereas the graph achieves the same result with one structural query (a formal comparison of the graph against a human gold standard follows in Chapter 7)

6.3.3 Groundedness

Groundedness continues the functional-equivalence layer over the same answerable cells, with the evidence footprint additionally reported for the two cross-session questions as a whole-corpus reach figure (citation-support is undefined there and marked with a dash). For each question Table 6.8 reports the two quantities defined in Section 5.3.1: the evidence footprint $\text{supp}(\Sigma, q)$, counted in evidence items on

Table 6.7: Per-motif detection of `struggle_signals` by reader, recall/precision against the motif set the graph fired. A dash is undefined precision.

Motif	30B	235B	M3
redundancy	0.00/–	0.60/1.00	0.90/1.00
repetition	0.00/–	0.29/0.50	0.14/1.00
code_rollback	0.00/–	0.60/0.27	0.60/0.60
error_burst	0.00/–	0.50/0.18	1.00/0.67
edit_test_fail_loop	0.00/–	1.00/0.04	1.00/0.20
regression	0.00/–	1.00/0.05	1.00/1.00

Table 6.8: Groundedness per question, over the same cells as functional equivalence. *Graph* and *Flat* are mean support-set sizes in evidence items, graph nodes against cited spans (over the readers). *C-sup* is the flat citation-support rate (binary entailment, mean over readers, validated $\kappa = 0.71$, $AC_1 = 0.82$). A dash marks no measure: `struggle_signals` uses detection (Table 6.7), cross-session questions are excluded. Move rows are over 90 sessions.

Question	Graph	Flat	C-sup
tool-call count	1	23	0.00
per-turn tool calls	82	23	0.00
operation distribution	6	24	0.00
test outcome	1	8	0.03
struggle signals	2	7	–
regression	2	1	0.04
edit_test_fail_loop	3	3	0.03
action spine	42	11	0.00
intent to artifact	4	3	0.03
evidence for action	25	13	0.20
signal-implicated actions	13	5	0.00
version lineage	6	9	0.08
sub-goal decomposition	56	25	0.00
cross-session files	53	1	–
cross_session_same_state	25	1	–
<i>Reasoning-move questions</i>			
move distribution	1	27	0.00
fix without localize	1	5	0.00
repro before fix	1	5	0.03

each side (graph nodes against cited spans), and the flat citation-support rate. The graph support set is the answer’s own size, one node for a scalar and the full matched subgraph for a traversal, against the span list the flat reader cites.

Citation-support follows the binary-entailment metric fixed in Chapter 5, its rater prompt given in Appendix A.2, and on a fresh blind human pass it validates cleanly (Table 6.8).

The flat citation-support rate stays near zero, 0.03, 0.10 and 0.02 along the reader ladder and mostly zero per question in the *C-sup* column. The reason is the kind of answer. A count, an operation distribution, a per-turn map or an action spine is a computed answer that appears nowhere in the transcript as text, so no single cited

span can *entail* it and the rate sits at zero. Only answers that are themselves a quoted fragment score above zero, the produced evidence and the recovered reasoning, where a cited span can hold the answer word for word. This is the groundedness gap. The graph has no such gap, because its answer *is* the result of the query, read straight from the matched subgraph, so the evidence supports it automatically. The metric itself carries a second lesson. The graded full/partial/none rater tried first failed to validate at Cohen’s $\kappa = 0.17$ and a stronger model did not rescue it, yet the entailment version validated on the same inexpensive model. Redefining the task fixed it, while a bigger model did not help, consistent with reports that automatic attribution remains hard even for frontier models [71].

6.4 Summary

The chapter’s three sections form one account. Construction coverage and lifting fidelity establish the lift as a faithful base for the comparison, complete in coverage, every action typed and every artifact version fingerprinted, and correct in labeling, its operation typing validated against a human gold. The graph the query-expressiveness section then evaluates is therefore both complete and correctly typed.

Extending this foundation, the two representations diverge across three analytical layers. Formally, the graph strictly dominates the flat log: it answers every question the flat log can, while unlocking new questions that only become answerable post-lift (leaving only one query impossible for both). Empirically, this separation widens because theoretical expressibility diverges from practical reproduction. A capable language-model reader recovers only a fraction of those answers from the unstructured log, whereas the graph’s typed structure lets a simple deterministic query recover them exactly. While stronger readers extract more, they still fall short and behave erratically even on formally native lookups, where the flat log provably holds the primitive and the shortfall is a reader-aggregation failure. Success does not scale cleanly with model capability, as stronger readers occasionally fail on questions that weaker ones solve. Motif detection reinforces this divergence. Although struggle motifs exist within the flat log, only a frontier reader can recover them, only by processing the entire trace. In contrast, the graph surfaces them with a single structural query. When the flat log does yield an answer, it cannot ground it.

The graph’s value is therefore twofold: it expands the universe of answerable questions, and it ensures those answers are grounded, localizable and free from the need for recomputation.

7 Diagnostic application

This chapter answers RQ2. It expresses recurring struggle motifs, an action layer of structural Signals and a cognitive layer of reasoning-move motifs, as deterministic subgraph queries. It reports their prevalence over the corpus, validates the detectors against a two-annotator human gold, relates motif presence to session outcome, derives a causal-relation layer that renders a struggle as a traversable chain and contrasts the query mechanism with a per-trace large-language-model judge. The measurement protocol is the one fixed in Chapter 5.

7.1 Motif catalog and cross-walk

The diagnostic vocabulary spans two layers. The first consists of eight **struggle motifs**, structural patterns over the agent’s actions and their effects within the Interaction Graph, composed by adapting the nine “interaction smells” of Zhang et al. [28] to the autonomous single-agent setting. Motifs are categorized as *detected* (successfully mapped to a coded motif), *new* (a novel struggle motif introduced in this work) or *absent* (lacking a faithful structural equivalent in autonomous traces). Table 7.1 gives the result: four smells map to motifs the graph detects, three further motifs are introduced with no Zhang counterpart, and the remaining five smells are recorded absent with a stated reason that Appendix B.2 expands into the full per-smell rationale.

The eight motifs are implemented via two methodologies. Six are instantiated by the lifter as Signal nodes, defined in Chapter 4 (Table 4.5) and function directly as structural queries. The two other motifs, `no_net_change` and `cross_session_same_state`, are excluded from the Signal layer. `cross_session_same_state` is a cross-session pattern: it compares one file’s content state across separate sessions, so no single session graph can own it, and it is computed only at diagnosis time over the whole corpus. `no_net_change` verifies whether a file’s initial and final content

Table 7.1: Cross-walk of Zhang et al.’s nine interaction smells to Interaction Graph motifs.

Smell	Classification	Motif / reason absent
Cross-turn inconsistency	detected	<code>edit_test_fail_loop</code>
Partial functionality breakdown	detected	<code>regression</code>
Code rollback	detected	<code>code_rollback</code>
Repetitive response	detected	<code>repetition, redundancy</code>
–	new	<code>error_burst</code>
–	new	<code>no_net_change</code>
–	new	<code>cross_session_same_state</code>
Ambiguous instruction	absent	<i>no interactive user</i>
Incomplete instruction	absent	<i>no interactive user</i>
Must-do omission	absent	<i>needs issue-constraint semantics</i>
Must-not violation	absent	<i>needs prohibition semantics</i>
Signature mismatch	absent	<i>needs symbol-level nodes</i>

fingerprints are identical. Because the **Signal** layer evaluates the sequence of tool actions while this motif evaluates the resulting file states, it functions as a query over content identity.

The detected motifs adopt Zhang’s behavioral definitions but supply their own structural detectors, given verbatim in Appendix B.1, and the structural form is often richer than the original. The **redundancy** detector, for instance, is state-aware: it flags a read or test that repeats an earlier identical one only when no edit changed the relevant file between them, which separates wasted work from a legitimate re-check, sharper than the surface text match of the original smell.

The second layer is **cognitive**. It reads the sequence of reasoning moves, the per-turn classification of *why* the agent acted (Chapter 4), and detects six further struggle motifs over it: **repeated_generation**, **missing_verify**, **investigation_heavy**, **relocalization_loop**, **oscillating_test_fix** and **skipped_reproduce**, adapted from the thought-action taxonomy of Bouzenia and Pradel [43]. These MoveMotifs carry a validity caveat the Signals lack. A Signal is a deterministic rule over deterministic tool actions, while a MoveMotif is a deterministic rule over an LLM-classified move layer, so its detection inherits the move classifier’s reliability (Chapter 4).

7.2 Prevalence

Because the detectors are deterministic, rule-based systems that evaluate the Interaction Graph directly, they produce exact and repeatable counts. Table 7.2 reports the session frequency and total severity (defined in Section 5.3.2) for each motif across two corpora. Results from the validated 30B corpus show an uneven distribution: **redundancy** and **repetition** are frequent, while the edit-and-test motifs (**edit_test_fail_loop**, **regression**, **error_burst**) are rare, reflecting an agent that reads and searches more than it thrashes. The **no_net_change** motif never fires, and **cross_session_same_state** is a cross-session pattern whose mechanism is analyzed later.

These detectors are also applied to a second corpus from a larger LLM (Qwen3-235B) across the same instances. This comparison highlights expected capability differences: edit-level struggles (**redundancy**, **code_rollback**, **regression**) decrease for the stronger model, whereas non-edit motifs (**repetition**, **cross_session_same_state**) remain stable.

The cognitive layer shows the same corpus dependence (lower block of Table 7.2). The 30B agent localizes repeatedly, so **relocalization_loop** leads, while the stronger 235B agent reproduces and verifies less often, so **missing_verify** and **skipped_reproduce** become its most frequent motifs and the localization loops recede.

Table 7.2: Per-motif prevalence over the corpora.

Motif	Qwen3-Coder-30B		Qwen3-235B	
	Sessions	Severity	Sessions	Severity
<i>Action Signals</i>				
redundancy	90	766	71	477
cross_session_same_state	50	25	47	27
repetition	44	161	48	215
code_rollback	21	52	7	10
error_burst	8	24	6	19
edit_test_fail_loop	3	10	2	7
regression	3	4	0	0
no_net_change	0	0	0	0
<i>Reasoning-move motifs</i>				
relocalization_loop	35	57	11	18
skipped_reproduce	22	22	47	47
oscillating_test_fix	21	104	10	88
repeated_generation	16	39	29	81
missing_verify	9	15	48	94
investigation_heavy	8	5	17	13

7.3 Validation

While prevalence describes the corpus, it cannot confirm whether a fired detector reflects a genuine pattern or an over-eager rule. Thus detectors are validated against a human gold standard. Following the two-annotator protocol from Section 5.3.1, a repo-stratified sample of 30 sessions is created to mark motif presence or absence, with the annotation guide and the selected sessions set out in Appendix B.4. The annotators’ consensus forms the gold standard.

Human validation targets the five **behavioral** motifs, the ones that require perceptual judgment. The deterministic motifs (`code_rollback` and `no_net_change`) are structural comparisons checked by unit tests, so human relabeling could only add noise. Similarly, `cross_session_same_state` is an exact, cross-session metric evaluated mechanistically later in the study.

The gold proved highly reliable, securing an overall Cohen’s $\kappa = 0.986$ over 150 cells, with one contested. Evaluated against this gold standard, the detectors prove precise (Table 7.3). Precision is near-perfect, dipping only for `redundancy`, so a fired signal is almost never a false alarm. Recall is perfect for `error_burst`, `regression` and `redundancy` but drops for `repetition` and `edit_test_fail_loop`, where the strict structural rules miss instances that human annotators caught, with the per-motif confusion counts and the named false-positive and false-negative sessions given in Appendix B.4. These scores reflect small positive classes, however, with only three `regression`, four `error_burst` and five `edit_test_fail_loop` positive sessions in the gold. Consequently, these results indicate consistency across a limited sample. By design, the detectors prioritize precision over recall, ideal for diagnostics where false alarms are highly disruptive.

The cognitive layer is validated the same way. The six MoveMotif detectors are checked against a two-annotator human gold over a repo-stratified sample of 30

Table 7.3: Detector validation against the two-annotator gold, by layer, reporting inter-annotator agreement (IAA), precision, recall and F_1 . The three deterministic Signals, checked by unit test, are omitted.

Motif	IAA κ	Precision	Recall	F_1
<i>Action Signals</i>				
error_burst	1.00	1.00	1.00	1.00
regression	1.00	1.00	1.00	1.00
redundancy	1.00	0.93	1.00	0.97
repetition	1.00	1.00	0.73	0.84
edit_test_fail_loop	0.89	1.00	0.60	0.75
<i>Reasoning-move motifs</i>				
oscillating_test_fix	0.89	1.00	1.00	1.00
repeated_generation	1.00	1.00	0.83	0.91
missing_verify	0.87	1.00	0.75	0.86
relocalization_loop	0.84	0.80	1.00	0.89
skipped_reproduce	0.77	0.88	0.88	0.88
investigation_heavy	0.65	0.50	1.00	0.67

move-enriched sessions, the protocol above applied to the rendered move sequence. Agreement is high, an overall Cohen’s $\kappa = 0.861$ over 180 cells with eight contested and excluded, and the detectors are precise (lower block of Table 7.3), with `oscillating_test_fix` matching the gold exactly and `repeated_generation` firing at perfect precision with one missed positive. `investigation_heavy` is the weakest, its sixty-percent threshold the hardest to judge. As with the rare Signals these numbers reflect small positive classes. The validation covers the detection rules given the classified moves, and the move classifier’s own reliability is reported in Chapter 4.

7.4 Diagnostic value

A useful diagnostic should track the outcome it diagnoses. The previous section showed that a detector fires accurately. This section asks whether the pattern it detects actually matters, so that a session carrying a struggle motif is less likely to resolve its task. The measurement applies to both layers, the action Signals and the cognitive reasoning-move motifs. Each motif is linked to the session’s Outcome node, and the analysis uses the 171 sessions that carry a classified move sequence, so both layers are measured on the same sessions.

Every modeled motif fails the joint correction. The measurement fits, per motif, a population-averaged logistic model of resolution on motif presence, clustered on instance with the agent and epoch held as fixed covariates and read through robust standard errors (Section 5.3.2). Of the nine motifs with a real model fit, none clears a Benjamini-Hochberg-adjusted $q < 0.05$ once corrected for testing all nine together. The closest three are `repeated_generation` (odds ratio 0.53, 95% confidence interval [0.30, 0.92], raw $p = 0.024$, $q = 0.167$), `relocalization_loop` (0.57, [0.33, 0.97], raw $p = 0.037$, $q = 0.167$) and `code_rollback` (0.49, [0.23, 1.03], raw $p = 0.059$, $q = 0.177$), each lowering resolution to roughly 0.11–0.13 from a

baseline near 0.34. The remaining six modeled motifs span one at both the raw and adjusted level (`redundancy`, `repetition`, `missing_verify`, `investigation_heavy`, `oscillating_test_fix` and `skipped_reproduce`). Three action Signals stay descriptive and outside the corrected family: `edit_test_fail_loop` and `regression` resolve no session when present, a perfect separation the model cannot fit, and `error_burst` fires in only fourteen sessions, a positive class too small to support the clustered model. Figure 7.1 reports the per-motif odds ratios, robust intervals, and BH-adjusted and raw p -values.

The total amount of struggle does not independently predict task failure.

Although total struggle severity, summed across both the action and cognitive layers, correlates negatively with successful resolution, trace length is a much stronger predictor. Severity itself is a length-driven metric. When controlling for trace length, the partial correlation between struggle severity and resolution drops to near zero (Figure 7.1). A session flagged for struggle reliably indicates a more computationally expensive interaction, its severity scaling with turns, tokens and wall-clock time. The diagnostic value is therefore the type of struggle exhibited, whereas the aggregate amount adds no independent signal. This matches the code-smell literature, where specific patterns carry measurable effects while raw volume alone does not [66].

This finding qualifies the catalog’s diagnostic claim rather than closing it. `repeated_generation`, `relocalization_loop` and `code_rollback` all point the same way, lowering the raw resolution rate and the model odds ratio below one, and carry the lowest raw p -values of the twelve motifs, but none clears a joint correction for testing nine at this sample size. The catalog therefore names the patterns worth confirming without certifying any one of them as outcome-predictive here. A larger corpus is the only way to tell a true small effect from noise at this family size. The framing still accords with Bouzenia and Pradel’s use of action patterns to separate successful and failed trajectories [43], and the same step-level premise, that fine-grained signals can reveal what a final verdict obscures [72], motivates keeping the per-motif measurement even though this corpus cannot power it.

7.5 Explaining struggle: the causal-relation layer

Prevalence and validation treat motifs as session-level counts, but the graph introduces an explanatory layer. Using rule-derived semantic-relation edges, it transforms behavioral motifs into causal chains. Three edge types, defined in Chapter 4, decompose the counts to per-event granularity. Over the 30B corpus the layer adds 160 `trigger`, 13 `invalidate` and 3 `contradict` edges, whose derivation and participating sessions Appendix B.5 inventories. Each mirrors a detector: the 3 `contradict` edges are exactly the three regression sessions, the 13 `invalidate` edges localize across five sessions the edit each failing test overturns, and the 160 `trigger` edges span the 75 sessions with a failure-to-recovery step (with `error_burst` flagging only the eight densest instances).

These chains provide the critical context that raw counts omit. For example, while the regression Signal flags that a session regressed, the relation layer reconstructs

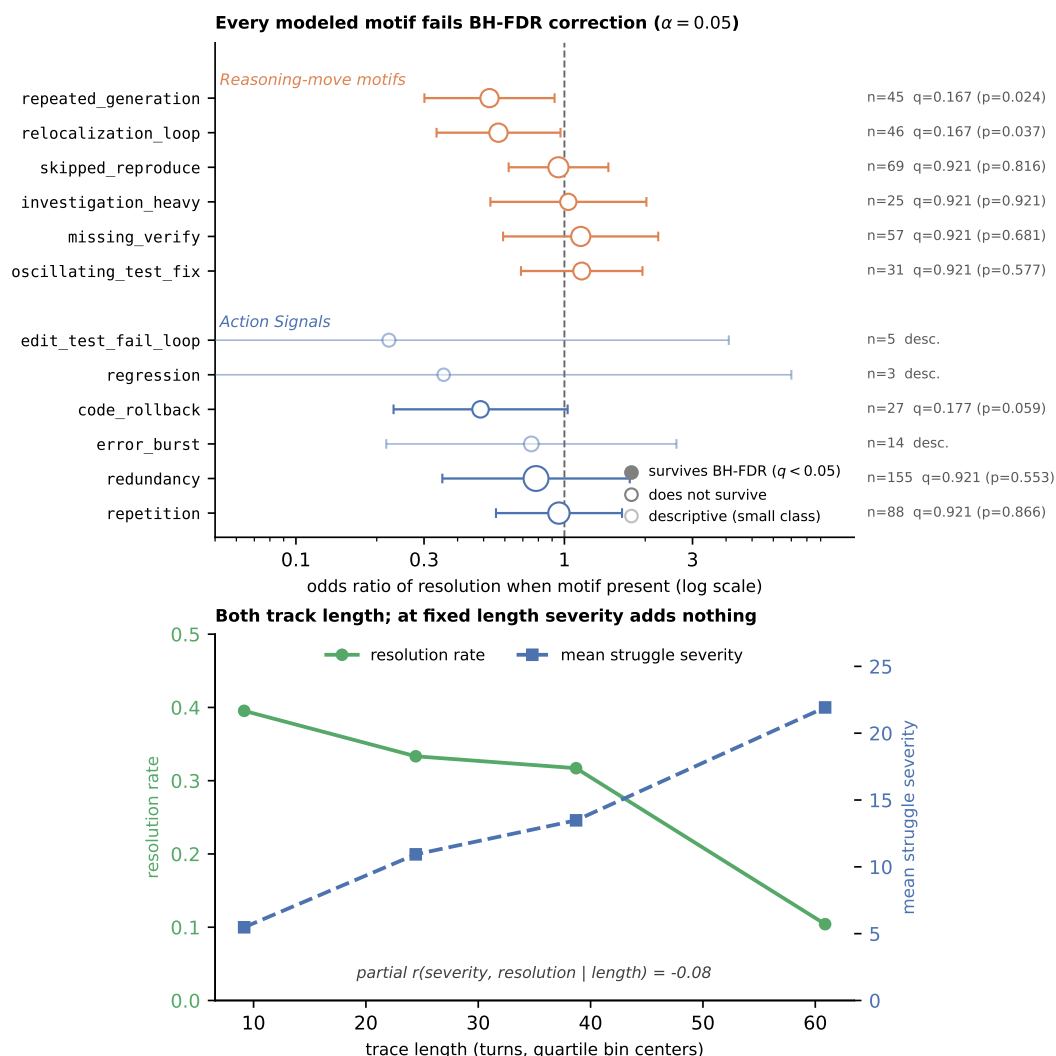


Figure 7.1: Diagnostic value across both layers, over the 171 move-enriched sessions. *Top:* population-averaged odds ratio of resolution per motif (log scale, robust 95% CI), Signals in blue and move motifs in orange, each row annotated with its BH-adjusted q and raw p . A filled marker would denote a motif whose adjusted q survives the correction across the nine motifs with a model fit ($q < 0.05$), a threshold none reaches, so every modeled motif is drawn hollow with a solid error bar, `repeated_generation` and `relocalization_loop` closest to the threshold. A hollow marker with a fainter error bar denotes a motif left descriptive by a small or separated positive class, outside the corrected family. *Bottom:* resolution rate and mean struggle severity by trace-length quartile, so length drives both while aggregate severity adds no independent signal.

the exact mechanism as a path: a source file edit `triggers` a test run, and the resulting failure both `invalidates` that edit and `contradicts` the test’s earlier pass (as depicted in Chapter 4). This allows a reader to trace a failure back to its specific edit.

This localization is highly faithful. In the five sessions with `invalidate` edges, four correctly target the module exercised by the test, while the fifth resolves to the suite root. Because this edit-and-test layer is deliberately small, the claims are correspondingly modest. However, it successfully demonstrates an explanatory capability absent in flat logs and PROV vocabularies (Chapter 4). By tracing the causal path of a failure, it directly complements the diagnostic result reported above.

7.6 Scalability under growing logs

The structural and LLM-judge detectors scale differently in cost as the corpus grows, and this section compares them on that cost under the model, provider and run configuration recorded in Appendix B.8. The corpus traces are too short to reproduce the length-driven reliability decay reported in the literature, so the comparison here is one of processing cost and not of detection reliability. The structural detectors classify every motif in a single pass over the Interaction Graph. This approach evaluates all 720 motif-cells (including 90 cross-session cells) in just 6.4 ms. The per-trace LLM judge, whose verbatim prompt and per-motif glosses appear in Appendix B.7, instead pays one model call for each of the 630 intra-session cells, at a measured 2.5 s per call and $\approx 1,597$ s in aggregate. The separation in total processing at the full corpus is about 2.5×10^5 , five orders of magnitude (Figure 7.2), and it holds regardless of concurrency. That aggregate parallelizes, since the judge’s calls are independent, so concurrency lowers wall-clock without changing the work each mechanism performs. Even a fully parallel judge is floored by one call’s latency of about 2.5 s, so the single 6.4 ms structural pass stays nearly 400 times faster in the judge’s best case.

The figures above represent query costs over an already constructed graph, and the one-time cost of building that graph is excluded from the comparison. The deterministic lift builds the 90-session corpus from its span stream in about 0.6 s, roughly 2.6 ms of graph assembly per session with no LLM call, and every later query reuses that graph. Constructing the cognitive layer does incur its own cost, requiring one move-classification call per sub-intent with the use of LLM. Independent of how many run in parallel, the work is 2,707 calls at about 1.5 s each and roughly 3,900 s of aggregate model time. The classification happens once at lift and the classified moves persist in the graph, so the action-layer query cost above stands while the cognitive motifs carry a separate construction cost.

Beyond cost, flat logs suffer from established reliability gaps. Chapter 6 showed that recognizing these motifs in a flat log requires a frontier model to ingest the full trace, while existing literature confirms that extraction reliability erodes as logs grow longer [14, 15] (Section 2.3). Since the single-agent corpus does not reach the trace lengths necessary to trigger this degradation, this section focuses on identifying the cost differences.

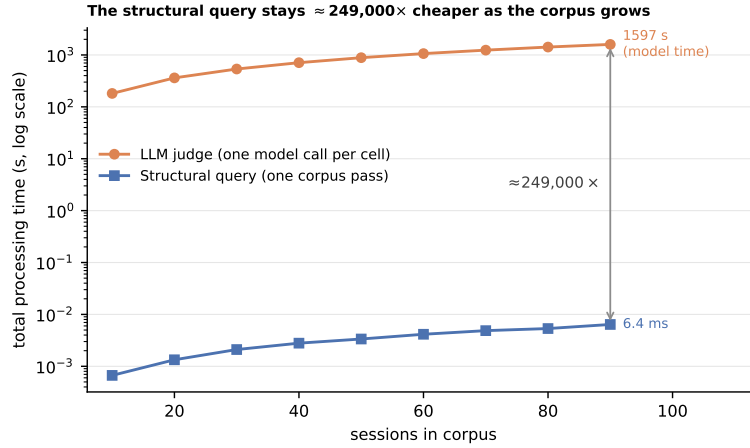


Figure 7.2: Total processing time as the corpus grows (log scale). The structural query answers all 720 cells in one pass (6.4 ms), the per-trace judge one call per cell (1,597 s of aggregate model time), roughly 2.5×10^5 times more at every size.

7.7 Summary

The Interaction Graph turns agent struggle into a diagnostic surface that can be queried directly, using a two-layer vocabulary. The action layer defines tool-action struggle motifs, while the cognitive layer adds reasoning-move motifs detailing why the agent acted. Defined by deterministic rules, these motifs were validated against a human gold standard. Applying these detectors to a stronger agent reveals a capability-dependent shift, demonstrating that the vocabulary scales effectively across models.

Crucially, this diagnostic surface points at where struggle costs resolution, even though no single motif’s association with resolution survives correction for testing nine at once at this sample size. `repeated_generation`, `relocalization_loop` and `code_rollback` carry the lowest raw p -values and the same directional story, lowering resolution when present, and remain the motifs most worth confirming in a larger corpus. Conversely, aggregate struggle counts reflect only a session’s resource consumption. By implementing a relation layer, the graph translates motifs into causal chains that point to the exact edits causing failures, a feature completely absent in flat logs. Chapter 8 uses this surface as the memory an agent retrieves across attempts.

8 Memory application

This chapter answers RQ3. It describes how the Interaction Graph is wired into a coding agent as a cross-session retrieval source and presents a feasibility pilot, with an overview of evaluation results.

8.1 Background: memory for autonomous agents

This chapter uses the Interaction Graph as a cross-session memory store, grounding it in the design space of memory for autonomous LLM agents. The core foundation for this is Du’s survey on agent memory [73], which structures the 2022–2026 literature around a write-manage-read loop spanning three axes: temporal scope, representational substrate and control policy. This matches earlier frameworks by Zhang et al., who catalog similar operations [12], and Zhou et al., who define memory as state held outside an agent’s weights [5]. The memory arms in this chapter adapt the experiential-learning line of agent memory, in which an agent distills guidance from its own past attempts, from Reflexion’s verbal self-reflection [74] through ExpeL [75] and AutoGuide [76].

On the representational axis, Du contrasts vector-indexed stores with structured stores. As a structured store, the Interaction Graph accepts an upfront schema cost to enable queries across causal, temporal and source-scoped relations. This chapter also borrows Du’s evaluation vocabulary, specifically a four-layer metric stack covering task effectiveness, memory quality, efficiency and governance. The pilot measures two of these, task effectiveness and efficiency, and treats governance structurally, as the leave-one-out access guard and the graph’s auditable provenance are properties of the design (Section 8.2), while no pilot metric measures them. The memory quality metric demands labeled relevance judgments that fall outside the scope of this feasibility pilot.

8.2 Cross-attempt retrieval design

The Interaction Graph is integrated into the agent as a same-repository, leave-one-out memory of prior attempts. "Same-repository" ensures the store contains only historical attempts on the current task’s codebase. "Leave-one-out" guarantees that every prior attempt is included except the current task itself. This allows the agent to observe how the repository was previously handled without ever accessing the solution to its active problem. The store is built offline from those prior runs and is read-only during a task: the agent retrieves from it but never writes to it. The store is served by a standalone MCP server. It exposes the named Cypher queries used in previous chapters, and a query returns the graph’s own rows with no summarizing step in between. The harness enforces the leave-one-out guard and a budget of ten retrieval calls per task: it launches one server per sample with the current instance and budget fixed in that

server’s environment, so the agent can neither name its own task nor exceed its budget.

The retrieval process is entirely agent-driven. Du’s survey identifies *control policy*, how an agent decides what to read from memory and when, as the most consequential and least-discussed memory dimension [73]. To place that decision with the agent, the retrieval queries are mounted as tools the agent calls on its own initiative, and the harness never injects prior records automatically. Three tools are available, their full contracts and glosses given in Appendix C.2:

repo_memory_summary returns a compact summary of the repository’s recurring struggle motifs and signals, the difficulty patterns of Chapter 7. For each motif, it reports the number of prior attempts that exhibited it and how many of those included a failed test run. Because this failure count relies strictly on internal test logs without consulting the benchmark’s final verdict, the tool remains evaluation-agnostic.

list_prior_attempts provides a ranked list of prior attempts alongside one-line issue summaries. It ranks these attempts based on lexical overlap (the number of shared words) between the prior issue and the current task. Since issue text is inherently public, this ranking safely avoids leaking any solutions protected by the leave-one-out guard.

attempt_query runs one named graph query against a single attempt the agent selects, for instance `move_motifs`, which returns the typed action motifs of Chapter 7.

8.3 Feasibility pilot

This evaluation is the null-tolerant feasibility pilot set out in Chapters 3 and 5: a formative episode that establishes the artifact functions as intended and de-risks a later summative study, without estimating an effect size [35, 70]. It reuses the paired bootstrap protocol fixed in Chapter 5.

The pilot compares four arms (Table 8.1): a no-memory **baseline** and three memory arms, **free**, **expel** and **guide**, that share one agent prompt and the same retrieval tools and differ only in a single *synthesis clause*, the instruction telling the agent how to convert retrieved records into guidance, reproduced verbatim for each arm in Appendix C.1 where the table paraphrases it in one line. Holding everything else constant isolates the effect of this management policy from that of retrieval.

Because the **free** arm omits synthesis instructions entirely, it directly addresses RQ3. The **free**-vs-**baseline** pairing is the pre-specified primary contrast, the only pairing with enough samples to support a statistical claim. The **expel** and **guide** arms are reported descriptively, focusing on the behavior of their synthesis strategies without claiming significant effect sizes.

The pilot evaluates a full corpus of thirty SWE-bench Verified instances [2] distributed evenly across three repositories (`django`, `sympy` and `sphinx`). To account for run-to-run variation, each instance is run three times, yielding ninety total attempts

Table 8.1: The four pilot arms, differing only in the synthesis clause that converts retrieved records into guidance.

Arm	Synthesis clause	Modeled after
baseline (S0)	no memory tools	
free	none	
expel	condense retrieved attempts into a flat list of insights	ExpeL [75]
guide	conditional “when [state], do or avoid X” rules	AutoGuide [76]

per experimental arm. The statistical unit of analysis, however, remains the instance. Averaging an instance’s three runs mitigates that variation while preserving task-level differences, effectively reducing the ninety attempts to thirty independent tasks. Because every experimental arm evaluates the same thirty tasks as the baseline, the comparison is strictly *paired*. The pilot evaluates performance task-by-task, recording whether the memory-enabled arm resolved a task the baseline failed, or the reverse. This paired design ensures that no arm can score higher simply by drawing an easier set of problems. Accordingly, the confidence intervals are calculated with a paired bootstrap that resamples the thirty instances. All evaluations use the same instruction-tuned model (qwen3-235b), which also produced the no-memory baseline, serving as both the S0 comparator and the retrieval corpus, with the model and provider pins, the bootstrap seed and the frozen log identifiers listed in Appendix C.4.

Two metric families are reported: the task *outcome* (the paired resolution rate) and the task *process*. The process is measured at two levels:

Trace level. Computable from the flat evaluation log alone, it covers trajectory length, token cost and a length-normalized redundancy rate (the fraction of an attempt’s actions that repeat an earlier (function, arguments) pair).

Diagnostic level. Supplied only by the Interaction Graph, it gives the per-instance MoveMotif and signal rates of Chapter 7, which a flat log cannot recover.

Reporting both levels tests whether trace-level repetition reaches the deeper cognitive-struggle layer. Both are computed deterministically and require no LLM judge. Retrieval telemetry, including the per-arm call rate and a retrieval-by-outcome crosstab, is detailed in Appendix C.7.

8.4 Findings

The agent reliably uses the memory on every attempt. Every one of the ninety attempts in each arm called at least one retrieval tool, so the agent used the memory every time. It made two to two-and-a-half calls on average (free 2.0, expel 2.5, guide 2.3). Every attempt read the repository summary, and most (93%) also retrieved the ranked list of prior attempts. Only a minority (14%) went further and queried a specific attempt in detail.

Table 8.2: Pilot results across the four arms, ninety attempts each (thirty instances, three epochs). Values are per-attempt means, the redundancy rate length-normalized and Resolved a count out of ninety. Paired differences from baseline in Figure 8.1.

Arm	Resolved	Redundancy rate	Turns	Tokens (k)
baseline (S0)	22/90	0.082	25.8	19.9
free	21/90	0.052	26.6	24.2
expel	23/90	0.068	34.6	26.3
guide	25/90	0.045	24.9	22.8

This usage pattern matters for reading the outcome result below. Du’s survey warns that external memory often fails because the agent *ignores* what it retrieves, or uses it only superficially [73]. The agent here read from the store on every attempt, which addresses the first concern, so any result below is not the trivial case of an unopened memory. However, the second concern, superficial engagement, is harder to dismiss: the agent read high-level summaries but rarely queried a prior attempt in depth, so it may have consulted the memory without engaging it deeply. Whether this shallow use could explain a weak outcome is examined in Chapter 9.

One correlation could mislead. Attempts that retrieve more resolve fewer tasks and repeat more actions (Table C.7), but the cause is task difficulty: a struggling agent on a hard task queries more, repeats more and resolves less, so all three rise together. The number of memory calls is therefore a sign of a hard task and does not measure how much the memory helped.

The memory does not change the net outcome. Table 8.2 reports the results across all four arms, while Figure 8.1 plots each arm’s paired difference from the baseline, alongside 95% confidence intervals for both outcome and process metrics. The differences in resolution rate compared to the baseline are minimal, and the paired bootstrap intervals cross zero for every arm (Figure 8.1, Table C.5). Consequently, there is no effect on task outcome, at the aggregate or the instance level. The few borderline instances that flip between resolved and unresolved (eleven of the thirty for *free*, seven for *expel* and ten for *guide*) do so through run-to-run variance: the arms retrieve identical content yet flip in opposite directions, the retrieved priors are near-irrelevant and the same file is edited with or without the memory (Appendix C.8). Because the no-memory baseline resolves only twenty-two of its ninety attempts, this lack of improvement is not a ceiling effect. A null result on this metric is expected by design. The pilot is not statistically powered to detect a small outcome effect, so at thirty tasks it cannot conclusively exclude one [77].

The memory-augmented arms reduce tool-call redundancy. Where the outcome is flat, the process metrics move directionally. The length-normalized redundancy rate, the share of an attempt’s actions that repeat an earlier one, falls under all three memory arms in point estimate. The *free*-vs-baseline contrast was pre-specified as the confirmatory test and stays exempt from the family correction: it reduces the rate by 0.030 (CI $[-0.054, -0.006]$, raw $p = 0.014$), an interval whose upper bound only just excludes zero. The remaining process metrics are exploratory and carry a Benjamini-Hochberg false-discovery-rate correction [64] across the eleven

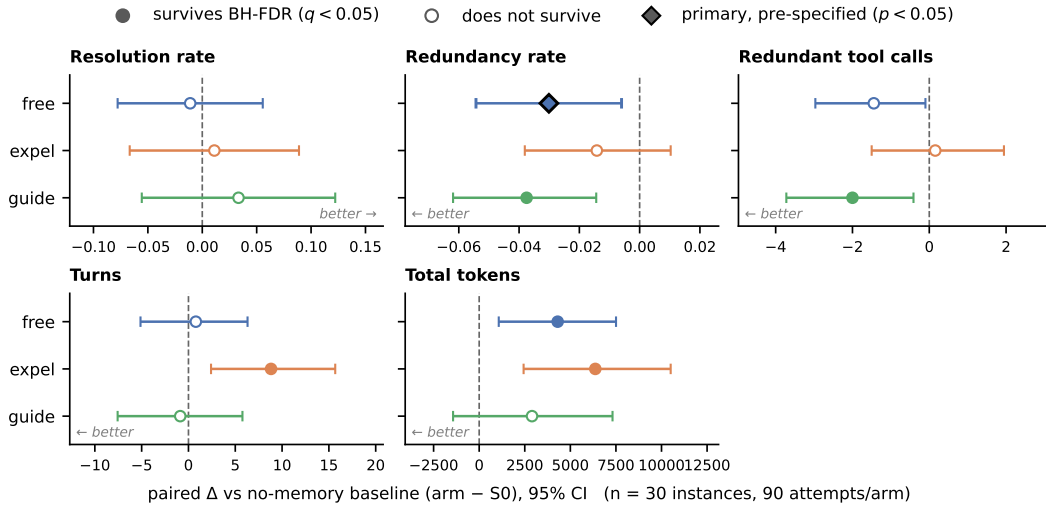


Figure 8.1: Paired difference of each memory arm from the baseline (arm – S0), with 95% bootstrap intervals over the thirty instances, for the outcome and process metrics. A filled marker denotes a contrast that survives the Benjamini-Hochberg FDR correction across the process family, the diamond the pre-specified primary contrast at raw $p < 0.05$.

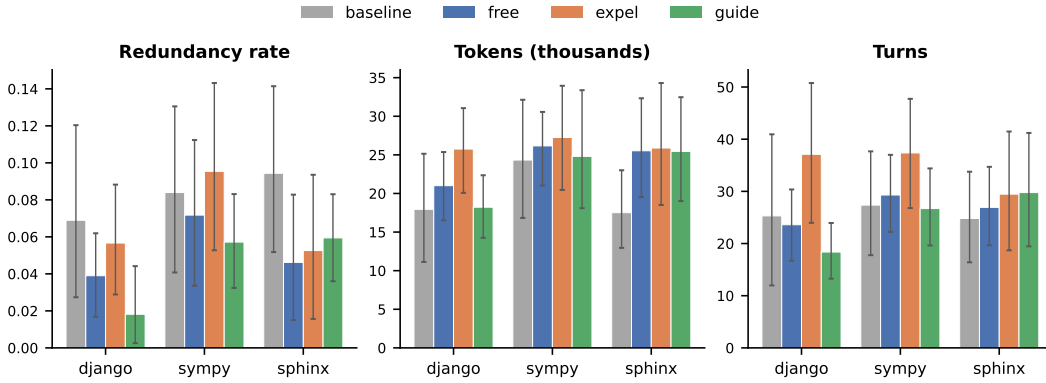


Figure 8.2: Per-repository means by arm, with 95% bootstrap intervals, for the three process metrics. Resolution is omitted.

trace-level contrasts. Within that family the **guide** redundancy-rate reduction survives the correction (BH-adjusted $p = 0.018$), while **expel**’s spans zero and does not (Table C.5). The redundancy-rate reduction is the one clear benefit the pilot records, and it is directionally consistent across the three repositories, though each per-repository interval is individually underpowered (Figure 8.2), so it does not reduce to a single-project artifact. Whether the retrieved content or the surrounding prompt drives it is unresolved, and the same-model-corpus threat is taken up in Chapter 9.

The reduction is confined to the tool-call layer. The decrease in repetition is superficial and fails to reach the deeper cognitive struggles captured by the Interaction Graph. Analyzing the higher-level reasoning patterns from Chapter 7 reveals no

significant effects across any arm (Figure 8.3). The `expel` arm shows a negligible motif-rate reduction (-0.053) with an interval crossing zero, while `free` and `guide` show no reduction. The only directional change of note is on `guide`, an increase in the rate of identical fix generation (the `repeated_generation` pattern, $+0.156$, raw $p = 0.007$), a motif Chapter 7 finds the strongest raw association with task failure among twelve, though not one that survives that chapter’s own correction for testing nine at once. Under a Benjamini-Hochberg correction across the thirty-nine typed contrasts here this signal does not survive either (BH-adjusted $p = 0.28$), and no typed motif or signal survives the correction on any arm. It stays an exploratory signal for a summative study to confirm, not a corrected effect.

The struggle signals support this. Only two signals occur frequently enough to measure: redundant reads and back-to-back near-identical commands (Appendix C.6). Neither decreases, and the repeated-reads signal rises in point estimate in the `free` and `expel` arms ($+0.078$ and $+0.089$, with intervals touching zero). This diverges from the surface metric because they measure different things: the surface metric calculates the fraction of total actions that repeat, whereas the signal strictly marks sessions that repeat a command with zero intermediate changes.

Memory only prevents the behavioral looping (repeating identical commands). It does not resolve deeper cognitive struggles, such as returning to failed reasoning paths. This critical distinction is only visible because the trajectories are lifted into the Interaction Graph. A flat evaluation log would report the positive reduction in surface commands, entirely missing the increase in identical fix generation the `guide` arm suggests on a failure-predicting motif.

The synthesis strategy shapes the trajectory. Although the three memory arms access identical content, they differ in their synthesis clauses (Table 8.1, adapting AutoGuide and ExpeL [76, 75]). As illustrated in Figure 8.4, `guide` is the leanest strategy, yielding the fewest repeated calls and steps with no token cost increase. However, this leanness incurs a penalty, as it directly drives the `guide` arm’s tendency to regenerate the same fix (the behavioral regression noted previously). Conversely, `expel` is the longest and most expensive arm. its flat list drives it to take more steps and spend more tokens (Table C.5), without achieving a corresponding reduction in repetition. Both `free` and `expel` also spend reliably more tokens than the baseline, the price of adding the retrieved memory to every prompt.

8.5 Summary

The pilot returns a null-tolerant feasibility result. The graph functions as a queryable cross-session memory, the agent adopts it at full uptake with zero abstention, and the memory arms show no detectable change in resolution, while the pre-specified `free`-vs-baseline contrast reduces the tool-call redundancy rate, a reduction directionally consistent across the three repositories, though underpowered within each, that does not reach the typed struggle motifs of Chapter 7 and is paid for in tokens. What these layer-specific results establish against Du’s metric stack [73] and the decoupling of process from outcome they expose are taken up in Chapter 9.

9 Discussion

This chapter synthesizes the three research questions. It states what the measurements collectively establish, bounds the claims with the limitations of the schema and corpus, examines the threats to validity and develops the struggle motifs into a process-quality target.

9.1 What the measurements establish

The three results form one chain. The faithfulness established in RQ1 is the precondition for the two applications. Because Chapter 6 measured the lift complete and correct, the motifs of Chapter 7 and the memory of Chapter 8 query a validated substrate without re-checking it. RQ1’s expressiveness then dictated their form, since a flat-log reader recovers those answers only unreliably (Chapter 6), so both applications are built as structural queries over the graph. RQ2 and RQ3 repay the choice from two sides, the action-layer detectors run five orders of magnitude cheaper per query than a per-trace judge, while the cognitive layer carries a one-time language-model construction cost amortized across later queries (Chapter 7) and the memory pilot’s key insight, that its surface redundancy drop does not reach the deeper struggle motifs and on one arm suggests an increase in a failure-predicting pattern, is legible only in the graph’s typed layers (Chapter 8). Collectively, these evaluations establish the central thesis: a single typed representation, lifted once from the stream, serves as a query substrate, diagnostic surface and cross-session memory without modification.

One result invites a reading beyond its chapter. Read against Du’s four-layer memory metric stack [73], the RQ3 result is layer-specific. *Governance* is the layer the representation is structurally suited to, matching Du’s account of a non-parametric store as the design that supports inspection and auditing [73]. One of Du’s three governance metrics is realizable here. Access-scope violation is controlled by construction, because each record carries the session and repository it came from, and a retrieval boundary is a filter over those two keys. The pilot fixes that filter as the same-repository, leave-one-out scope the harness sets in each server’s environment beyond the agent’s reach (Chapter 8), so the agent can neither name its own task nor widen what the server returns, and a deployment redefines the boundary by rekeying the filter to a user, team or project. Auditability follows from the same structure, since a retrieval returns the graph’s typed rows with no summarizing step and every row traces to the source span that produced it (Chapter 4), leaving the retrieved set and its ranking over public issue text open to inspection. Du’s other two metrics, privacy leakage and deletion compliance, lie outside this study, so the claim is structural fitness and the pilot measures no governance score of its own. The feasibility positive accompanies it: the agent adopted the store at full uptake, every attempt in every memory arm calling at least one retrieval tool, which excludes the unopened-memory failure mode Du’s stack identifies (Chapter 8). On *efficiency* the memory pays the token overhead the stack expects a retrieval layer to incur, reliably for free and expel at roughly

4,300 and 6,400 additional tokens per attempt. *Task effectiveness* shows no gain at this scale, and *memory quality* is left to the summative study.

The outcome null follows from what the memory offers. It surfaces how prior attempts behaved and where they struggled, while solving the task depends on locating the bug in the code, which the memory does not supply. A flat resolution rate is therefore what a process-oriented memory should be expected to produce. The one adverse move, *guide* raising the *repeated_generation* motif while lowering the tool-call redundancy rate, is consistent with its state-conditioned rules trading repeated tool calls for repeated patch generation, shifting the repetition from the action layer to the reasoning-move layer. On these independent hard bugs the bottleneck is producing a correct patch, and the memory arms only cut the repeated tool calls around that step, so a cleaner process does not convert into a better outcome. The null is therefore a boundary condition: unlike a shortfall, it separates where a process-oriented memory helps from where it does not, and directs the summative study toward more strongly related task families, where a prior attempt can genuinely inform the next. The decoupling this exposes is developed as a process-quality target in Section 9.4.

One claimed contribution lies outside this chain. The human-agent instantiation of the schema is exercised only in part by the autonomous corpus, a scope limit the next section states.

9.2 Limitations

Intent and Signal layers. Although the schema is explicitly designed for human-agent workflows, where an external human intent dynamically steers actions and evolves over a session, the evaluated corpus is entirely autonomous. Consequently, this evaluation exercises only a portion of the schema’s intended design. The Intent layer is populated and typed (Chapter 4), yet its sub-intents come from the agent’s own narration, and each action binds to the session issue alone. An external human intent revised over the session and steering specific actions does not occur in these traces. The evaluation therefore exercises the agent-environment part of the schema in full, while human-intent revision remains unexercised.

Lift heuristics. Three stages of the lift pipeline use heuristics. Their failure modes limit the completeness of the derived layers, though the underlying recorded spine remains unaffected. For instance, when a git observer is absent, artifact recovery parses write targets directly from commands. Similarly, evidence extraction uses pattern matching to distinguish genuine test failures from command errors. Struggle motifs trigger on manually inspected, untuned thresholds, while the MoveMotif layer inherits the reliability of the underlying language-model classifier (Chapter 4). Appendix C.9 reports that classifier’s fidelity and the end-to-end gap no gold yet closes for the combined classifier-and-rule layer.

Content-fingerprint coverage. The content-identity primitive is complete only over the version-controlled history. A write outside version control, such as a `.backup` copy, is never fingerprinted. This gap bounds edit-path recall at 0.80 (Chapter 6).

Sample size. Every empirical claim is bounded by the scale of the corpus, thirty

instances and ninety sessions. The rare motifs carry positive classes of three to five sessions, so their validation indicates consistency on a small sample (Chapter 7). The memory pilot is unpowered by design, so it cannot exclude a small outcome effect (Chapter 8). Both derive from the deliberate scope decision of Chapter 3, which defers a powered comparison to the summative study.

9.3 Threats to validity

External validity. Because SWE-bench frames tasks as autonomous attempts against fixed issues, the task source lacks native multi-turn context. To test the memory application, a cross-session environment is constructed by accumulating related and repeated runs on the same repository. The RQ3 feasibility pilot evaluates retrieval over this aggregated record, so its result is a lower bound on what true multi-turn, cross-session benchmarks (Section 2.3) would demand. Whether the diagnostic and memory results transfer directly to those native multi-session settings remains untested. The corpus also spans only three repositories, `django`, `sympy` and `sphinx` (Chapter 5), which further bounds how far these results generalize.

Benchmark realism is a second external-validity limit. SWE-bench’s issue descriptions are polished, and terse, telemetry-grounded prompts make the same bugs harder to resolve [48], while an interactive user-simulator setting introduces a further sim-to-real gap [50]. Reproducing these conditions lies outside the scope of this thesis, and both bound the resolution rates the memory pilot reports. The diagnostic claims are unaffected. Because the diagnostic uses only the telemetry the agent emits and the structure the lift recovers, it holds regardless of how the task is elicited.

Construct validity. Because the Interaction Graph is constructed almost entirely from the agent’s own execution trace, the primary threat is that the benchmark’s final grader verdict (the Outcome node) might be driving the applications’ performance. Aside from this verdict, every node and edge is recovered directly from the OpenTelemetry and OpenInference span stream (Chapter 5), which any deployment carrying the same instrumentation also produces. The resolved verdict is the exception. It is generated by the evaluation framework and never emitted by the agent, making it specific to both the benchmark and the harness that scores it. In a real-world scenario, no suite certifies a fix, and acceptance comes instead from continuous-integration results, human review or a downstream revert. Consequently, the Outcome node is the only element of the graph a deployment could not reproduce.

This exception does not weaken the applications, because neither of them reads the verdict. It serves only as evaluation ground truth, to score the research questions, and both applications leave it unread at runtime. The diagnostic motifs are defined over in-session Evidence and graph structure, the agent’s own test runs, command outputs and the shape of its trace. The memory ranks prior attempts by their public issue text and reads their in-session test outcomes, drawn from the agent’s own test logs. Because every signal either function consumes is one a deployment reproduces, the verdict bounds the scoring alone, so the diagnostic and memory results reflect what a deployed setting also has.

Memory-pilot confounds. Two methodological confounds bound the interpretation of the RQ3 results: one affects the single positive finding (the reduction in redundant tool calls), and one affects the interpretation of the null outcome.

The first confound is *self-recognition*. The memory store is built from the same model’s own earlier runs, so each agent reads a record of how it behaved before. A benefit could then mean one of two things. Either the memory held something the agent could learn from, genuine cross-attempt transfer, or the agent settled back into its own familiar behavior on seeing its prior run, which needs no useful content at all. The pilot cannot separate the two, and whether the memory helps a different model, one not reading itself, is untested.

The second concerns how the agent used the memory. A null outcome could mean the memory has no value, or that the agent barely used it and so never gave it a chance to help. Chapter 8 leaves this open, and the usage supports the worry. Every attempt read the high-level summary, but only 14% opened a specific prior attempt in detail, so the flat outcome could reflect shallow use, leaving a real absence of benefit unproven. The churn analysis makes the worry less likely. On the few occasions the agent did look deeper, the query came back empty or belonged to the worst-performing arm (Appendix C.8), so there was little worth finding even for a reader that dug in. Telling the two apart would need a study that forces the agent to engage deeply, on tasks whose past attempts are genuinely related, unlike the near-irrelevant priors here.

Internal validity. The representation evaluation uses two language-model judges (the functional-equivalence judge and the citation-support rater), both validated on a bounded human sample. Both clear the 0.6 reliability floor (Chapter 6), yet a larger or differently stratified sample could shift these figures. The failure of the graded attribution rater carries a sharper lesson: construct validity depends more on task formulation than on judge capability. The same model that failed to grade attribution in three degrees succeeded at rating binary entailment, showing that validation certifies the question design as much as the model itself.

A residual annotation threat persists. Both golds are now double-coded, the operation gold adjudicated between two annotators and the reasoning-move gold’s representative sample agreeing at an inter-annotator κ of 0.76 (Chapter 5), yet two annotators can still share blind spots a larger or more diverse pool would surface. A related asymmetry is that the golds used to validate the language-model judges themselves, for functional equivalence and for the reasoning-move question, are single-coded, unlike the double-coded lifting and motif golds above, so those judge-reliability figures rest on one annotator’s labels and stand as an acknowledged internal-validity limitation. Finally, the reproduction rate is sensitive to the strict directly-usable criterion. Under this rule, a flat answer that correctly identifies the relevant spans but fails to state the value is penalized as a non-reproduction. This reflects the intended strict reading, and it widens the performance gap compared to a looser criterion that would credit a resolvable pointer.

Operation taxonomy. The six-operation construct is bounded by two primary limitations. First, adjacent classes, particularly *test* versus *run*, share contested boundaries that are the cause of most human annotator disagreements (Chapter 6). The lifting-fidelity κ is therefore an agreement estimate over a taxonomy with inherent

edge cases. Second, because only one operation is recorded per command (Chapter 4), commands bundling multiple actions (e.g., an edit followed by a test) are labeled strictly by the first action. This under-represents agents that chain operations and limits κ to measuring agreement on the primary label alone, masking any secondary operations. Splitting compound commands into distinct sub-actions remains a direction for future work.

9.4 Struggle motifs as a process-quality target

The divergence exposed by the pilot, where a run improves behaviorally without altering its outcome, is not unique to RQ3. A strict pass-or-fail metric treats any outcome-neutral change as if nothing occurred. It is structurally blind to the fact that one agent might reach the same conclusion with far less wasted effort than another. The struggle motifs introduced in Chapter 7 explicitly quantify these hidden differences. Consequently, they provide the foundation for a secondary evaluation axis: process quality. This metric can be computed automatically from the execution trace and reported alongside the standard resolution rates currently dominating the field.

This dual-axis approach aligns with broader trends across the agent-evaluation literature. Recent studies on benchmark validity demonstrate that aggregate pass rates often overstate true agent capabilities, which drop when prompts are made more realistic [48]. Trajectory analyses reveal that the true divider between successful and failed runs is behavioral, for instance, repeating an action without adaptation or generating a fix without testing it [43]. These are precisely the failure modes the motif layer formalizes. Just as the reasoning-model literature has transitioned from grading only final answers to evaluating intermediate steps [78, 72], the Interaction Graph introduces this paradigm to software agents.

One caution keeps the motifs a diagnostic. Goodhart’s law warns that once a measure becomes a target, it stops measuring what it did [79]. An agent rewarded for a low `missing_verify` count, for example, could emit an empty test that runs but checks nothing, so the score improves while the behavior does not. The motifs are therefore best used as a held-out check on process quality.

10 Conclusion and future work

10.1 Conclusion

This thesis addressed a gap in the record of agent work: an agent session leaves behind its result, the code and the commit, while how the agent reached it stays unrecorded, so a run can be reread but not queried. The response is the **Interaction Graph**, a typed provenance graph for human-agent software workflows, lifted from a raw OpenTelemetry span stream, that extends the W3C PROV model [16] with seven node types specific to language-model agent runs (Intent, ToolAction, Artifact, Evidence, Outcome, Signal and MoveMotif). The artifact is the contribution, and the three research questions evaluated it in one representational role and two applied ones, each against objectives fixed in advance under the Design Science strategy of Chapter 3.

RQ1 established the representation as a faithful and expressive substrate. The lift reconstructs the corpus with complete coverage and labels each action at an agreement with human annotators strong enough to certify the graph as a reliable base before any application reads it (Chapter 6). On this base the graph strictly dominates the flat evaluation log in query expressiveness: it answers every question the flat log can and a larger set that becomes answerable only after the lift, leaving a single question beyond the reach of both. The separation widens in practice, because a language model asked to recover those answers from the flat log succeeds on only a fraction of them, improves but does not close the gap as it grows stronger, and where it does answer cannot ground the answer in the trace. Formal expressibility bounds what is *possible* while the reproduction rate measures what a reader *realizes*, and the two diverge. A scope limit qualifies this substrate claim. The schema is designed for human-agent workflows, yet the autonomous corpus exercises its human-intent portion only in part, because each action binds to the session issue without an external directive revised over the run (Chapter 9).

RQ2 turned the graph into a diagnostic surface. Recurring struggle motifs, an action layer of structural Signals and a cognitive layer of reasoning-move motifs, are expressed as deterministic subgraph queries and validated against a human gold, where a fired detector is almost never a false alarm (Chapter 7). Their diagnostic value is the type of struggle exhibited: `repeated_generation`, `relocalization_loop` and `code_rollback` lower the odds of resolution most among the twelve motifs, though none survives a joint false-discovery-rate correction for testing nine at once at this sample size, while the total volume of struggle reflects only how long and costly the session was. The action-layer queries answer in a single pass over the graph where a per-trace language-model judge pays one model call for every trace, so the query-time cost gap widens with the corpus, while the cognitive layer pays a one-time language-model construction cost amortized across later queries, and a rule-derived relation layer renders a motif as a causal chain a reader traverses from a failure back to the edit that caused it, an explanatory capability absent from flat logs and PROV alike.

RQ3 connected the graph as a cross-session memory a coding agent queries. The

feasibility pilot returns a null-tolerant result: the agent adopts the store on every attempt, and giving it the memory does not change how many tasks it resolves, the outcome null a process-oriented memory should be expected to produce, since the store surfaces how prior attempts behaved while resolving the task depends on locating a bug the memory does not supply (Chapter 8). Where the outcome is flat, the process is not. Retrieving the memory measurably cuts the agent’s repeated tool calls, yet the reduction stays on the surface and never reaches the deeper cognitive motifs, and on one arm it suggests an increase in a failure-predicting pattern, a divergence legible only in the graph’s typed layers.

These three evaluations converge on one claim: a single typed representation, lifted once from the event stream, serves as a query substrate, a diagnostic surface and a cross-session memory without modification. The unification is the contribution, joining the agent-provenance and failure-attribution literatures that Chapter 2 shows no prior system combines. Beyond the result itself, the graph makes a run’s process legible, the wasted effort and struggle a pass-or-fail metric cannot see (Chapter 9). The directions that enrich and deploy the artifact are developed next.

10.2 Future work

Two enrichments follow directly from the ontology. The first is a fine-grained change classification. The content fingerprint records that a file changed and whether it returned to an earlier state, but not the kind of change, and separating a signature change from a statement edit or a refactor would sharpen both the change provenance of the representation evaluation and the refactor motifs of the diagnostic one. The established method is a tree differencing of the before and after syntax trees [45, 80], which requires the whole-file content of each version, so it needs an evaluation run that captures file content alongside the hash the frozen corpus keeps. The second is already in the schema. Each ToolAction carries its wall-clock latency and the token usage of the model call that produced it, so cost localizes to individual actions, the action-level signal recent work argues is missing from agent trajectories [81], and a cost-localized reading of the diagnostic motifs, for example the tokens and latency a debug-thrash loop accumulates, is a direct next measurement the enriched graph supports.

The central piece is the summative study the Design Science strategy of Chapter 3 places after this thesis, the powered, naturalistic comparison whose scope decision the thesis discloses. Its design follows from the boundary the memory pilot exposed. Because a process-oriented memory can inform the next attempt only where a prior one genuinely relates to it, the study belongs on task families whose runs share structure, iterative and multi-turn work, unlike the independent SWE-bench issues here, so a retrieved attempt carries something the next can use (Chapter 9). Two confounds the pilot could not resolve shape the rest of the design. A reader model distinct from the one that produced the corpus separates genuine cross-attempt transfer from an agent merely repeating its own prior behavior on seeing it, and a protocol that forces the agent to open a related prior attempt in depth separates a weak memory from

a shallowly consulted one. That study is also where the two memory metrics left unmeasured here belong, the memory-quality score that labeled relevance judgments would supply and the privacy-leakage and deletion-compliance governance scores this feasibility pilot does not reach [73].

Three directions extend the artifact itself. A registry of validated workflow graphs would let a coding agent retrieve a vetted prior solution by structural match, the memory application of Chapter 8 promoted from a per-repository store to a shared one. The retrieval surface is already exposed as a standalone MCP server, so any MCP-capable agent can query the lifted corpus as cross-session memory without the evaluation harness, the deployment path this reuse needs. An online lift, reading a live span exporter where the present lift reads a frozen file, would build the graph during a run and expose the diagnostic motifs as the session unfolds. The schema's human-agent design, meanwhile, is only partly exercised by the autonomous corpus, since its Intent layer carries the agent's own narration without an external directive revised over a session (Chapter 9). A trace with a human or a user simulator steering the run, together with splitting a compound command into its distinct sub-actions (Chapter 9), would exercise the intent-revision and multi-operation parts of the schema the present corpus leaves idle, and would carry the evaluation to the multi-turn and cross-session benchmarks of Section 2.3, where the diagnostic and memory results can be tested at scale.

One direction reaches beyond the artifact. The struggle motifs the graph makes queryable form a second evaluation axis, process quality, computable from the trace and reportable alongside the resolution rates that currently dominate the field, carrying to software agents the step-level evaluation the reasoning-model literature already adopted [78, 72]. Developing that axis into a reported metric is the natural next step, held out as a diagnostic and never optimized against, since a measure that becomes a target stops measuring what it did [79].

Bibliography

- [1] Junwei Liu et al. “Large Language Model-Based Agents for Software Engineering: A Survey”. In: *ACM Transactions on Software Engineering and Methodology* (2025). DOI: [10.1145/3796507](https://doi.org/10.1145/3796507).
- [2] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *International Conference on Learning Representations (ICLR)*. 2024. DOI: [10.48550/arXiv.2310.06770](https://doi.org/10.48550/arXiv.2310.06770). URL: <https://arxiv.org/abs/2310.06770>.
- [3] R. Gómez et al. “Large Language Model Benchmarks: A Taxonomy of Capabilities, Scientific Quality Assessment, and Saturation Analysis”. In: *Machine Learning and Knowledge Extraction* 8.6 (2026), p. 141. DOI: [10.3390/make8060141](https://doi.org/10.3390/make8060141).
- [4] Thomas Kwa et al. *Measuring AI Ability to Complete Long Software Tasks*. arXiv preprint arXiv:2503.14499. METR. 2025. DOI: [10.48550/arXiv.2503.14499](https://doi.org/10.48550/arXiv.2503.14499). URL: <https://arxiv.org/abs/2503.14499>.
- [5] Chenyu Zhou et al. *Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering*. arXiv preprint arXiv:2604.08224. 2026. DOI: [10.48550/arXiv.2604.08224](https://doi.org/10.48550/arXiv.2604.08224). URL: <https://arxiv.org/abs/2604.08224>.
- [6] Anthropic. *How Claude Remembers Your Project: Claude Code Memory*. Anthropic Documentation. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code/memory> (visited on 07/28/2026).
- [7] OpenAI. *Memories: Codex Documentation*. OpenAI Codex Documentation. 2026. URL: <https://developers.openai.com/codex/memories> (visited on 07/28/2026).
- [8] Charles Packer et al. *MemGPT: Towards LLMs as Operating Systems*. arXiv preprint arXiv:2310.08560. 2023. DOI: [10.48550/arXiv.2310.08560](https://doi.org/10.48550/arXiv.2310.08560). URL: <https://arxiv.org/abs/2310.08560>.
- [9] Prateek Chhikara et al. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv preprint arXiv:2504.19413. 2025. DOI: [10.48550/arXiv.2504.19413](https://doi.org/10.48550/arXiv.2504.19413). URL: <https://arxiv.org/abs/2504.19413>.
- [10] Preston Rasmussen et al. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv preprint arXiv:2501.13956. 2025. DOI: [10.48550/arXiv.2501.13956](https://doi.org/10.48550/arXiv.2501.13956). URL: <https://arxiv.org/abs/2501.13956>.
- [11] Shirui Pan et al. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. In: *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), pp. 3580–3599. DOI: [10.1109/TKDE.2024.3352100](https://doi.org/10.1109/TKDE.2024.3352100).
- [12] Zeyu Zhang et al. “A Survey on the Memory Mechanism of Large Language Model-Based Agents”. In: *ACM Transactions on Information Systems* (2025). DOI: [10.1145/3748302](https://doi.org/10.1145/3748302).

- [13] Mathis Pink et al. *Position: Episodic Memory is the Missing Piece for Long-Term LLM Agents*. arXiv preprint arXiv:2502.06975. 2025. DOI: [10.48550/arXiv.2502.06975](https://doi.org/10.48550/arXiv.2502.06975). URL: <https://arxiv.org/abs/2502.06975>.
- [14] Shaokun Zhang et al. “Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems”. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. arXiv:2505.00212. 2025. DOI: [10.48550/arXiv.2505.00212](https://doi.org/10.48550/arXiv.2505.00212). URL: <https://arxiv.org/abs/2505.00212>.
- [15] Mengzhuo Chen et al. *Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems*. arXiv preprint arXiv:2604.22708. 2026. DOI: [10.48550/arXiv.2604.22708](https://doi.org/10.48550/arXiv.2604.22708). URL: <https://arxiv.org/abs/2604.22708>.
- [16] Luc Moreau and Paolo Missier. *PROV-DM: The PROV Data Model*. W3C Recommendation. World Wide Web Consortium (W3C), 2013. URL: <https://www.w3.org/TR/prov-dm/> (visited on 07/28/2026).
- [17] Ken Peffers et al. “A Design Science Research Methodology for Information Systems Research”. In: *Journal of Management Information Systems* 24.3 (2008), pp. 45–77. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
- [18] Renan Souza et al. “PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows”. In: *Proceedings of the 21st IEEE International Conference on e-Science (e-Science)*. 2025. DOI: [10.1109/eScience65000.2025.00093](https://doi.org/10.1109/eScience65000.2025.00093).
- [19] Renan Souza et al. “LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology”. In: *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops)*. 2025. DOI: [10.1145/3731599.3767582](https://doi.org/10.1145/3731599.3767582).
- [20] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium (W3C), 2013. URL: <https://www.w3.org/TR/prov-o/> (visited on 07/28/2026).
- [21] Luc Moreau and Paolo Missier. *PROV-N: The Provenance Notation*. W3C Recommendation. World Wide Web Consortium (W3C), 2013. URL: <https://www.w3.org/TR/prov-n/> (visited on 07/28/2026).
- [22] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. “Provenance in Databases: Why, How, and Where”. In: *Foundations and Trends in Databases* 1.4 (2009), pp. 379–474. DOI: [10.1561/19000000006](https://doi.org/10.1561/19000000006).
- [23] Nilesh Jain. *Enhancing Data Integrity through Provenance Tracking in Semantic Web Frameworks*. Applications Track paper, experience report on PROV-DM/PROV-O practice at SURROUND Australia. Jan. 2025. DOI: [10.48550/arXiv.2501.09029](https://doi.org/10.48550/arXiv.2501.09029). arXiv: 2501.09029 [cs.CR]. URL: <https://arxiv.org/abs/2501.09029>.

- [24] Dirk Fahland. *Extracting and Pre-Processing Event Logs*. Study material for an advanced lecture in a process-mining course, Eindhoven University of Technology. Nov. 2022. DOI: [10.48550/arXiv.2211.04338](https://doi.org/10.48550/arXiv.2211.04338). arXiv: [2211.04338](https://arxiv.org/abs/2211.04338) [cs.DS]. URL: <https://arxiv.org/abs/2211.04338>.
- [25] OpenTelemetry Authors. *Semantic Conventions for Generative AI Systems*. Cloud Native Computing Foundation, OpenTelemetry project. 2026. URL: <https://github.com/open-telemetry/semantic-conventions-genai> (visited on 06/17/2026).
- [26] Arize AI. *OpenInference: A Specification and Conventions for Tracing AI Applications*. Arize AI. 2026. URL: <https://github.com/Arize-ai/openinference> (visited on 06/17/2026).
- [27] Vincent Koc et al. *Mind the Metrics: Patterns for Telemetry-Aware In-IDE AI Application Development using Model Context Protocol (MCP)*. 2025. DOI: [10.48550/arXiv.2506.11019](https://doi.org/10.48550/arXiv.2506.11019). arXiv: [2506.11019](https://arxiv.org/abs/2506.11019) [cs.SE]. URL: <https://arxiv.org/abs/2506.11019>.
- [28] B. Zhang et al. *An Empirical Study of Interaction Smells in Multi-Turn Human-LLM Collaborative Code Generation*. arXiv preprint arXiv:2603.09701. 2026. DOI: [10.48550/arXiv.2603.09701](https://doi.org/10.48550/arXiv.2603.09701). URL: <https://arxiv.org/abs/2603.09701>.
- [29] Mert Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* arXiv preprint arXiv:2503.13657. 2025. DOI: [10.48550/arXiv.2503.13657](https://doi.org/10.48550/arXiv.2503.13657). URL: <https://arxiv.org/abs/2503.13657>.
- [30] Qixing Zhou et al. “FeatureBench: Benchmarking Agentic Coding for Complex Feature Development”. In: *International Conference on Learning Representations (ICLR)*. arXiv:2602.10975. 2026. DOI: [10.48550/arXiv.2602.10975](https://doi.org/10.48550/arXiv.2602.10975). URL: <https://arxiv.org/abs/2602.10975>.
- [31] Sizhe Wang et al. *CodeFlowBench: A Multi-Turn, Iterative Benchmark for Complex Code Generation*. arXiv preprint arXiv:2504.21751. 2026. DOI: [10.48550/arXiv.2504.21751](https://doi.org/10.48550/arXiv.2504.21751). URL: <https://arxiv.org/abs/2504.21751>.
- [32] Ruchit Rawal et al. *Benchmarking Correctness and Security in Multi-Turn Code Generation*. arXiv preprint arXiv:2510.13859. 2025. DOI: [10.48550/arXiv.2510.13859](https://doi.org/10.48550/arXiv.2510.13859). URL: <https://arxiv.org/abs/2510.13859>.
- [33] Jie Qiu et al. *LoCoBench-Agent: An Interactive Benchmark for LLM Agents in Long-Context Software Engineering*. arXiv preprint arXiv:2511.13998. 2025. DOI: [10.48550/arXiv.2511.13998](https://doi.org/10.48550/arXiv.2511.13998). URL: <https://arxiv.org/abs/2511.13998>.
- [34] Alan R. Hevner et al. “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: [10.2307/25148625](https://doi.org/10.2307/25148625).

- [35] John Venable, Jan Pries-Heje, and Richard Baskerville. “FEDS: A Framework for Evaluation in Design Science Research”. In: *European Journal of Information Systems* 25.1 (2016), pp. 77–89. DOI: [10.1057/ejis.2014.36](https://doi.org/10.1057/ejis.2014.36).
- [36] Ken Peffers et al. “Design Science Research Evaluation”. In: *Design Science Research in Information Systems: Advances in Theory and Practice (DESRIST)*. Vol. 7286. Lecture Notes in Computer Science. 2012, pp. 398–410. DOI: [10.1007/978-3-642-29863-9_29](https://doi.org/10.1007/978-3-642-29863-9_29).
- [37] UK AI Security Institute. *Inspect: An Open-Source Framework for Large Language Model Evaluations*. UK AI Security Institute. 2024. URL: <https://inspect.aisi.org.uk/> (visited on 07/01/2026).
- [38] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: [2210.03629 \[cs.CL\]](https://arxiv.org/abs/2210.03629).
- [39] Andrew J. Ko, Htet Htet Aung, and Brad A. Myers. “Eliciting Design Requirements for Maintenance-Oriented IDEs: A Detailed Study of Corrective and Perfective Maintenance Tasks”. In: *Proceedings of the 27th International Conference on Software Engineering (ICSE)*. 2005. DOI: [10.1145/1062455.1062492](https://doi.org/10.1145/1062455.1062492).
- [40] Mik Kersten and Gail C. Murphy. “Using Task Context to Improve Programmer Productivity”. In: *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 2006. DOI: [10.1145/1181775.1181777](https://doi.org/10.1145/1181775.1181777).
- [41] John Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. Advances in Neural Information Processing Systems (NeurIPS); arXiv:2405.15793. 2024. DOI: [10.48550/arXiv.2405.15793](https://doi.org/10.48550/arXiv.2405.15793). URL: <https://arxiv.org/abs/2405.15793>.
- [42] Model Context Protocol. *Server Features: Tools (Tool Annotations)*. Model Context Protocol specification, version 2025-11-25. 2025. URL: <https://modelcontextprotocol.io/specification/2025-11-25/server/tools> (visited on 07/28/2026).
- [43] Islem Bouzenia and Michael Pradel. *Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories*. arXiv preprint arXiv:2506.18824. To appear, IEEE/ACM International Conference on Automated Software Engineering (ASE). 2025. DOI: [10.48550/arXiv.2506.18824](https://doi.org/10.48550/arXiv.2506.18824). URL: <https://arxiv.org/abs/2506.18824>.
- [44] Ram Chillarege et al. “Orthogonal Defect Classification—A Concept for In-Process Measurements”. In: *IEEE Transactions on Software Engineering* 18.11 (1992), pp. 943–956. DOI: [10.1109/32.177364](https://doi.org/10.1109/32.177364).
- [45] Beat Fluri and Harald C. Gall. “Classifying Change Types for Qualifying Change Couplings”. In: *14th IEEE International Conference on Program Comprehension (ICPC)*. 2006. DOI: [10.1109/ICPC.2006.16](https://doi.org/10.1109/ICPC.2006.16).

- [46] Stanislav Levin and Amiram Yehudai. “Boosting Automatic Commit Classification into Maintenance Activities by Utilizing Source Code Changes”. In: *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. 2017, pp. 97–106. DOI: [10.1145/3127005.3127016](https://doi.org/10.1145/3127005.3127016).
- [47] Stephan Rabanser et al. *Towards a Science of AI Agent Reliability*. arXiv preprint arXiv:2602.16666. 2026. DOI: [10.48550/arXiv.2602.16666](https://doi.org/10.48550/arXiv.2602.16666). URL: <https://arxiv.org/abs/2602.16666>.
- [48] Spandan Garg, Benjamin Steenhoeck, and Yufan Huang. *Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation*. arXiv preprint arXiv:2510.08996. To appear in CAIN ’26. 2026. DOI: [10.48550/arXiv.2510.08996](https://doi.org/10.48550/arXiv.2510.08996). URL: <https://arxiv.org/abs/2510.08996>.
- [49] Shunyu Yao et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. 2024. DOI: [10.48550/arXiv.2406.12045](https://doi.org/10.48550/arXiv.2406.12045). URL: <https://arxiv.org/abs/2406.12045>.
- [50] Xuhui Zhou et al. *Mind the Sim2Real Gap in User Simulation for Agentic Tasks*. arXiv preprint arXiv:2603.11245. 2026. DOI: [10.48550/arXiv.2603.11245](https://doi.org/10.48550/arXiv.2603.11245). URL: <https://arxiv.org/abs/2603.11245>.
- [51] Ron Artstein and Massimo Poesio. “Inter-Coder Agreement for Computational Linguistics”. In: *Computational Linguistics* 34.4 (2008), pp. 555–596. DOI: [10.1162/coli.07-034-R2](https://doi.org/10.1162/coli.07-034-R2).
- [52] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. “ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks”. In: *Proceedings of the National Academy of Sciences* 120.30 (2023), e2305016120. DOI: [10.1073/pnas.2305016120](https://doi.org/10.1073/pnas.2305016120).
- [53] Lianmin Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2306.05685. 2023. DOI: [10.48550/arXiv.2306.05685](https://doi.org/10.48550/arXiv.2306.05685). URL: <https://arxiv.org/abs/2306.05685>.
- [54] Michael Grüninger and Mark S. Fox. “Methodology for the Design and Evaluation of Ontologies”. In: *Proceedings of the Workshop on Basic Ontological Issues in Knowledge Sharing (IJCAI-95)*. Montreal, Canada, 1995. URL: <http://www.eil.utoronto.ca/wp-content/uploads/enterprise-modelling/papers/gruninger-ijcai95.pdf> (visited on 07/28/2026).
- [55] Denny Vrandečić. “Ontology Evaluation”. In: *Handbook on Ontologies*. 2nd. Springer, 2009, pp. 293–313. DOI: [10.1007/978-3-540-92673-3_13](https://doi.org/10.1007/978-3-540-92673-3_13).
- [56] E. F. Codd. “Relational Completeness of Data Base Sublanguages”. In: *Data Base Systems*. Vol. 6. Courant Computer Science Symposia Series. Prentice-Hall, 1972, pp. 65–98.

- [57] Zhilin Yang et al. “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2018, pp. 2369–2380. DOI: [10.18653/v1/D18-1259](https://doi.org/10.18653/v1/D18-1259).
- [58] Hannah Rashkin et al. “Measuring Attribution in Natural Language Generation Models”. In: *Computational Linguistics* 49.4 (2023), pp. 777–840. DOI: [10.1162/coli_a_00490](https://doi.org/10.1162/coli_a_00490).
- [59] Tianyu Gao et al. “Enabling Large Language Models to Generate Text with Citations”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023, pp. 6465–6488. DOI: [10.18653/v1/2023.emnlp-main.398](https://doi.org/10.18653/v1/2023.emnlp-main.398). URL: <https://aclanthology.org/2023.emnlp-main.398/>.
- [60] Or Honovich et al. “TRUE: Re-evaluating Factual Consistency Evaluation”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2022, pp. 3905–3920. DOI: [10.18653/v1/2022.naacl-main.287](https://doi.org/10.18653/v1/2022.naacl-main.287). URL: <https://aclanthology.org/2022.naacl-main.287/>.
- [61] Alvan R. Feinstein and Domenic V. Cicchetti. “High agreement but low kappa: I. The problems of two paradoxes”. In: *Journal of Clinical Epidemiology* 43.6 (1990), pp. 543–549. DOI: [10.1016/0895-4356\(90\)90158-L](https://doi.org/10.1016/0895-4356(90)90158-L).
- [62] Kilem L. Gwet. “Computing inter-rater reliability and its variance in the presence of high agreement”. In: *British Journal of Mathematical and Statistical Psychology* 61.1 (2008), pp. 29–48. DOI: [10.1348/000711006X126600](https://doi.org/10.1348/000711006X126600).
- [63] Kung-Yee Liang and Scott L. Zeger. “Longitudinal Data Analysis Using Generalized Linear Models”. In: *Biometrika* 73.1 (1986), pp. 13–22. DOI: [10.1093/biomet/73.1.13](https://doi.org/10.1093/biomet/73.1.13).
- [64] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: [10.1111/j.2517-6161.1995.tb02031.x](https://doi.org/10.1111/j.2517-6161.1995.tb02031.x).
- [65] Foutse Khomh et al. “An exploratory study of the impact of antipatterns on class change- and fault-proneness”. In: *Empirical Software Engineering* 17.3 (2012), pp. 243–275. DOI: [10.1007/s10664-011-9171-y](https://doi.org/10.1007/s10664-011-9171-y).
- [66] Tracy Hall et al. “Some Code Smells Have a Significant but Small Effect on Faults”. In: *ACM Transactions on Software Engineering and Methodology* 23.4 (2014), pp. 33:1–33:39. DOI: [10.1145/2629648](https://doi.org/10.1145/2629648).
- [67] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173. DOI: [10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638).

- [68] Cheng-Ping Hsieh et al. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *Conference on Language Modeling (COLM)*. arXiv:2404.06654. 2024. DOI: [10.48550/arXiv.2404.06654](https://arxiv.org/abs/2404.06654). URL: <https://arxiv.org/abs/2404.06654>.
- [69] Kiran Kate et al. *LongFuncEval: Measuring the Effectiveness of Long Context Models for Function Calling*. arXiv preprint arXiv:2505.10570. 2025. DOI: [10.48550/arXiv.2505.10570](https://arxiv.org/abs/2505.10570). URL: <https://arxiv.org/abs/2505.10570>.
- [70] Lehana Thabane et al. “A Tutorial on Pilot Studies: The What, Why and How”. In: *BMC Medical Research Methodology* 10 (2010), p. 1. DOI: [10.1186/1471-2288-10-1](https://doi.org/10.1186/1471-2288-10-1).
- [71] Yifei Li et al. *AttributionBench: How Hard is Automatic Attribution Evaluation?* arXiv preprint arXiv:2402.15089. 2024. DOI: [10.48550/arXiv.2402.15089](https://arxiv.org/abs/2402.15089). URL: <https://arxiv.org/abs/2402.15089>.
- [72] Hunter Lightman et al. *Let’s Verify Step by Step*. arXiv preprint arXiv:2305.20050. 2023. DOI: [10.48550/arXiv.2305.20050](https://arxiv.org/abs/2305.20050). URL: <https://arxiv.org/abs/2305.20050>.
- [73] Pengfei Du. *Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers*. arXiv preprint arXiv:2603.07670. 2026. DOI: [10.48550/arXiv.2603.07670](https://arxiv.org/abs/2603.07670). URL: <https://arxiv.org/abs/2603.07670>.
- [74] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2303.11366. 2023. DOI: [10.48550/arXiv.2303.11366](https://arxiv.org/abs/2303.11366). URL: <https://arxiv.org/abs/2303.11366>.
- [75] Andrew Zhao et al. “ExpeL: LLM Agents Are Experiential Learners”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 17. 2024, pp. 19632–19642. DOI: [10.1609/aaai.v38i17.29936](https://doi.org/10.1609/aaai.v38i17.29936).
- [76] Yao Fu et al. “AutoGuide: Automated Generation and Selection of Context-Aware Guidelines for Large Language Model Agents”. In: *Advances in Neural Information Processing Systems 37 (NeurIPS)*. 2024, pp. 119919–119948. DOI: [10.52202/079017-3811](https://doi.org/10.52202/079017-3811).
- [77] Douglas G. Altman and J. Martin Bland. “Statistics Notes: Absence of Evidence Is Not Evidence of Absence”. In: *BMJ* 311.7003 (1995), p. 485. DOI: [10.1136/bmj.311.7003.485](https://doi.org/10.1136/bmj.311.7003.485).
- [78] Jonathan Uesato et al. *Solving Math Word Problems with Process- and Outcome-Based Feedback*. arXiv preprint arXiv:2211.14275. 2022. DOI: [10.48550/arXiv.2211.14275](https://arxiv.org/abs/2211.14275). URL: <https://arxiv.org/abs/2211.14275>.
- [79] David Manheim and Scott Garrabrant. *Categorizing Variants of Goodhart’s Law*. arXiv preprint arXiv:1803.04585. 2018. DOI: [10.48550/arXiv.1803.04585](https://arxiv.org/abs/1803.04585). URL: <https://arxiv.org/abs/1803.04585>.

- [80] Beat Fluri et al. “Change Distilling: Tree Differencing for Fine-Grained Source Code Change Extraction”. In: *IEEE Transactions on Software Engineering* 33.11 (2007), pp. 725–743. DOI: [10.1109/TSE.2007.70731](https://doi.org/10.1109/TSE.2007.70731).
- [81] Boqin Yuan, Yue Su, Renchu Song, et al. *ClawTrace: Cost-Aware Tracing for LLM Agent Skill Distillation*. arXiv preprint arXiv:2604.23853. 2026. DOI: [10.48550/arXiv.2604.23853](https://doi.org/10.48550/arXiv.2604.23853). URL: <https://arxiv.org/abs/2604.23853>.

Appendix A RQ1

query-expressiveness

reproducibility

This appendix records the competency questions and, for each, the class and query key that names its graph query. It also records the per-question Layer-2 and Layer-3 results behind the summaries of Chapter 6 and the prompts of the flat-log answerer and the two judges. The full Cypher for every query key lives in the submitted source (code/src/ig/queries.py, the CATALOGUE dict) and is not reproduced here.

A.1 Competency questions and their graph queries

Each question below is followed by its query key and kind (lookup or traversal). The key names the entry in code/src/ig/queries.py (CATALOGUE) whose Cypher answers it.

A.1.1 Native questions

Which tool calls did each assistant turn issue? List them per turn.

assistant_turn_tool_calls (lookup)

How many tool calls did the agent make in this session?

tool_call_count (lookup)

A.1.2 Derivable questions

How did the agent spend its actions across operation classes (edit/test/read/search/run/submit)?

operation_distribution (lookup)

Reading this session, did the agent show each of these struggle patterns? Answer yes or no for each: (1) redundancy – repeating an action whose effect an earlier action already achieved; (2) repetition – three or more consecutive identical actions; (3) code rollback – an edit that undoes a prior edit to the same file; (4) error burst – three or more consecutive failing or erroring actions; (5) edit-test-fail loop – an edit followed by a failing test, repeated; (6) regression – a test that passed earlier and then failed after an intervening edit.

struggle_signals (lookup)

How many test runs passed versus failed in this session?

test_outcome_summary (lookup)

What is the version-derivation chain of a file edited several times?

artifact_version_lineage (traversal)

Which files were touched across more than one session of the same repo?

cross_session_repo_recurrence (traversal)

Which file reached an identical content state in more than one session of the same repo?

cross_session_same_state (traversal)

Which failing tests are causally downstream of an earlier edit in the same session?

edit_test_fail_motif (traversal)

What observational evidence did the agent's tool actions produce in this session?

List each piece of evidence.

evidence_for_action (traversal)

Which edited artifacts trace back to the session's stated goal?

intent_to_artifact (traversal)

Which multi-edit file ended with the same content it started with (churn that achieved nothing)?

no_net_change (traversal)

Which test target passed earlier and then failed after an intervening edit?

regression (traversal)

What is the full action spine – the ordered sequence of action types (search/read/edit/test/run/...) – the session followed?

resolving_provenance_trail (traversal)

Reading this session, for each of these struggle patterns that the agent showed, list the specific tool actions that were part of it (answer 'none' for a pattern that did not occur): (1) redundancy – repeating an action whose effect was already achieved; (2) repetition – consecutive identical actions; (3) code rollback – an edit undoing a prior edit; (4) error burst – consecutive failing actions; (5) edit-test-fail loop – repeated edit-then-failing-test; (6) regression – a test that passed then failed after an edit.

signal_implicated_actions (traversal)

Which per-turn sub-goals did the session pursue under its issue, and which tool actions carry out each turn?

sub_goal_decomposition (traversal)

A.1.3 Impossible questions

What was the agent's internal reasoning between two tool calls that it never emitted as text?

reasoning_content_recovery (lookup)

No graph query: the required primitive lies outside both representations, so the question is impossible rather than answered.

A.2 Prompts

A.2.1 Flat-log answerer (base variant)

Reproduced verbatim from code/src/rq1/answer.py (_FLAT_PROMPT).

```
1 Answer a question about ONE autonomous coding-agent session, using ONLY the flat evaluation log
  below (the transcript, the raw tool spans, and the final grader score). Do not assume any
  structure that is not present in the text.
2
3 Question: {question}
4
5 Give a SHORT, DIRECT answer: just the value, list, count, or yes/no the question asks for -- no
  explanation, reasoning, or restating the question. Then on a final line cite the span ids
  or transcript indices that support it as JSON:
6 CITED: ["span_ab12", "[3]"]
7
8 # Flat evaluation log
9 {log}
```

A.2.2 Functional-equivalence judge

Abridged from code/src/rq1/judge.py (_EQUIV_PROMPT). One boolean-metadata bullet and one further worked example are omitted here for space.

```
1 Decide whether A_flat gives the SAME answer to the question as A_IG (computed from a typed
  graph -- treat it as the correct reference).
2
3 Method: FIRST identify the CORE answer each gives to the question (the value, yes/no, list,
  count, or per-item mapping it asserts). THEN compare ONLY those core answers.
4 - If the core answers AGREE, reply equivalent=true -- even if A_flat adds extra explanation,
  reasoning, caveats, or detail. Extra words NEVER make it non-equivalent.
5 - Reply equivalent=false ONLY if the core answers disagree, A_flat asserts a wrong value (e.g.
  a different count or set), or A_flat does not answer the question.
6 - DIRECTLY USABLE: the core answer must state the asked-for value, list, count or mapping
  itself. An answer that only points at raw identifiers -- span ids (e.g. 0x3fla...),
  transcript indices, or file line numbers -- that would still have to be resolved against
  the log to obtain the asked-for value does NOT count; reply equivalent=false.
7 - PER-ITEM BREAKDOWN: if the question asks for a mapping per item (per turn, per file, per
  signal, per action), A_flat must give that per-item mapping. An answer that collapses it
  to an aggregate, a count, or the deduplicated SET of distinct values is STRICTLY WEAKER
  and does NOT answer the question -> equivalent=false.
8 - EQUIVALENT REFINEMENT / NOTATION: reply equivalent=true when A_flat states the same items as
  A_IG but (a) at a finer granularity consistent with and contained in A_IG's answer (e.g.
  naming the specific failing test function that belongs to A_IG's test module/target), or (
  b) in a different information-preserving notation for the same items (e.g. 0-indexed vs 1-
  indexed positions, a file path vs the id that denotes it) -- PROVIDED the set, count and
  order of items match. A difference of indexing base or notation ALONE never makes it non-
  equivalent.
9 - STRUGGLE MOTIFS (struggle_signals / signal_implicated_actions): A_flat is a per-pattern
  detection -- a yes/no (or an action list) for each named pattern (redundancy, repetition,
  code_rollback, error_burst, edit_test_fail_loop, regression). A_IG lists ONLY the patterns
  that fired; any pattern not in A_IG is a "no". Treat equivalent iff the SET of patterns
  A_flat marks present equals the set A_IG reports -- a missed pattern or an invented one
  makes it not-equivalent. Map the question's friendly names to A_IG's labels ("edit-test-
  fail loop" == edit_test_fail_loop, "code rollback" == code_rollback). Ignore severity. For
  signal_implicated_actions, additionally require that, for each present pattern, the
  actions A_flat lists plausibly correspond to A_IG's implicated actions (by what they do,
  not by id).
10
11 Worked examples:
12 (1) Question: "Did this session resolve the issue?"
13 A_IG: [{"resolved": false}]
```

```

14 A_flat: "No, this session did not resolve the issue, although the agent analyzed the problem
    and made several edits."
15 Core answers: A_IG = NO, A_flat = NO. They agree -> equivalent=true. (The clause about edits
    is extra detail and is irrelevant.)
16 (2) Question: "Which tool calls did each assistant turn issue?"
17 A_IG: per-turn tool names, e.g. turn 1 -> bash, turn 2 -> text_editor.
18 A_flat: "0x74739d4167ee4183, 0x0217afeb449cee3b" (raw span ids).
19 The span ids are not the asked-for tool names and would need resolving against the log, so
    the answer is not directly usable -> equivalent=false.
20 (3) Question: "Which tool calls did each assistant turn issue?"
21 A_IG: per-turn mapping -- turn 1 -> [bash], turn 2 -> [text_editor], turn 3 -> [bash], ...
22 A_flat: "bash, text_editor"
23 A_flat gives only the SET of distinct tools, not the per-turn mapping the question asks for
    -- strictly weaker -> equivalent=false.
24 (4) Question: "Which test target passed earlier and then failed after an edit?"
25 A_IG: [{"target": "tests.test_domain_py"}]
26 A_flat: "test_pyfunction_signature_full_py38"
27 A_flat names the specific failing test function inside A_IG's module -- a consistent
    refinement of the same regression (count matches) -> equivalent=true.
28
29 Question: {question}
30
31 A_IG (reference, correct):
32 {a_ig}
33
34 A_flat (to judge):
35 {a_flat}
36
37 Reply with exactly one JSON object:
38 {"equivalent": true or false, "rationale": "<one short sentence naming each core answer>"}

```

A.2.3 Entailment citation-support rater

Reproduced verbatim from code/scripts/rq1_entail_smoke.py (ENTAIL_PROMPT), the binary-entailment rater that supersedes the earlier graded attribution prompt.

```

1 You are checking ATTRIBUTION by strict textual ENTAILMENT (NLI), the way ALCE / TRUE evaluate
  citations.
2
3 Given an ANSWER and the CITED EVIDENCE the author pointed to, decide one thing only: does the
  cited evidence ENTAIL the answer -- can the answer be derived from the cited text ALONE,
  using no outside knowledge?
4
5 - entailed=true ONLY if the cited text states or directly implies the answer's content, so a
  reader could reconstruct the answer from the citation alone.
6 - entailed=false if the citation is merely ON THE SAME TOPIC, only PARTIALLY supports the
  answer, or needs information not present in the cited text. Topical relevance is NOT
  entailment. Partial support is NOT entailment.
7 - Judge ONLY whether the cited text entails the stated answer. Do NOT judge whether the answer
  is a good response to any question, and do NOT use outside knowledge.
8
9 ANSWER:
10 {a_flat}
11
12 CITED EVIDENCE:
13 {cited}
14
15 Reply with JSON only: {"entailed": true, "rationale": "<one sentence>"}

```

Table A.1: Model, provider and decoding configuration for the RQ1 readers and judges. All calls go through OpenRouter at temperature 0 with `allow_fallbacks=false`. (*default*) marks a provider whose quantization is not separately pinned, and *Reason.* marks reasoning-enabled decoding.

Role	Model	Provider (quant)	Max tok / Reason.
Reader — matched 30B	qwen/qwen3-coder-30b-a3b-instruct	SiliconFlow (default)	1024 / no
Reader — strong 235B	qwen/qwen3-235b-a22b-2507	DeepInfra (fp8)	1024 / no
Reader — frontier	minimax/minimax-m3	Minimax (default)	16384 / yes
Equiv. judge — main table	deepseek/deepseek-v3.2-exp	Novita (fp8)	4000 / no
Equiv. judge — move block	qwen/qwen3-coder-30b-a3b-instruct	SiliconFlow (default)	— / no
Citation-support rater	deepseek/deepseek-v3.2-exp	Novita (env-set)	4000 / no

A.3 Model, provider and run configuration

Every reader and judge in the RQ1 evaluation runs through the OpenRouter API (base URL `https://openrouter.ai/api/v1`) at temperature 0, with no `top_p` or `seed` set, so determinism depends on temperature alone. Each model is pinned to a single provider with `allow_fallbacks=false`, and where the provider exposes a fixed quantization it is pinned too (Table A.1). Consistent with Section 6.3.2, the main reproduction table (Table 6.6, upper panels) is scored by the frontier `deepseek-v3.2-exp` judge, while the reasoning-move block is scored by a distinct matched-class `qwen3-coder-30b` judge, validated against a blind 40-cell human sample at Cohen’s $\kappa = 0.95$ (raw agreement 0.975). The citation-support rater’s provider is taken from the run environment and is not frozen in the output. Novita is the value used.

Appendix B RQ2

diagnostic-application

reproducibility

This appendix records, verbatim from the implementation and the frozen run artifacts, the material behind Chapter 7: the motif detector rules and their two content queries, the full nine-smell cross-walk, the per-motif prevalence over both corpora, the human-gold annotation protocol and per-motif confusion detail, the relation-layer derivation and its edge inventory, the LLM-judge baseline (prompt, glosses and flat-log rendering), and the model, provider and run configuration. The five behavioral detectors are surfaced to queries through `struggle_signals` and `signal_implicated_actions` of Appendix A.1. Those are cross-referenced without repetition.

B.1 Motif detectors

The six Signal motifs are deterministic rules over the session's ordered ToolAction sequence (`src/ig/signals.py`). The two content-state motifs are Cypher over the Artifact content fingerprints. Each detector emits a Signal whose `severity` equals the number of implicated action spans, so the matched subgraph is the whole explanation. The rules, verbatim in effect:

1	redundancy	an observe/mixed action repeats an earlier action with the
2		same (runner_family, target) and NO intervening mutation
3		invalidated its read-set (observe: the target itself; mixed:
4		any mutation in the session). shell_nav / shell_setup runners
5		excluded. severity = count of redundant actions.
6	repetition	>= 3 consecutive actions with identical (tool, op, target).
7		severity = longest such run.
8	error_burst	>= 3 consecutive failing actions. severity = longest run.
9	edit_test_fail_loop	an edit followed by a failing test, repeated >= 2 times.
10		severity = number of such cycles.
11	regression	a test target that passed, then failed after an intervening
12		edit (the pass is consumed). severity = number of regressed
13		failures.
14	code_rollback	an edit (old, new) that inverts an earlier edit (old', new')
15		on the same path -- new' == old and old' == new (hunk-level,
16		distinct from the whole-file content query below).
17		severity = number of reverting edits.

The two content-state motifs have their own Cypher, keyed here because they do not appear among the RQ1 competency questions of Appendix A.1 (the intra-session `no_net_change` and the cross-session `cross_session_same_state` queries are in that appendix): the whole-file content-return query `code_rollback` and the `signals_overview` rollup of all Signals with their implicated-action counts. Both, like every other query key, are defined in `code/src/ig/queries.py` (CATALOGUE).

Prevalence is computed over the in-memory corpus by re-running the build-time detectors (the reliable path, no Neo4j). The Cypher is the thesis-facing artifact for the same motifs.

Table B.1: The nine interaction smells of Zhang et al., classified for the autonomous SWE-bench setting with the full rationale (verbatim from `src/rq2/catalogue.py`).

Smell		Class	Rationale
Ambiguous Instruction	Instruction	absent	No interactive user issues instructions mid-session; the only instruction is the static issue Intent.
Incomplete Instruction	Instruction	absent	No interactive user turn that could be incomplete in an autonomous session.
Must-Do Omission		absent	Would require semantic tracking of issue constraints; candidate proxy (Intent with no FULFILLS path to a resolved Outcome) deferred as low-precision.
Must-Not Violation		absent	Requires understanding a prohibition; no structural analogue with the current node types.
Signature Mismatch		absent	Needs symbol-level nodes the schema does not carry.
Cross-Turn Inconsistency		detected	The agent’s actions contradict prior state – the edit → test → fail loop.
Partial Functionality Breakdown		detected	New regression detector: a test target passes, then fails after an intervening edit.
Code Rollback		detected	New detector: an edit reverts a prior edit on the same file (inverse old/new swap).
Repetitive Response		detected	Maps to repetition plus the state-aware redundancy detector (mechanistically richer than a text match).

Table B.2: Example evidence identifier for each motif (30B corpus).

Motif	Example evidence id
redundancy	sig:django__django-11490:redundancy:0
cross_session_same_state	tox.ini×6sessions
repetition	sig:django__django-16560:repetition:0
code_rollback	sig:sphinx...-8265:code_rollback:0
error_burst	sig:django__django-13512:error_burst:0
edit_test_fail_loop	sig:sphinx...-10435:edit_test_fail_loop:0
regression	sig:sphinx...-10435:regression:0
no_net_change	—

B.2 Nine-smell cross-walk, full rationale

Table B.1 gives the full per-smell rationale that Table 7.1 compresses. The motif registry that pairs each motif with its intra/cross-session scope, canonical Cypher name, build-time detector and relation edge is the MOTIFS tuple in `code/src/rq2/catalogue.py` and is not reproduced here.

B.3 Prevalence, per-motif detail

Table B.2 names one example evidence identifier for each motif of Table 7.2, from the 30B corpus recomputed by `src.rq2.cli catalogue`.

Table B.3: The thirty stratified sessions (seed 7) behind the human gold, grouped by repository.

django/django	sphinx-doc/sphinx	sympy/sympy
django-10999 e1	sphinx-10435 e2	sympy-12489 e1
django-11490 e1	sphinx-10435 e3	sympy-12489 e3
django-11490 e3	sphinx-10614 e1	sympy-13974 e3
django-12663 e1	sphinx-10614 e2	sympy-14248 e1
django-12663 e2	sphinx-10614 e3	sympy-14976 e1
django-12663 e3	sphinx-11510 e2	sympy-20154 e1
django-13512 e2	sphinx-7757 e1	sympy-24066 e1
django-13512 e3	sphinx-7757 e3	sympy-24066 e3
django-13810 e1	sphinx-8638 e1	sympy-24539 e2
django-13810 e3	—	—
django-14672 e1	—	—
django-14672 e2	—	—

B.4 Human-gold annotation protocol

The gold that validates the five behavioral detectors (Section 7.3) is a two-annotator, blind, stratified label set. The annotation guide given to both annotators (`results/rq2/ANNOTATION_GUIDE.md`), abbreviated:

```

1 Label whether each of five behavioral motifs is present/absent in each of 30
2 sessions, reading only the session's rendered trace (issue text + numbered
3 action list). Independent and blind: label your own sheet, do not look at the
4 other annotator's labels or the detector output. Leave blank if you genuinely
5 cannot decide (blank = contested; excluded from the gold, not guessed).
6
7 repetition (tightened 2026-06-25): the SAME action (same op AND same target)
8   >= 3 times STRICTLY consecutive; any other action between breaks the run.
9 error_burst: >= 3 consecutive failing actions (any actions).
10 redundancy: a read/test repeating an earlier identical one with NO edit between.
11 edit_test_fail_loop: edit -> failing test, MORE THAN ONCE (a single edit->fail
12   is not the loop).
13 regression: the same target [test Y](ok) earlier and [test Y](fail) later, with
14   an edit somewhere between.
15
16 code_rollback and no_net_change are excluded from human labelling: they are
17 exact content/edit comparisons validated by unit tests, not perceptual judgments.
```

The sample is drawn by `src.rq2.cli sample -n 30 -seed 7`, which guarantees at least one positive and one negative session per intra-session motif and then fills deterministically to thirty. The thirty selected (instance, epoch) sessions (Table B.3) give $30 \times 5 = 150$ labeled cells per annotator. Consensus is the gold. Disagreement is recorded contested and excluded. Exactly one cell is contested: `django__django-11490 epoch 3, edit_test_fail_loop`.

B.5 Per-motif confusion detail

Table B.4 gives the full confusion counts behind the precision/recall of Table 7.3, with the named false-alarm and missed sessions (`results/rq2/scores.json`). The two redundancy false positives are the `django-14672/sympy-24066` calls named in

Table B.4: Detector confusion against the two-annotator gold, five behavioral motifs, with the named false-positive (FP) and false-negative (FN) sessions.

Motif	κ	tp	fp	fn	tn	P/R/ F_1	Named FP / FN
error_burst	1.00	4	0	0	26	1.00/1.00/1.00	—
regression	1.00	3	0	0	27	1.00/1.00/1.00	—
redundancy	1.00	28	2	0	0	0.93/1.00/0.97	FP: django-14672 e2, sympy-24066 e3
repetition	1.00	16	0	6	8	1.00/0.73/0.84	FN: django-12663 e2, django-14672 e2, sphinx-7757 e3, sympy-12489 e3, sympy-14976 e1, sympy-24066 e3
edit_test_fail_loop	0.89	3	0	2	24	1.00/0.60/0.75	FN: django-12663 e1, django-12663 e2

Chapter 7. The recall gaps are two django-12663 sessions for edit_test_fail_loop and six sessions for repetition.

B.6 Relation-layer derivation and edge inventory

The three relation edges of Section 5.3.2 (trigger/invalidate/contradict, defined in Chapter 4, Table 4.2) are derived deterministically from the same test/edit/Evidence nodes the motif detectors read (src/ig/relations.py), so an edge and the motif it realizes agree by construction. The rule:

```

1 trigger    : for each i>0, if action[i-1] failed then action[i] --trigger-->
2              action[i-1] (the recovery step follows the failed action).
3 invalidate : a failing test-result Evidence --invalidate--> the most recent edit
4              before it (the edit that test ran against).
5 contradict : a failing test-result Evidence --contradict--> the earlier passing
6              test-result Evidence for the SAME target, when an edit landed
7              between the pass and the fail (the regression pattern); the pass is
8              then consumed.

```

Table B.5 is the edge inventory over the persisted, deduplicated 90-session graphs (src.rq2.cli report). It reproduces the 160/13/3 of Chapter 7 and names the participating sessions. The three contradict edges fall in exactly the three sessions the regression motif fires in, and invalidate spans two further sessions where a failing test overturns an edit without a prior passing run to regress against. A fourth raw contradict triple (a target contradicted twice by the same passing/failing Evidence pair in sphinx-10435 e2) collapses to one edge under the graph's (src, dst, type) deduplication, which is why the edge count is three while the regression severity total is four.

Table B.5: Relation-edge inventory over the persisted 90-session corpus. Edge counts, the number of participating sessions, and (for the two sparse relations) the named sessions.

Relation	Edges	Sess.	Named sessions
trigger	160	75	(the error-burst-dense sessions; not enumerated)
invalidate	13	5	django-16560 e1, sphinx-10435 e2, sphinx-10435 e3, sphinx-7757 e1, sphinx-7757 e3
contradict	3	3	sphinx-10435 e2, sphinx-10435 e3, sphinx-7757 e1

B.7 The LLM-judge baseline: prompt, glosses and rendering

The per-trace judge (`src/rq2/judge.py`) reads one session’s flat action log and returns present/absent for one motif. The prompt is a single template with the motif id and its description substituted per cell, the “bespoke prompt per motif” of Chapter 7:

```

1 You analyse one autonomous coding-agent session for a single "struggle motif".
2
3 Motif "{mid}": {desc}
4
5 Does the session below exhibit this motif? Reply with exactly one JSON object:
6 {"present": true or false, "rationale": "<one short sentence>"}
7
8 # Session
9 {log}

```

The seven {desc} strings substituted for the judgeable motifs:

```

1 edit_test_fail_loop : An edit followed by a failed test, repeated -- the debug-thrash loop.
2 redundancy          : A read/test repeating an earlier identical one with no intervening
   mutation.
3 repetition           : Three or more consecutive identical (tool, op) actions -- a stalled loop.
4 error_burst          : Three or more consecutive failing tool actions.
5 regression           : A test target that passed, then failed after an intervening edit.
6 code_rollback        : A file returns to an earlier content state (Zhang’s Code Rollback).
7 no_net_change        : A file edited more than once whose final content equals its initial --
   wasted churn.

```

The cross-session motif `cross_session_same_state` is never sent to the judge: it is recorded `unjudgeable` because a single-trajectory view categorically cannot decide it (the cross-session-reach evidence). The {log} is the blind flat rendering of `src/rq2/render.py`, issue text plus a numbered action list, each line `N. [op target] (ok|fail)` with any Evidence summary trimmed to 160 characters, carrying no Signal or verdict, so it cannot anchor the judge to the detector’s output. The thirty rendered contexts are frozen under `results/rq2/context/`. One excerpt (`sphinx-doc__sphinx-10435-e2.md`):

```

1 20. [edit /testbed/sphinx/writers/latex.py] (ok)
2 21. [edit /testbed/sphinx/writers/latex.py] (ok)
3 22. [test tests.test_build_latex.test_latex_code_role] (ok)
4 25. [edit /testbed/sphinx/writers/latex.py] (ok)
5 28. [edit /testbed/sphinx/writers/latex.py] (ok)
6 30. [test tests.test_build_latex.test_latex_code_role] (fail)

```

Table B.6: Run configuration for the RQ2 LLM-judge baseline and the structural detectors.

Setting	Value
Judge model	deepseek/deepseek-v3.2-exp (via OpenRouter)
Provider pin	none (default routing; OPENROUTER_PROVIDER override), not frozen in the artifact
Decoding	temperature 0, max_tokens 2000, k=1
Cells	720 total, 630 judgeable, 90 unjudgeable (cross-session)
Concurrency	12
Detectors	deterministic rules over the in-memory corpus, no model
Corpora	ig_out/agent (30B, 90 sessions); ig_out/agent_235b/agent (235B, 90 sessions)

Table B.7: The frozen RQ2 artifacts (under code/results/rq2/) behind every reported number.

Artifact	Contents
judge-verdicts.json	720 judge verdicts (per-cell verdict, rationale, latency_s)
scores.json, gold.csv	per-motif $\kappa/P/R/F_1$; consensus gold + the contested cell
relations.json	relation-edge inventory (160/13/3 + session lists)
prevalence.json,	per-motif prevalence, 30B and 235B
235b/prevalence.json	
sample.json, ANNOTATION_	seed-7 stratified sample; annotator instructions
GUIDE.md	
labels-student.csv,	the two blind label sheets
labels-supervisor.csv	
context/*.md	the thirty blind rendered traces

```

7 | 33. [edit /testbed/sphinx/writers/latex.py] (ok)
8 | 34. [test tests.test_build_latex.test_latex_code_role] (fail)

```

B.8 Model, provider, run and log configuration

The judge runs deepseek/deepseek-v3.2-exp through OpenRouter at temperature 0, one verdict per cell ($k=1$), across $90 \times 8 = 720$ cells of which 630 are judgeable (the ninety cross_session_same_state cells are recorded unjudgeable). Consistent with Appendix A.3, the provider for this model is *not* pinned in the RQ2 harness (OpenRouterClient pins a provider only for the corpus model and leaves ablation models to OpenRouter’s default routing unless OPENROUTER_PROVIDER is set), and the verdict artifact records no provider. RQ1 observed Novita (fp8) for the same model. Table B.6 lists the configuration and Table B.7 the frozen artifacts.

The pipeline that regenerates every artifact:

```

1 | python -m src.rq2.cli sample --corpus ig_out/agent --n 30 --seed 7
2 | python -m src.rq2.cli score --corpus ig_out/agent \

```

```

3 | --labels results/rq2/labels-student.csv results/rq2/labels-supervisor.csv
4 | python -m src.rq2.cli catalogue --corpus ig_out/agent
5 | python -m src.rq2.cli --out results/rq2/235b catalogue --corpus ig_out/agent_235b/agent
6 | python -m src.rq2.cli judge      --corpus ig_out/agent \
7 |   --model openrouter/deepseek/deepseek-v3.2-exp
8 | python -m src.rq2.cli compare  --corpus ig_out/agent --verdicts results/rq2/judge-verdicts.
9 |   json
  | python -m src.rq2.cli report   --corpus ig_out/agent --verdicts results/rq2/judge-verdicts.
  |   json

```

On timing (Section 7.6): the 6.4 ms structural figure is a single `time.monotonic()` span over `structural_verdicts` on one machine and one run, reported as an indicative wall-clock. Its load-bearing claim is length-invariance by construction, the exact millisecond incidental. The $\approx 1,597$ s LLM figure is the mean per-cell latency (≈ 2.5 s) times the 630 judgeable cells. The per-cell `latency_s` values are stored in `judge-verdicts.json`.

Appendix C RQ3

memory-application reproducibility

This appendix records, verbatim from the implementation and the frozen run artifacts, the material behind the feasibility pilot of Chapter 8: the arm prompts, the retrieval tools and their guards, the run configuration and log identifiers, the full result tables behind the figures and the move-classifier reliability the typed-layer analysis depends on.

C.1 The three memory arms

The arms share one investigate-and-apply preamble and differ only in a single synthesis clause substituted for {synthesis}. The shared preamble, mounted alongside `bash` and `text_editor`:

```
1 You have read-only memory of this repository's prior fix attempts, as tools
2 alongside 'bash' and 'text_editor'. Before you dive in, mine it for lessons:
3 - Start with 'repo_memory_summary': a one-call, repo-wide rollup -- the
4   working test runner, how attempts recovered from failed tests, and the
5   recurring STRUGGLE MOTIFS (each with a plain-language 'means'). Those
6   motifs are what prior attempts DID WRONG here -- patterns to AVOID.
7 - If a prior attempt looks relevant, drill in with
8   'attempt_query(instance_id, query)' (use an id from 'list_prior_attempts');
9   'failures_and_recovery' shows what worked after a failure, 'move_motifs'
10  and 'motif_causes' show the shape of the trouble. Avoid 'action_log'.
11 - Be purposeful: a handful of queries is usually plenty. {synthesis} Consult
12   the memory again if a test fails.
13 Investigate as much as you genuinely need, but do not start blind and do not
14 spend the whole task exploring memory.
```

The three synthesis clauses, which are the only text that differs across arms:

```
1 free : Then use what you learned however is most helpful as you fix the bug.
2
3 expel : Then write down the concrete DO/AVOID rules grounded in what you found
4         -- as many as the evidence supports, no fixed number -- contrasting
5         what worked with what went wrong, and FOLLOW them as you fix the bug.
6
7 guide : Then write state-conditioned guidelines -- each as 'When [situation],
8         do/avoid X' tied to specific prior-attempt evidence, as many as the
9         evidence supports -- and FOLLOW them as you fix the bug.
```

C.2 Retrieval tools and glosses

Three tools are mounted. `repo_memory_summary` returns a repository-wide rollup (the prior-attempt count, the most frequent working test runner, the glossed move motifs and signals with per-motif attempt counts and test-runner recovery substitutions), `list_prior_attempts` returns each prior attempt's issue summary ranked most-similar-first, and `attempt_query(instance_id, query)` runs one named graph query on a chosen prior attempt. The twelve query names `attempt_query` accepts

Table C.1: The thirty frozen SWE-bench Verified instances, ten per repository.

django/django	sphinx-doc/sphinx	sympy/sympy
django__django-10999	sphinx-doc__sphinx-7440	sympy__sympy-12489
django__django-11490	sphinx-doc__sphinx-7757	sympy__sympy-13798
django__django-12663	sphinx-doc__sphinx-8265	sympy__sympy-13974
django__django-13512	sphinx-doc__sphinx-8638	sympy__sympy-14248
django__django-13810	sphinx-doc__sphinx-9229	sympy__sympy-14976
django__django-14672	sphinx-doc__sphinx-9258	sympy__sympy-18189
django__django-15278	sphinx-doc__sphinx-10435	sympy__sympy-20154
django__django-15629	sphinx-doc__sphinx-10466	sympy__sympy-24066
django__django-16560	sphinx-doc__sphinx-10614	sympy__sympy-24539
django__django-16662	sphinx-doc__sphinx-11510	sympy__sympy-24562

are the AGENT_QUERIES tuple in `code/src/ig/mcp_server.py`. Their Cypher is keyed as in Appendix A.1.

The summary attaches a plain-language gloss to every motif and signal, because a bare label is not self-describing. The gloss strings are the `_MOVE_GLOSS` and `_SIGNAL_GLOSS` maps in `code/src/ig/mcp_server.py`.

C.3 Leave-one-out construction and guard enforcement

Reuse is repo-scoped leave-one-out. The corpus is thirty SWE-bench Verified instances, ten from each of `django/django`, `sphinx-doc/sphinx` and `sympy/sympy`, run at three epochs (Table C.1). Each target retrieves only its own repository’s other nine instances at three epochs each, so exactly twenty-seven prior attempt trajectories (nine instances $\times$ three epochs) are retrievable per target and none from another repository or from the target’s own instance. The `repo_memory_summary` rollup consolidates these to the nine sibling instances, which is why its `n_prior_attempts` reads nine. The no-memory baseline run doubles as both the S0 comparator and the retrieval corpus.

Both the leave-one-out boundary and the per-task budget are enforced by the harness, out of the agent’s reach. The evaluation launches one memory server per sample with the current instance id and the call budget in its environment (`IG_CURRENT_INSTANCE`, `IG_MAX_CALLS=10`). The server refuses any query that names the current instance or an instance outside its repository, and a per-process counter caps retrieval at ten calls. An early pilot episode reused a cached server subprocess across samples, so a later task inherited an earlier task’s current-instance environment. The cache key was made unique per sample and a re-run confirmed zero self-retrieval.

C.4 Model, provider and run configuration

Every arm and the baseline use one instruction-tuned model pinned to one provider and quantization for reproducibility (Table C.2). Generation is left at the Inspect and

Table C.2: Run configuration for the RQ3 pilot.

Setting	Value
Model	qwen/qwen3-235b-a22b-2507 (instruct, non-thinking)
Provider pin	DeepInfra only, allow_fallbacks=false, fp8
Epochs per instance	3 (repeat seeds)
Message limit	200 per sample
Time limit	1800 s per sample
Retrieval budget	10 calls per task
Fail-on-error tolerance	0.2 of samples
Retry on error	1
Sample concurrency	4
Bootstrap	10,000 resamples, seed 20260517, paired over the 30 instances, 2.5/97.5 percentile CI

Table C.3: The frozen evaluation logs behind every reported number.

Arm	Log identifier (under code/logs/)
baseline	inspect_react/2026-07-01T17-37-50-..._ 2uzbisCFzmgI4oZ4rsiLYi.eval
free	ig_agentic_free/2026-07-01T23-30-58-..._ XKL32K576MQtrw7X6aMGkT.eval
expel	ig_agentic_expel/2026-07-02T00-56-03-..._ 5VzMfLUHasDJABtQ7HdQ78.eval
guide	ig_agentic_guide/2026-07-02T02-43-00-..._ hXE7NGQ5nENpibbRg6jxDw.eval

provider defaults, and the three epochs are repeat seeds. The three completed arm logs and the reused baseline are named in Table C.3, and the per-arm pipeline is:

```

1 # lift + enrich one arm's trajectories into a corpus
2 python -m src.ig.build <arm-spans.json> --eval-log <arm.eval> \
3   --arm agent --out-dir ig_out/rq3_arms/<arm>
4 python -m src.ig.enrich.move_type --corpus ig_out/rq3_arms/<arm>/agent
5 python -m src.ig.enrich.move_motif --corpus ig_out/rq3_arms/<arm>/agent
6 # run one arm (baseline uses the 'agent' arm, no memory)
7 python -m src.eval.cli ig_agentic-<arm> \
8   --model openrouter/qwen/qwen3-235b-a22b-2507 --epochs 3

```

C.5 Full process-metric results

Table C.4 gives the per-attempt means of the two process metrics the main table omits, and Table C.5 gives the paired bootstrap difference from the baseline for every metric. These are the numeric backbone of Figure 8.1.

C.6 Typed motif and signal deltas

Table C.6 gives the raw corpus counts and the paired delta for every motif and signal type, behind Figure 8.3. Every interval spans zero except guide's repeated_

Table C.4: Per-attempt redundant-tool-call count and wall-clock seconds, ninety attempts per arm.

Arm	Redund. calls	Wall (s)
baseline	3.6	306
free	2.2	206
expel	3.8	276
guide	1.6	218

Table C.5: Paired difference from the baseline (arm – S0) with 95% bootstrap CI, resampled over the thirty instances. Intervals excluding zero in bold.

Metric	free	expel	guide
Resolved	–0.011 [–0.078, +0.056]	+0.011 [–0.067, +0.089]	+0.033 [–0.056, +0.122]
Redundancy rate	–0.030 [–0.054, –0.006]	–0.014 [–0.038, +0.010]	–0.037 [–0.062, –0.014]
Redundant calls	–1.44 [–2.97, –0.10]	+0.16 [–1.50, +1.94]	–2.00 [–3.72, –0.41]
Turns	+0.79 [–5.12, +6.31]	+8.82 [+2.41, +15.68]	–0.88 [–7.57, +5.77]
Tokens	+4305 [+1073, +7505]	+6368 [+2436, +10503]	+2898 [–1432, +7314]

generation. The three signals marked with a dagger are single-digit corpus events, so their nulls reflect absent data rather than a tested absence.

C.7 Retrieval intensity and outcome

Table C.7 is the retrieval-by-outcome crosstab fixed in Chapter 5: the two hundred seventy memory-arm attempts, split at the median retrieval-call count of two, cross-tabulated against the resolved outcome, with the mean redundancy rate per bin. Retrieval intensity is the number of `repo_memory_summary`, `list_prior_attempts` and `attempt_query` calls in the attempt (per-arm means free 2.00, expel 2.54, guide 2.26).

Heavier retrieval tracks a lower resolution rate (0.11 against 0.28) and a higher redundancy rate (0.090 against 0.049). The association runs opposite to a treatment dose-response: retrieval intensity is endogenous, since a struggling agent both queries the memory more and repeats more actions, so the extra calls mark hard attempts rather than cause their outcome. The crosstab therefore cannot attribute the between-arm redundancy reduction to memory content, an attribution a sham-memory control in the summative study would isolate.

C.8 Outcome churn is run-to-run noise

The flat net resolution rate hides per-instance flips against the baseline: eleven of the thirty instances flip between resolved and unresolved for `free`, seven for `expel` and ten for `guide`. A per-instance forensic comparison of the flipped trajectories attributes these flips to run-to-run stochasticity rather than the memory, on four grounds.

First, the three memory arms retrieve byte-identical content for a given instance

Table C.6: Typed struggle-motif and signal layer: raw corpus counts (baseline / free / expel / guide) and the paired difference from the baseline with 95% CI. Interval excluding zero in bold.

Type (count b/f/e/g)	free	expel	guide
<i>Move motifs</i>			
repeated_generation 29/27/27/43	-0.02 [-0.14, +0.10]	-0.02 [-0.13, +0.09]	+0.16 [+0.04 , +0.26]
missing_verify 48/53/50/53	+0.06 [-0.11, +0.22]	+0.02 [-0.14, +0.19]	+0.06 [-0.04, +0.16]
relocalization_loop 11/10/12/12	-0.01 [-0.09, +0.08]	+0.01 [-0.07, +0.10]	+0.01 [-0.07, +0.09]
oscillating_test_fix 10/7/15/8	-0.03 [-0.11, +0.03]	+0.06 [-0.02, +0.14]	-0.02 [-0.11, +0.07]
skipped_reproduce 47/55/52/55	+0.09 [-0.08, +0.26]	+0.06 [-0.10, +0.21]	+0.09 [-0.04, +0.21]
investigation_heavy 17/25/16/23	+0.09 [-0.04, +0.21]	-0.01 [-0.11, +0.09]	+0.07 [-0.07, +0.20]
<i>Signals</i>			
redundancy 71/78/79/75	+0.08 [-0.00, +0.17]	+0.09 [+0.00, +0.18]	+0.04 [-0.03, +0.13]
repetition 48/53/53/44	+0.06 [-0.10, +0.22]	+0.06 [-0.08, +0.20]	-0.04 [-0.17, +0.09]
edit_test_fail_loop [†] 2/3/4/1	+0.01 [-0.02, +0.04]	+0.02 [-0.02, +0.08]	-0.01 [-0.06, +0.02]
error_burst [†] 6/7/5/2	+0.01 [-0.06, +0.08]	-0.01 [-0.09, +0.06]	-0.04 [-0.11, +0.01]
code_rollback [†] 7/3/9/7	-0.04 [-0.13, +0.03]	+0.02 [-0.09, +0.12]	-0.00 [-0.11, +0.10]
Motif total	+0.17 [-0.21, +0.54]	+0.11 [-0.27, +0.49]	+0.36 [-0.00, +0.70]
Signal total	+0.11 [-0.16, +0.38]	+0.18 [-0.09, +0.49]	-0.06 [-0.30, +0.20]

Table C.7: Retrieval-by-outcome crosstab, pooled over the three memory arms (two hundred seventy attempts), split at the median retrieval-call count.

Retrieval intensity	Resolved	Unresolved	Resolution rate	Mean redund. rate
Low (≤ 2 calls)	65	167	0.280	0.049
High (> 2 calls)	4	34	0.105	0.090

and differ only in the prompt synthesis clause, yet several instances flip in opposite directions across arms: `django-15278` and `sphinx-10466` fall under `free` but rise under `expel`, `sympy-14976` rises under `free` but falls under `expel` and `guide`, and `sympy-18189` falls under `free` but rises under `guide`. Identical content cannot move an instance up under one arm and down under another. Second, the retrieved priors are irrelevant: the highest issue-text similarity between any retrieved prior and the current task across all twelve flipped instances is 0.165, below a 0.2 relevance floor, and most sit near 0.05. Third, the deep `attempt_query` tool rarely fired on flipped instances, and where it did it returned no rows or the arm that queried most finished worst (`sphinx-10466`). Fourth, every flip sits on a borderline baseline instance, with one or two of three epochs resolved, and the same source file is edited with or without the memory. The churn is therefore epoch-level pass-or-fail variance on borderline tasks, so the outcome null holds at the instance level as well as in aggregate.

C.9 Move-classifier reliability

The typed-layer result of Section 8.4 depends on the move layer being trustworthy, so its reliability is stated here. The *move-type* classifier that labels each sub-intent was validated against a human coder on a stratified sample: on the representative Part A cells (ninety cells, ten per label) the deployed v3 classifier agrees at Cohen’s $\kappa = 0.74$, and an all-cells estimate that oversamples the fifty hardest disagreement cells gives a pessimistic floor of $\kappa = 0.59$ over the full 140-cell sample. A competing v4 prompt scored 0.72 on the same cells and lost the head-to-head, so v3 is locked. A second annotator independently re-labeled the representative cells, and the two coders agree at an inter-annotator $\kappa = 0.76$ (raw 0.79 over 89 both-coded cells), so the classifier fidelity now rests on a gold whose reliability is confirmed between two humans.

Two honest limits follow. The agreement is measured on the *move-type* labels, whereas the six MoveMotifs are a deterministic rule layer over those labels. The diagnostic evaluation validates those rules against a separate two-annotator gold (Cohen’s $\kappa = 0.861$, Chapter 7), but that gold holds the classified moves fixed, so the motifs still inherit the classifier’s error end to end and no gold yet certifies the combined classifier-and-rule layer. That end-to-end validation is genuinely owed, and it bounds the strength of the typed-layer null in Section 8.4: the null holds against a move layer of documented but imperfect fidelity.